Sri Lanka Institute of Information Technology



SE3090 – Software Engineering Frameworks

Year 3, Semester 1 - 2026

Final Assignment Submission

Y3.S

Group ID: 2026 – AI – 34

Name	ID
Rajapaksha R.A.J.K	IT24101236
Matharaarachchi D.C.M	IT24101252
Hasanga H N S	IT24100962
De Silva L.J.	IT24101297

Table of Contents

1.1 Business Problem 3

1.2 Project Objectives 3

1.3 Scope 3

2. Requirements and User Roles 4

2.1 User Roles 4

2.2 Functional Requirements 4

2.3 Non-Functional Requirements 5

3. System Architecture 5

3.1 Technology Stack 5

3.2 High-Level Architecture Diagram 6

3.3 Repository Structure 6

4. Database Design and ER Model 7

4.1 Core Entities 7

4.2 ER Diagram 8

4.3 Data Integrity and Indexes 9

5. ASP.NET Core API Design 10

5.1 Major API Areas 10

5.2 API Documentation Evidence 11

6. React Web Application Design 11

6.1 Main Web Workflows 11

6.2 Design System and Quality 12

7. Flutter Mobile Application Design 12

8. Agentic AI Architecture and Evaluation 12

8.1 Specialized Agents 13

8.2 Workflow and Human Approval 13

8.3 Controls, Validation and Security 14

8.4 Agent Evaluation Evidence 15

9. Technical Report 16

9.1 Backend and Integration Approach 16
9.2 Authentication and Authorization 17
9.3 Error Handling and Validation 17
9.4 Cross-Platform Integration 17
9.5 Third-Party Integration 18
10. Software Testing Report 19
10.1 Testing Strategy 19
10.2 Test Matrix 19
10.3 Defects and Resolutions 20
11. Performance Report 20
12. Deployment Report 21
12.1 Deployment Summary 21
12.2 Required Environment Variables 21
12.3 Startup Instructions (local) 22
12.4 Test Accounts 22
13. Architecture Decision Records (ADRs) 23
13.1 ADR-01: React State Management (docs/adr/0002) 23
13.2 ADR-02: Flutter State Management 23
13.3 ADR-03: Agentic AI Framework and Orchestration 24
13.4 ADR-04: Agent Workflow State Schema (docs/adr/0003) 24
13.5 ADR-05: Cloud Deployment Platform 25
13.6 ADR-06: Payments and Payment Gateway Scope 25
14. Security Considerations 25
15. Consolidated Group AI Usage Declaration 26
16. Individual Report Sections 26
16.1 Member 1 - Tour Package & Booking Management + Coordinator / Planning Agent 26
16.1.1 Contribution Statement 26
16.1.2 Owned Technical Scope 26
16.1.3 Coordinator Agent Contract 27

16.1.4 Key Implementation Evidence 27
16.1.5 Screenshots / Demonstration Evidence 28
16.1.6 Challenges and Learning 36
16.1.7 Individual AI Usage Log 36
16.1.8 Individual AI Reflection 37
16.1.9 Individual Declaration 38
16.2 Member 2 - Guide & Itinerary Management + Guide Matching Agent 38
16.2.1 Contribution Statement 38
16.2.2 Owned Technical Scope 38
16.2.3 Guide Matching Agent Contract 39
16.2.4 Matching Considerations 39
16.2.5 Key Implementation Evidence 40
16.2.6 Screenshots / Demonstration Evidence 41
16.2.7 Challenges and Learning 43
16.2.8 Individual AI Usage Log 43
16.2.9 Individual AI Reflection (Approximately One Page) 43
16.2.10 Individual Declaration 43
16.3 Member 3 - Fleet & Transport Management + Fleet & Capacity Agent 43
16.3.1 Contribution Statement 43
16.3.2 Owned Technical Scope 43
16.3.3 Fleet & Capacity Agent Contract 43
16.3.4 Key Implementation Evidence 43
16.3.5 Screenshots / Demonstration Evidence 43
16.3.6 Challenges and Learning 43
16.3.7 Individual AI Usage Log 43
16.3.8 Individual AI Reflection 43
16.3.9 Individual Declaration - 43
16.4 Member 4 - Payments, Reviews & Reporting + Pricing & Validation Agent 43
16.4.1 Contribution Statement 43

16.4.2 Owned Technical Scope 43
16.4.3 Pricing & Validation Agent Contract 43
16.4.4 Key Implementation Evidence 43
16.4.5 Main Functional Implementation 43
16.4.6 Screenshots / Demonstration Evidence 43
16.4.7 Challenges and Learning 43
16.4.8 Individual AI Usage Log 43
16.4.9 Individual AI Reflection 43
16.4.10 Individual Declaration 43
17. References 43
18. Appendices 43
Appendix A - Final URLs and Evaluator Access 43
Appendix B - Screenshots and Evidence Index 43
Appendix C - Installation Quick Reference 43
Appendix D - Git / Pull Request Evidence 43

1. Project Overview and Scope

TrailWise is a smart tour management and travel planning system for Sri Lankan tour operators. It brings traveler booking, tour package management, operational coordination, fleet and guide management, itinerary handling and a controlled Agentic AI workflow into one platform. The system combines a React web application, a Flutter mobile application, an ASP.NET Core 8 Web API and a PostgreSQL database. A multi-agent workflow checks every booking request and pauses high-impact decisions for human approval.

1.1 Business Problem

Local tour operators coordinate bookings, guide assignments, vehicle allocation and itinerary changes manually, through phone calls, spreadsheets and disconnected messaging. A traveler requesting a trip waits while staff manually check which guides speak the right language and are free on the right dates, which vehicles can fit the group, and whether the plan fits the traveler's budget. This causes double-booked guides, vehicle conflicts, last-minute itinerary problems and poor communication once a trip is underway.

TrailWise takes over from the moment a traveler requests a tour: it matches a suitable guide, checks vehicle capacity and air-conditioning requirements, prices the trip with its tier, and escalates anything outside standard policy to an Operations Manager for a recorded decision.

1.2 Project Objectives

- Provide one integrated platform for package browsing, booking, operational coordination, fleet and guide assignment, and itinerary management.
- Use one shared ASP.NET Core API and PostgreSQL database for both the React and Flutter clients.
- Apply role-based access control for Traveler, Tour Guide, Operations Manager, Fleet Coordinator, Driver and Admin users.
- Implement a controlled Agentic AI workflow that plans, delegates to distinct agents, uses allow-listed tools, validates results deterministically, persists state and requests human approval for high-impact actions.
- Support automated testing, continuous integration and reproducible setup.

1.3 Scope

In Scope (implemented)	Out of Scope / Team Decisions
Public tour explorer and package detail pages (browse without login)	Payment gateway sandbox: not implemented (team decision); payments use the existing bank-slip upload and staff verification flow (ADR-06)
Authentication, registration and role-based authorization	Booking add-on selection UI: not implemented (team decision); BookingAddOn entity kept as designed
Package and tier management (class, food, AC, price)	Weather API and route maps in itineraries (time constraint)
Booking requests and the six-step coordinator workflow	Flutter push notifications, ID/passport upload and live tour location (time constraint)

Three human-approval types: large group, budget override, late-cancellation refund exception	Currency conversion and multi-currency pricing (prices are in USD)
Guide availability, itinerary management, fleet and driver management	Direct client access to PostgreSQL or to AI services
Bank-slip payments, reviews, discounts, reports and audit log	
Agent workflow monitor and Ops dashboard	

2. Requirements and User Roles

2.1 User Roles

Role	Main Responsibilities	Client
Traveler	Browse packages and tiers, submit booking requests with dates, group size, budget, language and special requests, upload payment slips, track booking status, cancel, and review completed tours.	Web + Flutter
Tour Guide	Maintain availability, view assigned tours, edit itineraries, start and end tours, record attendance/completion and tour notes.	Web + Flutter
Operations Manager	Approve, reject or request revision for high-impact bookings (three approval types), monitor agent workflows, manage packages, tiers and discounts, verify payments, view reports and handle support tickets.	Web
Fleet Coordinator	Manage vehicles (capacity, AC, maintenance status) and drivers, review and adjust vehicle assignments, assign guides manually when the workflow cannot.	Web
Driver	View own assigned trips, vehicles, travelers' contact and itinerary (read only).	Web + Flutter
Admin	Cross-cutting: create and remove staff accounts, manage packages, access Ops functions.	Web

2.2 Functional Requirements

1. Authenticate users with JWT and enforce role-based permissions on every protected endpoint.
2. Allow anyone to browse, search, filter and sort tour packages; allow travelers to request bookings for a chosen tier.
3. Validate booking requests server-side (group size within package maximum, start date not in the past and within 365 days, end after start, budget above zero).
4. Run the coordinator workflow on every new booking: extract preferences, match a guide, check vehicle and driver availability, calculate the tier-aware price, validate deterministically and summarize.

5. Auto-confirm standard bookings; pause large-group (more than 10 travelers) and over-budget (total above budget x group x 1.15) bookings for Operations Manager approval.
6. Route traveler cancellations within 7 days of the tour (with an approved payment) to the Operations Manager as a refund exception.
7. Let the Operations Manager approve, reject (with note) or request revision (with note) for each pending approval.
8. Persist workflow runs, step logs, tool calls, validation results, approval decisions and audit entries.
9. Manage guides, availability, itineraries, vehicles, drivers, payments, discounts, reviews and support tickets.
10. Provide reports: occupancy, revenue by package, guide utilisation and an exportable audit log.

2.3 Non-Functional Requirements

- **Security:** JWT, role-based authorization, hashed passwords, rate limiting, input validation, secret redaction and safe failure.
- **Reliability:** deterministic business rules in code (not LLM judgement), transactional confirmation, safe failure to NeedsManualReview.
- **Maintainability:** layered architecture (Controllers, Services, Domain, Infrastructure), DTOs, dependency injection and documented ADRs.
- **Usability and accessibility:** shared design system with light and dark themes, WCAG AA contrast, responsive layouts, loading, empty and error states.
- **Traceability:** audit log, workflow run history with tool calls, Git history and AI usage logs.

3. System Architecture

TrailWise uses a layered full-stack architecture. React and Flutter communicate only with the ASP.NET Core Web API. The API contains controllers, application services, the Agentic AI coordinator and its agents, validation and EF Core data access to PostgreSQL. The agents run in-process inside the API, so clients never call AI services directly. Optional LLM calls (Groq) and third-party services (OpenStreetMap Nominatim, Notify.lk SMS) are reached only from the backend.

3.1 Technology Stack

Layer	Technology	Purpose
Web	React 19, TypeScript, Vite, React Router 7, Tailwind CSS 4, Zustand, react-hook-form + zod, lucide-react	Operations console, public explorer, role portals
Mobile	Flutter / Dart, Provider, flutter_secure_storage, file_picker	Traveler, Tour Guide and Driver app
Backend	ASP.NET Core 8 Web API, Swagger/OpenAPI	REST API, authentication, business rules, orchestration
Data access	Entity Framework Core + Npgsql	ORM, migrations, transactions

Database	PostgreSQL 16	Relational storage including agent workflow state
Agentic AI	Custom C# orchestration (CoordinatorAgentService + four agents); optional Groq LLM (openai/gpt-oss-120b) for preference extraction and summaries	Booking planning, matching, pricing, validation and approval
Third-party	OpenStreetMap Nominatim, Notify.lk SMS, Groq	Location search/geocoding, booking SMS, LLM steps
DevOps	Git, GitHub, GitHub Actions, Docker Compose	Collaboration, CI, reproducible local environment

3.2 High-Level Architecture Diagram

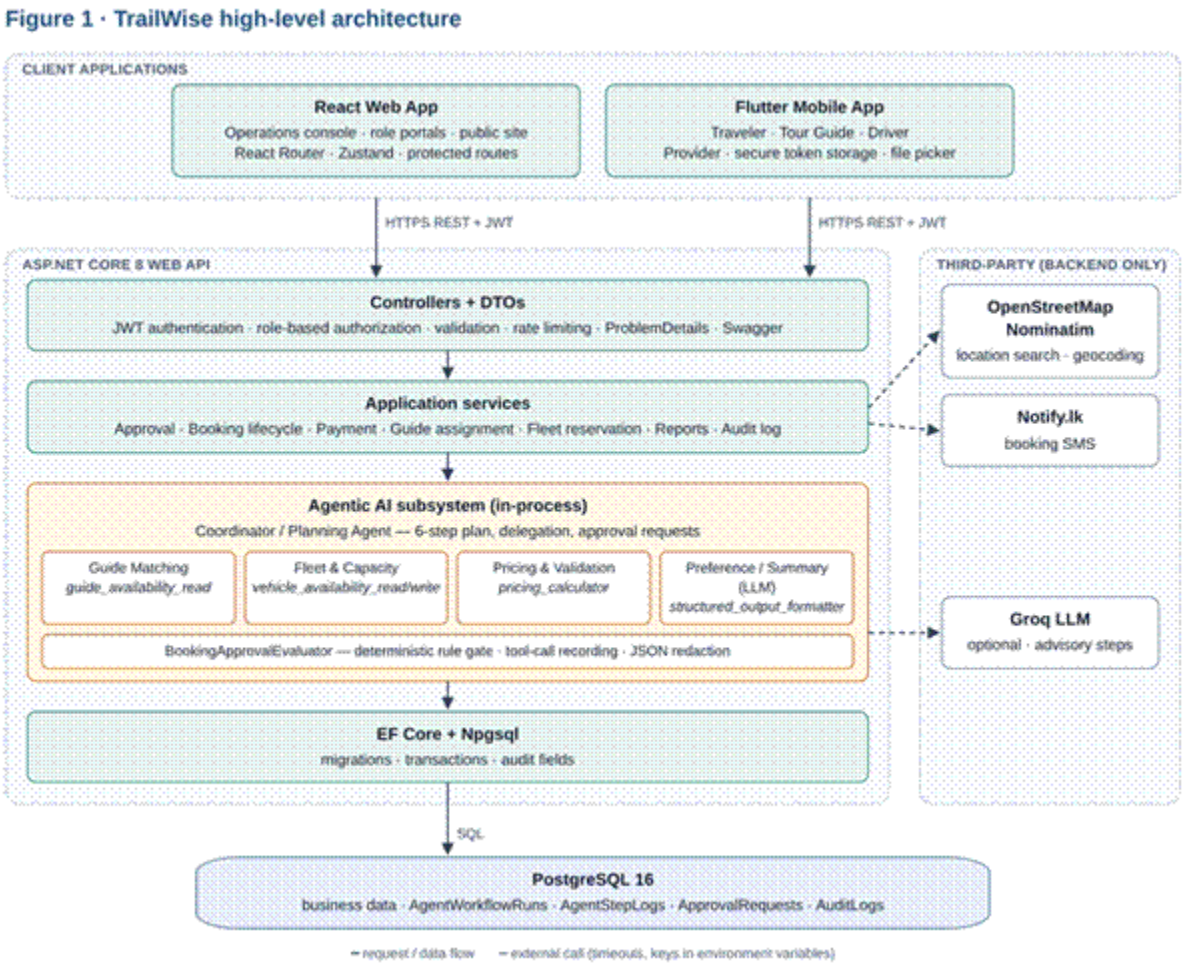


Figure 1. High-level architecture: clients, API layers, in-process agents, database and third-party services

3.3 Repository Structure

Path	Description
backend/src/TrailWise.Api	Controllers, DTO contracts, Program.cs (auth, CORS, rate limiting, Swagger, error handling)
backend/src/TrailWise.Domain	Entities and enums (BookingStatus, UserRole, ApprovalType ...)
backend/src/TrailWise.Infrastructure	Services, agents (Agents/Coordinator, GuideMatching, FleetCapacity, PricingValidation, PreferenceExtraction, ProposalSummary), EF Core DbContext, configurations, migrations (Persistence/db), seeder
backend/tests/TrailWise.Api.Tests	xUnit unit and API integration tests (WebApplicationFactory, in-memory provider)
frontend-web/	React web application (src/api, auth, components/ui design system, pages per role, stores)
frontend-mobile/	Flutter application (lib/auth, bookings, guides, drivers, packages, support)
docs/	ADRs (docs/adr), coordinator agent description, design document, improvement plan
docker-compose.yml, .env.example	Local db + backend + frontend-web services and documented environment variables
.github/workflows/	backend-ci.yml, frontend-web-ci.yml, frontend-mobile-ci.yml

4. Database Design and ER Model

The PostgreSQL schema is managed with EF Core migrations (backend/src/TrailWise.Infrastructure/Persistence/db). All entities inherit CreatedAt and UpdatedAt audit fields. Enums are stored as strings for readability.

4.1 Core Entities

Entity	Purpose	Owner
Users	Identity, hashed password, role, contact number	Shared
TourPackages, PackageTiers, PackageLocations	Package templates; tiers by class (First/Second/Normal), food, AC and price; locations with optional coordinates	Member 1
Bookings, BookingAddOns	Booking request (dates, group size, budget, language, special requests, status, revision note, tour lifecycle); optional extras	Member 1

AgentWorkflowRuns, AgentStepLogs	Persisted workflow state: objective, plan JSON, status, summary; per-step input, output, tool calls, validation result, duration	Member 1 (shared)
ApprovalRequests	Pending/decided human approvals (type, status, previous booking status, requester note, decision note, decided by/at)	Member 3
Guides, GuideAvailability, ItinerarySteps	Guide profiles (languages, specializations), per-date availability and assignment, day-by-day itinerary	Member 2
Vehicles, Drivers, VehicleAssignments	Fleet with capacity, AC, maintenance status; drivers linked to user accounts; date-range assignments	Member 3
Payments, Discounts, Reviews, AuditLogs	Bank-slip payments and verification, group discounts, reviews, full audit trail	Member 4
SupportTickets, SupportMessages	Traveler support conversations	Member 4 (Shared)

4.2 ER Diagram

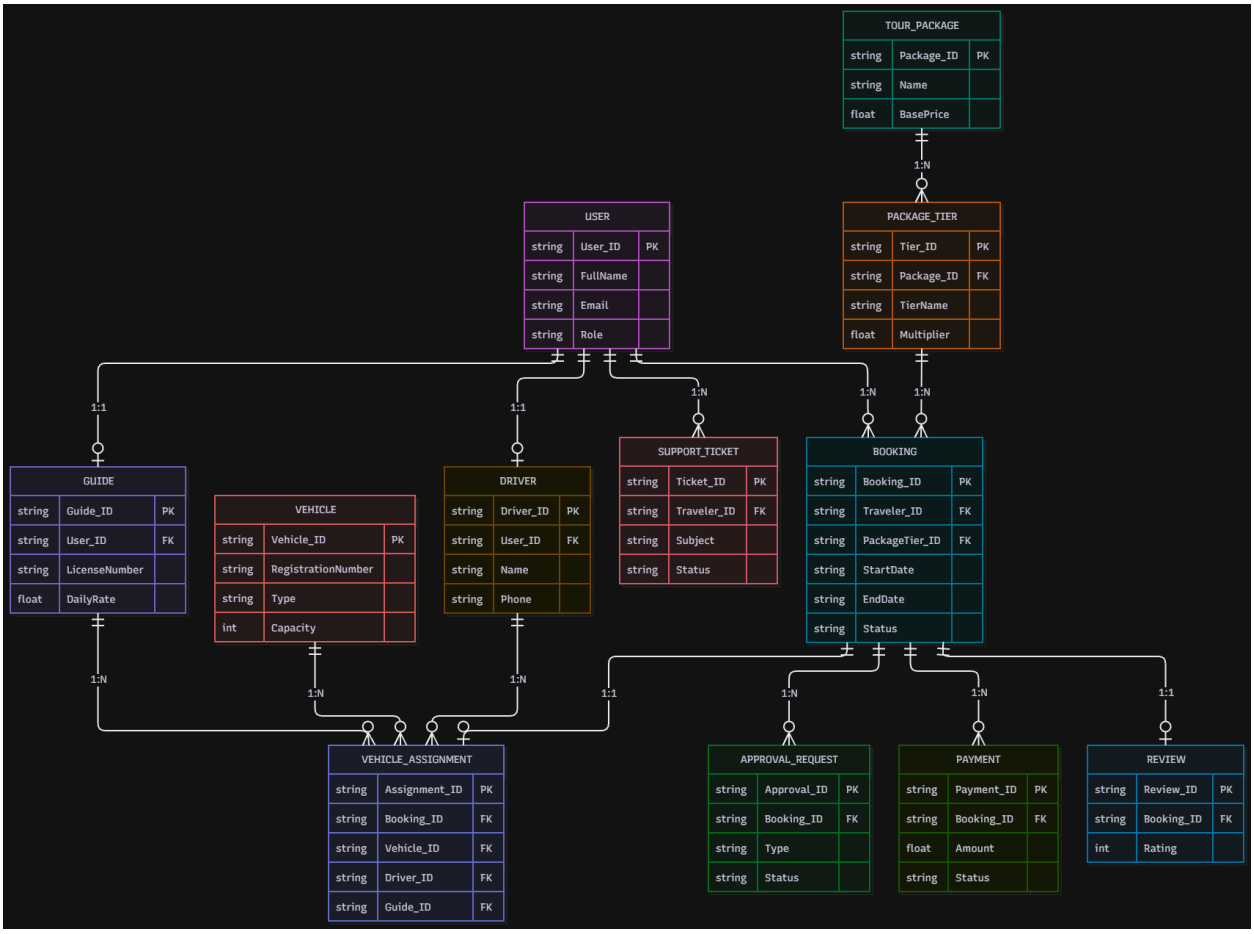


Figure 2. ER diagram (key columns and relationships; see Section 4.1 for all entities)

4.3 Data Integrity and Indexes

Constraint / Index	Purpose
Users.Email unique	One account per email
GuideAvailability (GuideId, Date) unique	Prevents duplicate availability rows and double-booking a guide on a date
VehicleAssignments (VehicleId, StartDate)	Fast overlap and conflict checks for vehicle reservations
Vehicles.RegistrationNumber unique	No duplicate registrations
Drivers.UserId unique; FK ON DELETE SET NULL	One driver record per login; deleting a login unlinks rather than breaking the driver record
Guides.UserId unique	One guide profile per login
Bookings: Status, TravelerId, PaymentDueAt, BalancePaymentDueAt	Dashboard filtering, "my bookings" and payment-deadline scans
ApprovalRequests: (Status, Type) index; filtered unique index allowing one open request per booking	Approval queue queries and no competing open approvals
Discounts (IsActive, MinGroupSize); SupportTickets indexes	Discount selection and support queue filtering
Transactions	Booking confirmation commits guide assignment, vehicle reservation, status change and audit entry together; failures roll back with no partial assignment

Migrations include InitialCreate and 19 later migrations, the most recent being AddApprovalRequests, AlignDriversWithModel (idempotent, guarded against data loss) and ApprovalRequest.RequesterNote.

5. ASP.NET Core API Design

The API follows REST conventions with asynchronous controller actions, DTOs, server-side validation, ProblemDetails error responses, correct status codes (200/201/400/401/403/404/409/429) and role-based authorization attributes. Both React and Flutter use the same endpoints.

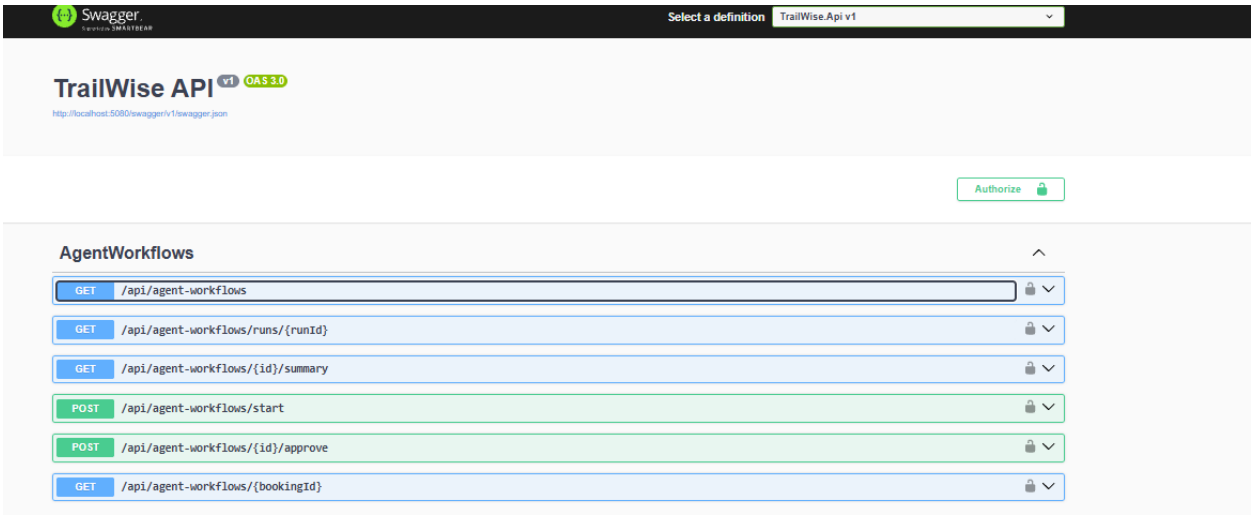
5.1 Major API Areas

Area	Key Endpoints	Roles
Auth	POST /api/auth/register, /login; GET /api/auth/me; PUT /me, /me/password; DELETE /me; POST/GET/DELETE /api/auth/admin/users	Public / any user / Admin
Packages	GET /api/packages (search, theme, duration, price, class, guests, rating, sort), GET /{id}, GET /facets; POST, PUT, DELETE; POST	Public read; Ops/Admin write

	/{id}/photo; POST, PUT, DELETE /{id}/tiers/{tierId}; POST /geocode-missing	
Bookings	POST /api/bookings; GET /mine, /{id}, /, /paged; PATCH /{id}/decision, /complete, /cancel; POST /{id}/assign-guide, /start-tour, /end-tour; GET/POST /{id}/itinerary	Traveler / staff by action
Approvals	GET /api/approvals/pending?type=; POST /api/approvals/{id}/decide (Approve, Reject, RequestRevision)	Ops Manager, Admin
Agent workflows	GET /api/agent-workflows (paged), /runs/{runId}, /{id}/summary, /{bookingId}; POST /start, /{id}/approve	Ops, Fleet, Admin (approve: Ops/Admin)
Guides & availability	GET/POST/PUT /api/guides; GET/PUT /api/guides/{id}/availability; GET /me/assigned-tours; PUT /me/profile	Guide / Ops / Fleet
Fleet	GET/POST/PATCH/DELETE /api/vehicles; /{id}/availability; /{id}/reservations; /assignments; GET/POST/PUT/DELETE /api/drivers; GET /api/drivers/me/assignments	Fleet/Admin; Driver (own trips)
Payments	POST /api/bookings/{id}/payments/bank-transfer; GET /api/payments/pending; POST /{id}/approve, /{id}/reject; GET /api/bookings/{id}/payment-status	Traveler / Ops, Admin
Reviews & discounts	POST /api/bookings/{id}/reviews; GET /api/packages/{id}/reviews, /api/reviews/featured; /api/discounts CRUD, /active	Traveler / public / Ops
Reports	GET /api/reports/occupancy, /revenue, /guide-utilization, /dashboard, /audit, /audit/export	Ops Manager, Admin
Support & locations	/api/support/tickets..., /api/staff/support/tickets...; GET /api/locations/search	Traveler / Ops

5.2 API Documentation Evidence

Swagger UI is enabled in the Development environment at /swagger and documents every controller, including the approvals and agent-workflow endpoints.



6. React Web Application Design

The React application is the operations console and public website. It uses functional components and hooks, React Router 7 with lazy-loaded routes, protected routes with role guards (wrong-role users see a No Access page), a shared design system (src/components/ui) and Zustand stores for shared approval and workflow state. All data comes from the ASP.NET Core API through a typed axios client.

6.1 Main Web Workflows

Area	Screens
Public	Home page with hero search, featured tours, "How TrailWise plans your trip"; tour explorer with URL-synced filters; package detail with tier comparison, map and reviews; login and register with booking intent preserved
Operations Manager	Dashboard (pending approvals by type, urgent refund requests, upcoming tours, guide and vehicle utilisation); Approval Queue (three tabs, evidence panel, approve/reject/request revision); Agent Workflow monitor (live run list, drill-down with plan, steps, tool calls, validation, execution summary, re-run); packages and tiers; bookings; payments; discounts; reports; support
Fleet Coordinator	Vehicles, drivers, allocation queue, manual guide assignment
Tour Guide / Driver	Assigned tours, tour detail and itinerary, availability calendar, profile; driver trips
Traveler (web)	Packages, booking request, my bookings, profile
Admin	Overview, staff management by role, package management

6.2 Design System and Quality

- Shared components: Button, Input, Select, DatePicker, Card, StatusBadge, Modal, Drawer, Tabs, DataTable, Skeleton, EmptyState, PageHeader, StatCard and toasts.

- Light and dark themes with semantic colour tokens; a test (noRawColors) fails the build if raw palette colours are reintroduced.
- Accessibility: WCAG AA contrast (primary buttons darkened to meet 4.5:1), labelled inputs, focus traps in dialogs, screen-reader text for status icons.
- Loading, empty, success and error states on every data screen; error toasts stay until dismissed.

7. Flutter Mobile Application Design

The Flutter application consumes the same ASP.NET Core API. Tokens are kept in flutter_secure_storage and attached by the shared API client; Provider manages authentication and screen state. A role-based bottom navigation shows different tabs for Travelers (Dashboard, Packages, My Bookings, Support), Tour Guides (Dashboard, Assigned Tours, Profile) and Drivers (Dashboard, Driving Tasks, Profile).

Role	Mobile Workflows
Traveler	Registration and login; package browsing and reviews; booking request form with date pickers; my bookings with status; bank-slip upload via file picker (device feature) with payment deadline countdown; cancellation; review submission; support tickets
Tour Guide	Assigned tours; tour detail with itinerary; itinerary editing; start/end tour; attendance/completion; guide notes; profile
Driver	Driving tasks with vehicle, traveler contact (call button), guide and itinerary highlights

Device features used: file picker for payment slips, date/time selection, phone dialer via url_launcher.

8. Agentic AI Architecture and Evaluation

Every new booking starts a coordinator workflow. The Coordinator / Planning Agent receives the booking objective, builds a fixed structured six-step plan, persists it to AgentWorkflowRuns.PlanJson and delegates each step to a specialist agent. The approval decision itself is made by a deterministic C# evaluator, never by an LLM. Each step is written to AgentStepLogs with input, output, tool calls, validation result and duration.

8.1 Specialized Agents

Agent (owner)	Responsibility	Output contract / tools
Coordinator / Planning Agent (Member 1)	Receives the booking objective, creates and persists the plan, delegates steps, maps the decision to booking and run status, creates approval requests	Plan JSON; run status (Running, AwaitingApproval, Completed, Failed); ApprovalRequest
Guide Matching Agent (Member 2)	Finds a suitable guide who is available for the complete booking period, matches the package specialization and supports the traveler's preferred language.	{ guideId, matchScore, reasoning }; tool: guide_availability_read

Fleet & Capacity Agent (Member 3)	Finds the smallest available vehicle with enough seats (and AC when the tier requires it) and a free driver	{ vehicleId, driverId, acMatch, seatConfigMatch, conflictCheck }; tool: vehicle_availability_read (write gated behind approval)
Pricing & Validation Agent (Member 4)	Calculates tier price x group + catering + add-ons - discount, and checks the budget	{ totalCost, breakdown, validationResult }; tool: pricing_calculator
Preference Extraction / Proposal Summary (LLM steps)	Extract structured preferences from special requests (advisory only); write a human-readable summary	tool: structured_output_formatter; only when LLM is enabled
BookingApprovalEvaluator (deterministic)	Business-rule gate before any high-impact action	Approved / NeedsApproval / ValidationFailed with reasons; recorded as deterministic_rule_check

8.2 Workflow and Human Approval

1. Traveler submits a booking request (Flutter or web); the API saves it as Requested and starts the workflow.
2. Plan: extract_preferences -> match_guide -> check_vehicle -> calculate_price -> validate -> summarize.
3. Deterministic validation: AC mismatch, vehicle conflict or guide match score below 0.5 -> NeedsManualReview (safe failure). Group above 10 or total above budget x group x 1.15 -> PendingApproval. Otherwise -> auto-confirm: guide assigned and vehicle reserved in one transaction.
4. PendingApproval creates an ApprovalRequest typed LargeGroupOrCustomItinerary or BudgetOverride. A traveler cancelling within 7 days of the start with an approved payment creates a RefundException request.
5. The Operations Manager reviews the evidence in the React Approval Queue (guide reasoning, vehicle checks, pricing breakdown, validation results) and approves, rejects (note required) or requests revision (note required).
6. Approve resumes the workflow: the guide is assigned and the vehicle reserved (vehicle_availability_write) in a transaction with the audit entry; refunds mark the payment Refunded. Reject cancels and releases resources. Revision returns the booking to the traveler with the note.
7. The traveler sees the updated status in the Flutter app and on the web.

8.3 Controls, Validation and Security

- Allow-listed tools only, named after design document 8.4: guide_availability_read, vehicle_availability_read, vehicle_availability_write (gated behind approval), pricing_calculator, structured_output_formatter; the rule check is recorded as deterministic_rule_check.
- Tool calls are recorded with name, short ID-only input and result summaries, status and duration; traveler free text is never copied into the log.
- A JSON redactor blanks secret-like keys (password, token, apiKey, card number ...) before step data is stored or returned.

- Prompt-injection resistance: special requests are treated as data; extracted preferences are advisory and cannot change the approval decision.
- Safe failure: exceptions in the background workflow mark the booking NeedsManualReview; LLM timeouts fall back to no summary.
- Role-based access: workflow monitor for Ops, Fleet and Admin; approvals for Ops Manager and Admin only.

Figure 3 · Agentic AI workflow and human approval

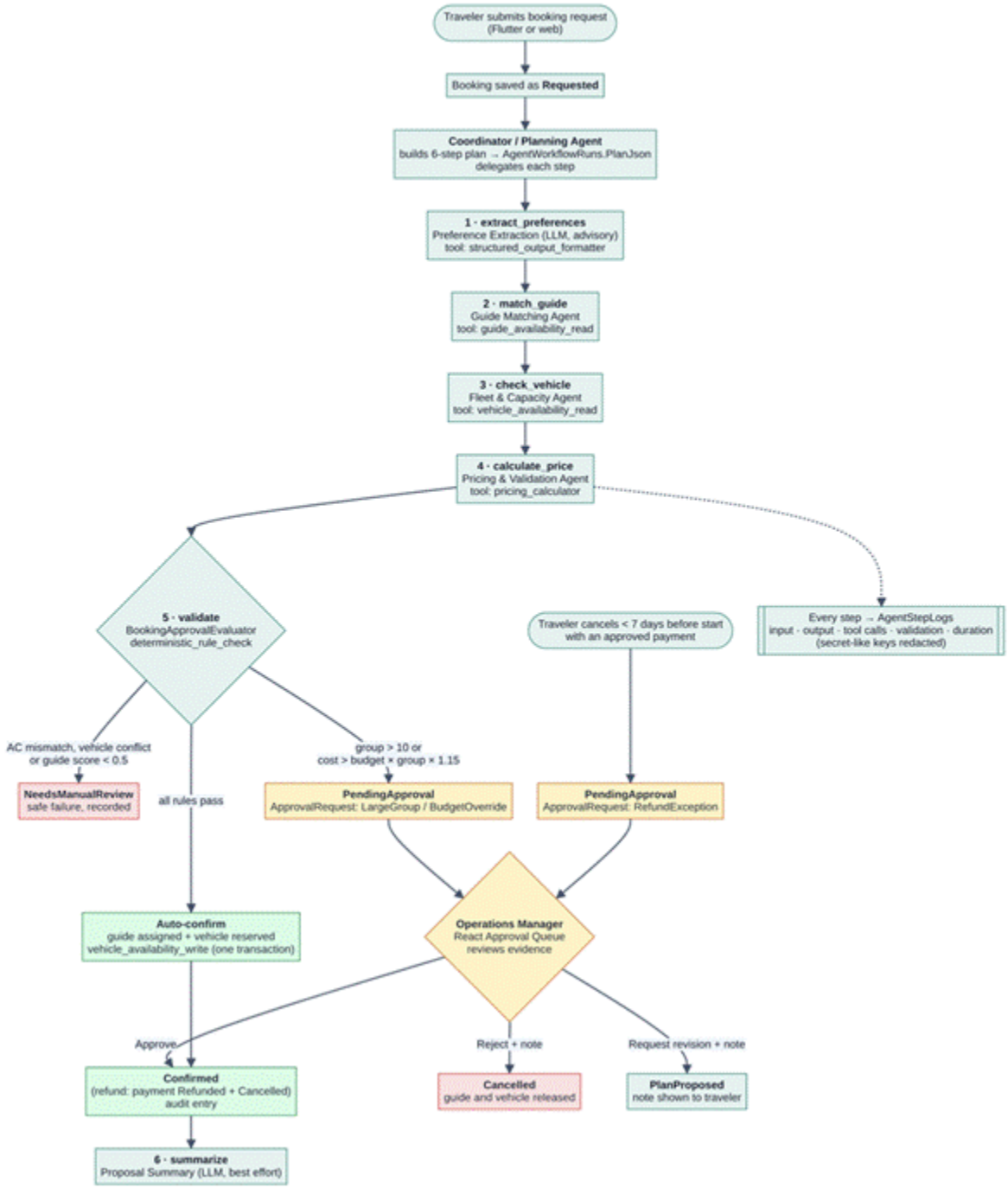


Figure 3. Coordinator workflow, deterministic approval gate and human-in-the-loop decisions

8.4 Agent Evaluation Evidence

An end-to-end smoke run against a fresh PostgreSQL 16 database (5 October 2026, all 20 migrations applied) produced these outcomes:

Golden case	Expected	Observed
12-traveler booking	Paused as LargeGroup approval; approve confirms with guide and vehicle; monitor shows tool calls	Paused; approve confirmed the booking; monitor showed vehicle_availability_write after approval
Over-budget booking (two requests)	BudgetOverride approval; revision and reject paths	One returned for revision (PlanProposed, note visible); one rejected (Cancelled)
Paid booking cancelled 3 and 2 days before start	RefundException; 2-day request flagged urgent	Both became refund exceptions; only the 2-day one urgent; approve refunded and cancelled; revision restored Confirmed with resources held
Reject without a note	Refused	400 Bad Request
Re-run a confirmed booking	Refused	409 Conflict
No matching guide (language not spoken)	Safe failure to NeedsManualReview	No eligible guide was found for the requested language. The Guide Matching Agent returned Guid.Empty with a MatchScore of 0. The booking was safely changed to NeedsManualReview instead of being automatically confirmed.
Injected instruction in special requests	Ignored; decision unchanged	The injected instruction in the special request was treated only as user data and did not affect the booking decision. The deterministic validation rules remained unchanged and the booking workflow continued according to the normal business rules.

Automated agent tests: BookingApprovalEvaluatorTests, CoordinatorAgentServiceTests, GuideMatchingAgentTests, FleetCapacityAgentTests, PricingValidationAgentTests, PreferenceExtractionAgentTests, ProposalSummaryAgentTests, ApprovalsEndpointsTests and tool-call recorder tests.

9. Technical Report

9.1 Backend and Integration Approach

Controllers handle HTTP concerns and delegate to services (for example ApprovalService, PaymentService, GuideAssignmentService, FleetReservationService, OperationsReportService). Shared approve/reject logic was extracted from BookingsController into ApprovalService so the legacy booking decide endpoint and the new approvals endpoint behave identically. DTOs prevent exposing entities directly, and all DTO changes were additive so the Flutter app kept working.

9.2 Authentication and Authorization

JWT bearer tokens carry the user id and role. Endpoints use [Authorize(Roles = ...)] attributes; ownership checks ensure travelers only see or change their own bookings, and drivers only see trips of the driver record linked to their account. Passwords are hashed with ASP.NET Core PasswordHasher. Travelers self-register; only Admin creates staff accounts.

9.3 Error Handling and Validation

A global exception handler returns ProblemDetails. Request DTOs use data annotations and explicit business validation (field errors returned as 400). Invalid state transitions return 409 (for example deciding an already confirmed booking or deleting the last tier of a package). Background workflow failures are caught and the booking is marked NeedsManualReview.

9.4 Cross-Platform Integration

The required cross-platform workflow: a traveler submits a booking in Flutter -> ASP.NET Core saves it in PostgreSQL and runs the agents -> a large group pauses for approval -> the Operations Manager approves in React -> the confirmed status, guide and vehicle appear back in the traveler's Flutter app.

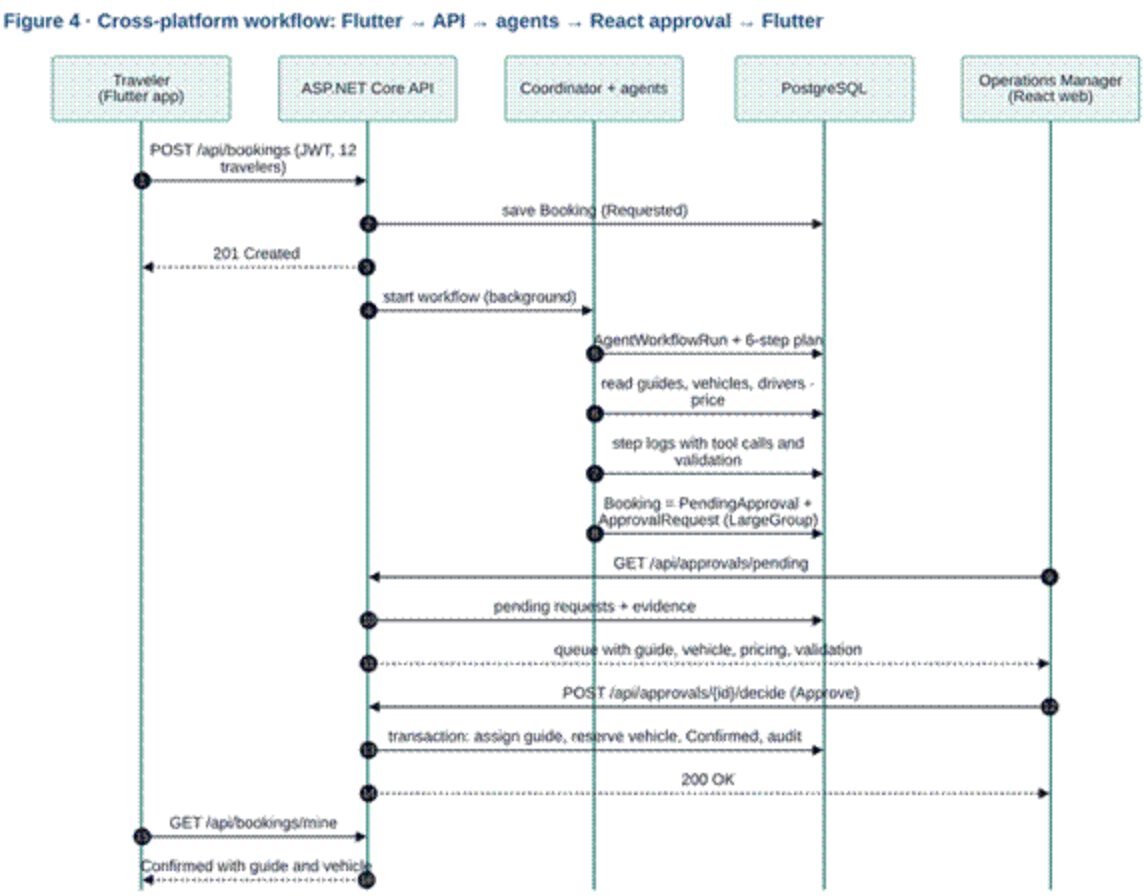


Figure 4. Cross-platform sequence: Flutter booking, agent run, React approval, Flutter result

9.5 Third-Party Integration

Service	Purpose	Controls
OpenStreetMap Nominatim	Location search and geocoding of package locations (Sri Lanka only)	Backend only; 1 request/second throttle; User-Agent set; never blocks saving; unmatched places left off the map
Notify.lk SMS	Booking confirmation SMS	Keys in environment variables; failures logged, never block confirmation
Groq LLM	Preference extraction and proposal summary	Disabled by default (LLM_ENABLED); timeout; advisory only

10. Software Testing Report

10.1 Testing Strategy

Backend tests use xUnit with WebApplicationFactory and the EF Core in-memory provider, plus real JWT login, covering services, validation, authorization, controllers and agents. Database behaviour (migrations, constraints, transactions) was verified against a throwaway PostgreSQL 16 container. React tests use Vitest and Testing Library. Flutter tests use flutter_test widget tests. All suites run in GitHub Actions on push and pull request to main.

10.2 Test Matrix

Test area	What is verified	Result
Backend API and services	CRUD, validation, status codes, role guards, approvals (3 types x 3 decisions), refund window boundary, rollback, audit entries	All backend tests passed successfully.
Database	All migrations apply to empty PostgreSQL; idempotent driver migration; unique and FK constraints; transaction rollback	PostgreSQL smoke test completed successfully and migrations/constraints were verified.
React	Components, forms, protected routes, approval queue, workflow monitor, dashboard, design-system guard	React test suite passed successfully.
Flutter	Widget, form-validation, navigation and API tests	Flutter test suite passed successfully.
Agents	Evaluator thresholds, coordinator flows, guide/fleet/pricing agents, tool-call recording, redaction	All agent-related tests passed as part of the backend test suite.
End to end	Booking -> agents -> approval -> confirmation	Booking -> agents -> approval -> confirmation workflow tested successfully.

CI	Backend build + test; web lint/test/build; Flutter analyze/test	GitHub Actions completed successfully for backend, web and Flutter pipelines.

10.3 Defects and Resolutions

Defect	Fix and retest
Four backend tests failed intermittently (status sort differs between PostgreSQL and in-memory; background workflow raced fleet tests)	Tests seed bookings directly and use provider-independent statuses; full suite green twice
Approving a manual-review booking auto-assigned any free guide, ignoring language and specialization (design doc 8.4)	Approval now only assigns guides that pass the Guide Matching rules; otherwise 409 "Missing: Tour Guide"; new tests
Cancelling a booking left the guide blocked and did not store the reason	Cancellation releases GuideAvailability rows and stores CancellationReason; tests confirm the guide is bookable again
Privacy: driver assignments endpoint matched drivers by name or phone, exposing travelers' phone numbers	Endpoint is Driver-only and uses the linked account only; unambiguous startup linking; 26 tests
Drivers table drifted from the model; deleting a driver's account failed with an FK violation	Idempotent migration aligning the schema (ON DELETE SET NULL), verified on fresh and drifted databases
Guide utilisation reported 100% when only assignment rows existed	Now assigned days / days in window; tests updated
White text on primary buttons failed WCAG AA (4.26:1)	Primary colour darkened (6.12:1)

11. Performance Report

Scenario	Metric	Observed result	Target / interpretation
POST /api/auth/login	p50 / p95 response time	p50: 77.32 ms, p95: 110.85 ms	Note the login rate limit (5 per minute per IP)
GET /api/packages (anonymous)	Response time under concurrent users	Avg: 153.73 ms, p50: 147.64 ms, p95: 187.67 ms, 30/30 successful	Public read limit 120 per minute per IP
POST /api/bookings	Response time	102.97 ms, HTTP 201 Created	Returns immediately; workflow runs in background
Agent workflow (booking -> final status)	End-to-end latency, LLM off / on	LLM off: 170 ms, LLM on: 618 ms	From AgentStepLogs durations
GET /api/approvals/pending	Response time	p50: 5.64 ms, p95: 19.36 ms	Indexed by (Status, Type)

Concurrent bookings	Success / failure rate	4/4 succeeded, 0 failed, 0 guide conflicts, 0 vehicle conflicts	No double-booked guides or vehicles

12. Deployment Report

12.1 Deployment Summary

Component	Platform / URL	Status
ASP.NET Core API	Render (Docker, free tier, Singapore/Virginia) – https://trailwise.onrender.com	Deployed
Health URL	GET /health (anonymous, includes a database check) – https://trailwise.onrender.com/health	Healthy
Swagger	https://trailwise.onrender.com/swagger	Deployed
PostgreSQL	Supabase PostgreSQL (Session pooler, ap-northeast-1); migrations applied on API start; RLS enabled	Hosted (Supabase)
React	Vercel – https://trailwise-xi.vercel.app (VITE_API_BASE_URL = Render API)	Deployed
Flutter	Android APK built with --dart-define=API_BASE_URL=https://trailwise.onrender.com	Built (app-release.apk)
Agentic AI	Runs inside the API process; no separate service. Groq optional via GROQ_API_KEY	Deployed with API

12.2 Required Environment Variables

Variable	Purpose	Secret?
CONNECTION_STRING	PostgreSQL connection string	Yes
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB / POSTGRES_PORT	Local Docker database	Password yes
JWT_SECRET	JWT signing key (32+ characters)	Yes
JWT_ISSUER / JWT_AUDIENCE / JWT_EXPIRY_MINUTES	Token settings	No
ADMIN_SEED_NAME / ADMIN_SEED_EMAIL / ADMIN_SEED_PASSWORD	First admin account created on startup	Password yes
CORS_ALLOWED_ORIGIN / CORS_ALLOWED_ORIGIN_FLUTTER	Allowed web origins	No

LLM_ENABLED / GROQ_API_KEY / LLM_EXTRACTION_MODEL / LLM_SUMMARY_MODEL	Optional LLM steps	Key yes
NOTIFY_LK_USER_ID / NOTIFY_LK_API_KEY / NOTIFY_LK_SENDER_ID	SMS notifications	Key yes
BACKEND_PORT / FRONTEND_WEB_PORT	Local ports (5080 / 5173)	No
VITE_API_BASE_URL	React API base URL	No
API_BASE_URL (Flutter --dart-define)	Mobile API base URL	No
Cancellation:RefundWindowDays (appsettings)	Late-cancellation window, default 7	No

12.3 Startup Instructions (local)

```
git clone <repository-url> && cd TrailWise
cp .env.example .env        # set JWT_SECRET; LLM_ENABLED=false unless a Groq key is set
docker compose up --build   # db (5432), backend (5080, applies migrations + seed), frontend-
web (5173)
# optional extra demo data:
docker compose exec -T db psql -U trailwise -d trailwise < backend/seed_all_tables.sql
# mobile:
cd frontend-mobile && flutter pub get
flutter run --dart-define=API_BASE_URL=http://localhost:5080   # Android emulator:
http://10.0.2.2:5080
# tests:
cd backend && dotnet test   |   cd frontend-web && npm test   |   cd frontend-mobile &&
flutter test
```

Swagger: http://localhost:5080/swagger. After the first start, run POST /api/packages/geocode-missing as Admin until remaining is 0 to add map coordinates to existing packages.

12.4 Test Accounts

Role	Email	Password
Admin	admin@trailwise.local (ADMIN_SEED_EMAIL)	Admin123
Driver	Register	Register
Tour Guide	Register	Register

Operations Manager	Register	Register
Fleet Coordinator	Register	Register
Traveler	Register	Register

13. Architecture Decision Records (ADRs)

Full ADR files are in docs/adr. Summaries:

13.1 ADR-01: React State Management (docs/adr/0002)

Field	Record
Context	The app used plain useState plus a typed axios client. Session work introduced shared state: the approval queue and counts (nav badge, queue page, dashboard) and a polled workflow run list.
Options considered	React Context only; Redux Toolkit (with RTK Query); Zustand; TanStack Query
Decision	Zustand for booking/approval and workflow state; React Context stays for authentication and theme.
Consequences	Small (about 1 kB), no providers or reducers, easy to reset in tests. Less enforced structure and no built-in request cache; revisit TanStack Query if server-state screens multiply.
Status	Accepted

13.2 ADR-02: Flutter State Management

Field	Record
Context	The mobile app needs authenticated user state, role-based navigation and screen state for bookings and tours.
Options considered	Provider; Riverpod; Bloc
Decision	Provider (ChangeNotifier) for authentication and app-wide state, with local widget state for screens.
Consequences	Simple and familiar for the team, well supported by flutter_test. Less compile-time safety than Riverpod; acceptable at this app size.
Status	Accepted

13.3 ADR-03: Agentic AI Framework and Orchestration

Field	Record
Context	The assignment requires planning, delegation to distinct agents, allow-listed tools, persisted state, deterministic validation and human approval, integrated through ASP.NET Core.
Options considered	LangGraph (Python service called by the API); Microsoft Agent Framework; custom C# orchestration
Decision	Custom C# orchestration inside the API: CoordinatorAgentService with a fixed six-step plan and four agent interfaces with structured result records. LLM (Groq) used only for advisory steps.
Consequences	One runtime and deployment, no extra service, fully deterministic decisions and easy testing. The plan is static rather than LLM-generated; the background task has no durable queue (a crash leaves a booking Requested; it can be re-run via POST /api/agent-workflows/start).
Status	Accepted

13.4 ADR-04: Agent Workflow State Schema (docs/adr/0003)

Field	Record
Context	Workflow state, step results, tool calls and approvals must be durable and auditable; pending refund requests need their own persisted state.
Options considered	Single JSONB blob per booking; dedicated relational tables
Decision	Dedicated tables: AgentWorkflowRuns (plan, status, summary), AgentStepLogs (input, output, ToolCallsJson, validation, duration) and ApprovalRequests (added beyond design doc 4) with a filtered unique index allowing one open request per booking.
Consequences	Queryable history for the monitor and dashboard and clear audit. More tables and migrations; JSON columns are redacted before storage.
Status	Accepted

13.5 ADR-05: Cloud Deployment Platform

Field	Record
Context	Evaluators need stable URLs without paid subscriptions until 21 October 2026.

Options considered	API: Render (Docker), Railway, Azure for Students. Database: Supabase, Neon, local Docker PostgreSQL. React: Vercel, Netlify.
Decision	Host PostgreSQL on Supabase (free tier, managed, Session pooler on port 5432 for Npgsql). Package the API as a Docker image ready for Render and the React app for Vercel; for the demo the API and web run locally against the hosted Supabase database.
Consequences	Free-tier limits (project pause after inactivity, connection limits). The Session pooler is required because the direct host is IPv6-only and the transaction pooler breaks Npgsql prepared statements. Secrets stay in environment variables; RLS stops the Supabase Data API exposing tables. Render free tier would add cold starts and lose uploaded files on redeploy.
Status	Accepted (database); API and web hosting proposed

13.6 ADR-06: Payments and Payment Gateway Scope

Field	**Record**
Context	The design document (9) lists a payment gateway sandbox as an option. A complete bank-slip payment flow (Member 4) already exists: upload, staff verification, deadlines and balance payment.
Options considered	Add a Stripe/PayHere sandbox alongside slips; replace slips with a gateway; in-person payments only; keep the existing slip flow
Decision	Keep the existing bank-slip flow unchanged; no gateway. Third-party integration is provided by Nominatim, Notify.lk and Groq. Refund approvals mark payments Refunded (status only; money returned outside the system).
Consequences	Documented deviation from design doc 9; no card data handled. Booking add-on selection was also left out of scope by team decision.
Status	Accepted (team decision)

14. Security Considerations

- JWT authentication with role claims; role-based [Authorize] on every protected endpoint; ownership checks for travelers and drivers.
- Passwords hashed with ASP.NET Core PasswordHasher (PBKDF2); minimum length 8.
- Rate limiting: login 5 requests per 60 seconds per IP; anonymous public reads 120 per minute per IP; generic 429 response.
- CORS restricted to configured origins; secrets only in environment variables; .env is git-ignored.
- Global exception handler returning ProblemDetails without stack traces.
- Audit log for bookings, approvals, refunds, cancellations, tier changes and workflow starts.

- Agent safety: allow-listed tools, ID-only tool summaries, JSON redaction of secret-like keys, special requests treated as data, deterministic rule gate, safe failure.
- Payment slips stored privately, limited to JPG/PNG/WEBP/PDF up to 5 MB, viewable only by the owner and staff.
- Privacy fix: driver trips readable only by the linked driver account; logs record IDs, never phone numbers.
- Known gaps: no refresh tokens; forwarded-headers configuration needed behind a reverse proxy so rate limiting sees client IPs.

15. Consolidated Group AI Usage Declaration

The TrailWise team used AI-assisted tools during development as permitted by Level 4 (Full AI) of the assignment, including chat assistants and agentic coding tools (for example Claude and Claude Code). AI tools were used for planning, code generation and refactoring, test generation, documentation drafting and review. Every AI-assisted change was reviewed, run against the automated test suites and checked by the owning member. Individual AI usage logs in Section 16 record the date, tool and model, task, output, changes or rejections, and verification. No fabricated commits, test results or AI log entries are included, and every member can explain, test and modify the work submitted under their name. No AI tools will be used during the demonstration or viva.

16. Individual Report Sections

16.1 Member 1 - Tour Package & Booking Management + Coordinator / Planning Agent

Hasanga H N S - IT24100962

16.1.1 Contribution Statement

Member 1 owns the Tour Package & Booking Management component and the Coordinator / Planning Agent. The work covers the package, tier, booking and add-on data model; the package and booking API; the booking validation rules; the coordinator workflow that plans and delegates to the other three agents; the human-approval system (three approval types); and the React operations screens for approvals, workflow monitoring, the Ops dashboard and package/tier management. Member 1 also led the shared UI design system and several cross-cutting fixes.

16.1.2 Owned Technical Scope

Layer	Owned work
Database	TourPackages, PackageTiers, PackageLocations, Bookings, BookingAddOns, ApprovalRequests; migrations AddApprovalRequests, AlignDriversWithModel, RequesterNote
Backend	Package CRUD, tier add/edit/delete with (class, food) uniqueness and last-tier rule; anonymous package search/filter/sort and facets; booking creation and validation; approvals API (pending list with evidence, decide); refund-exception cancellation rule (7-

	day window); agent-workflow list, run detail, summary, start and approve endpoints; Ops dashboard endpoint
Agentic AI	Coordinator / Planning Agent: plan creation, delegation, decision mapping, approval request creation; tool-call recording (IToolCallRecorder) and JSON redaction
React	Design system and app shell (Session 1), public explorer and package detail (Session 2), Approval Queue, Agent Workflow monitor, Ops dashboard, package catalogue with tier editor, Zustand stores
Flutter	Traveler booking flow, booking status tracking and itinerary viewer (c764472, b91ee6b); full mobile restyle onto the shared design system with light/dark theme (flutter_ui_updates)."
Testing	Approvals, refund exception, workflow endpoints, dashboard, coordinator and tier tests; React tests for approvals, workflows, dashboard and packages

16.1.3 Coordinator Agent Contract

Input: a booking-request event (booking ID, package tier, dates, group size, budget, language, special requests). Output: a persisted AgentWorkflowRun with the six-step PlanJson, one AgentStepLog per step (input, output, tool calls, validation result, duration) and a final status - Completed (auto-confirmed), AwaitingApproval (with an ApprovalRequest), or Failed (NeedsManualReview). The coordinator never decides approval itself; it calls the deterministic BookingApprovalEvaluator.

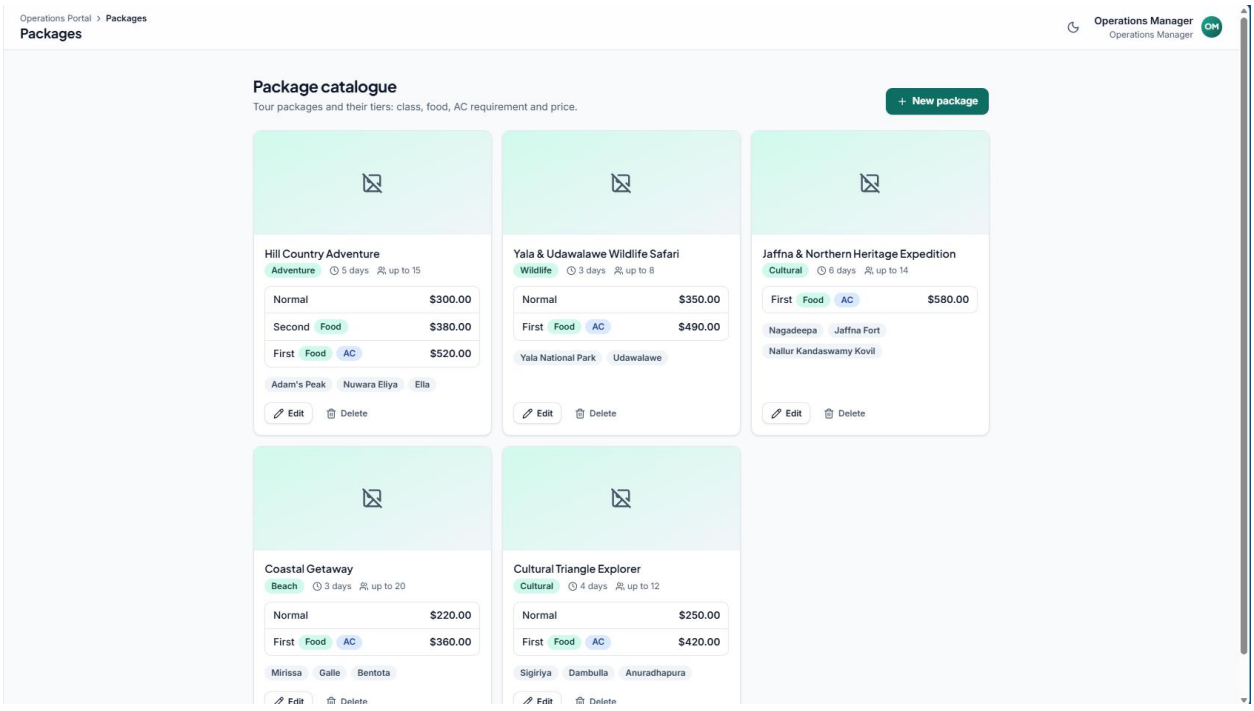
16.1.4 Key Implementation Evidence

Evidence	Details
Branch	booking_implementation, admin_dashboard, traveler_booking_flow, person1_phaseA_complete, Coordinator/Planning-Agent_implementation(P1-agent), testing_person1_phaseF, planing_agent-reconstruct(P1), system_improvments, sys_fixes_for_all, ui_improvements, flutter_ui_updates
Key commits	67872b2 booking and package management implementation (16 Sep) c764472 traveler-booking flow base implementation (17 Sep) b91ee6b Flutter: booking status tracking + itinerary viewer (18 Sep) fb27103 Coordinator/Planning Agent (P1agent) implementation (18 Sep) 70c8dc4 Automated tests (backend xUnit, React Testing Library, Flutter widget tests) (18 Sep) 3115e76 P1 agent LLM integration (27 Sep) e41fcef Only auto-assign guides that pass the Guide Matching rules on approval a82472e Add approvals queue API: ApprovalRequest model, pending list with evidence, decide 5f56c2c Add cancellation / refund exception approval (7-day window) 4c7dae2 Add workflow monitor and Ops dashboard endpoints f03763d Record real agent tool calls in AgentStepLog.ToolCallsJson 7bfa068 Add the Ops approval queue (React): evidence panel, approve / reject / request revision 4010732 Add the agent workflow monitor (React): run list and drill-down 074d4c1 Rebuild the Ops dashboard 1768e58 Add tier edit and delete endpoints for packages

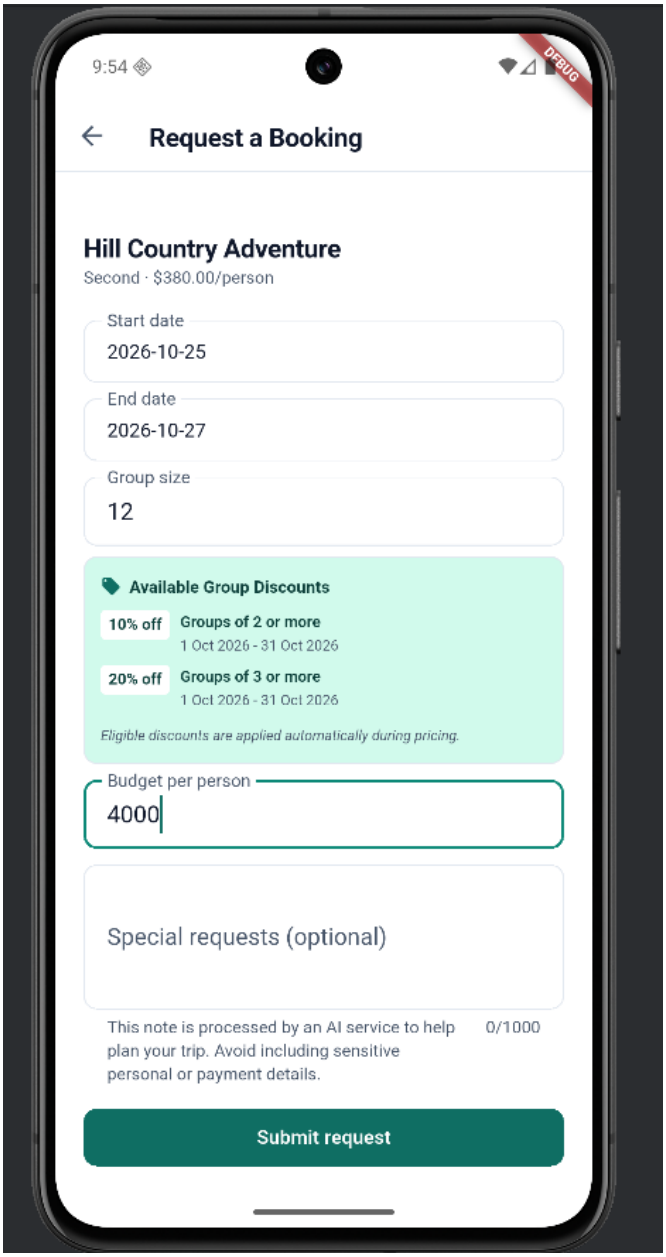
	5395816 Rebuild the package catalogue with tier configuration cb2baa2 Remove demo seed data; seed only the first Admin 7f75db1 Prepare for deployment: health check, PORT binding, CORS, Vercel rewrite 94694f2 Add a light/dark mode toggle, light by default 42d842b Add Flutter theme, light/dark toggle and shared widgets
Pull requests	#1 booking_implementation, #2 admin_dashboard, #3 traveler_booking_flow, #4 person1_phaseA_complete, #5 Coordinator/Planning-Agent, #7 testing_person1_phaseF, #10 and #14 planing_agent-reconstruct(P1), #18 system_improvments, #20 sys_fixes_for_all (github.com/Hasanga910/TrailWise/pull/N)
Backend files	Controllers/ApprovalsController.cs, AgentWorkflowsController.cs, PackagesController.cs, BookingsController.cs; Services/ApprovalService.cs, CancellationPolicy; Agents/Coordinator/CoordinatorAgentService.cs
React files	pages/ops/OpsApprovalsPage.tsx, OpsWorkflowsPage.tsx, OpsDashboardPage.tsx; components/ops/approvals/*, components/packages/*; stores/approvalsStore.ts, workflowStore.ts; components/ui/*
Automated tests	backend 888 passing · web 471 (53 files) · Flutter 297, all on 5 Oct 2026. Add a screenshot of each run.

16.1.5 Screenshots / Demonstration Evidence

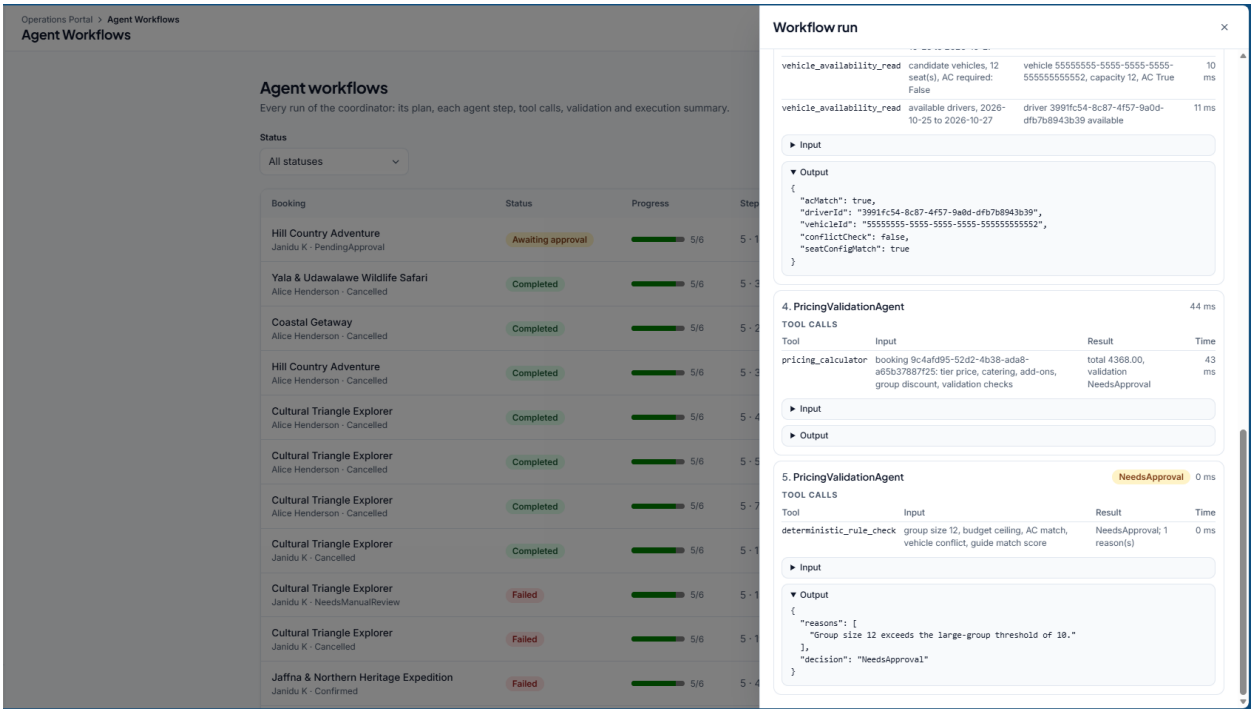
1. Package catalouge



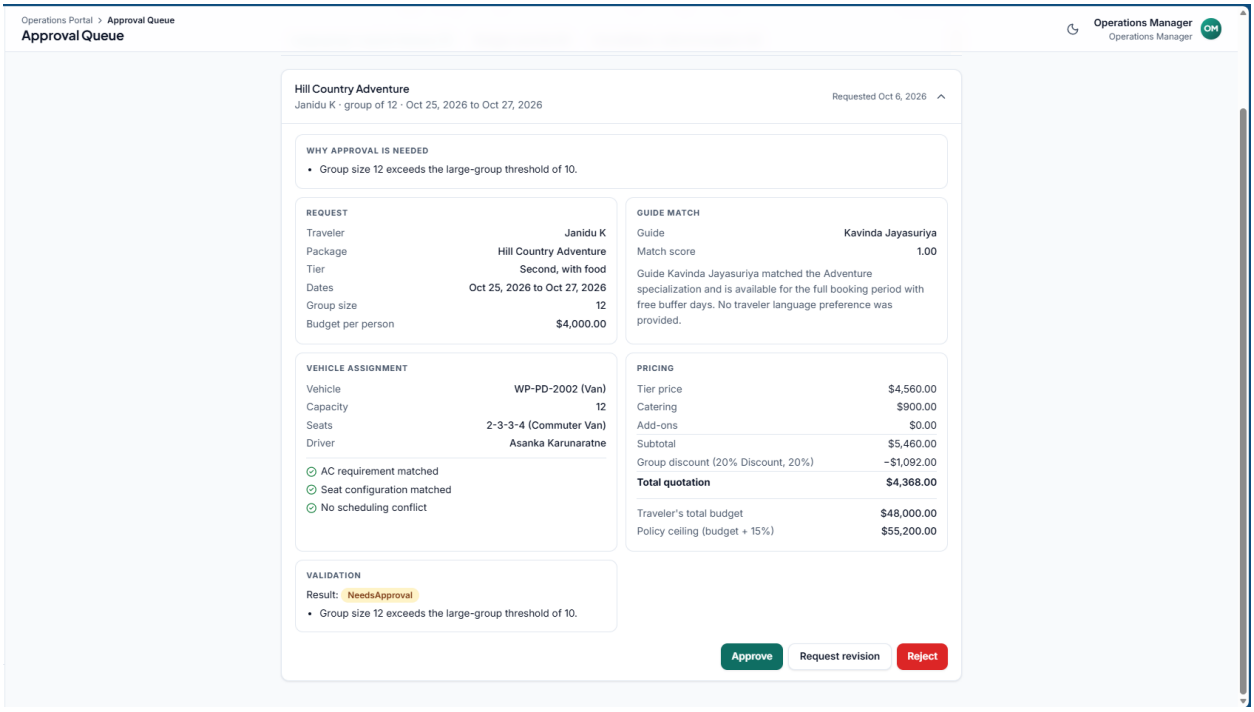
2. Traveler booking request



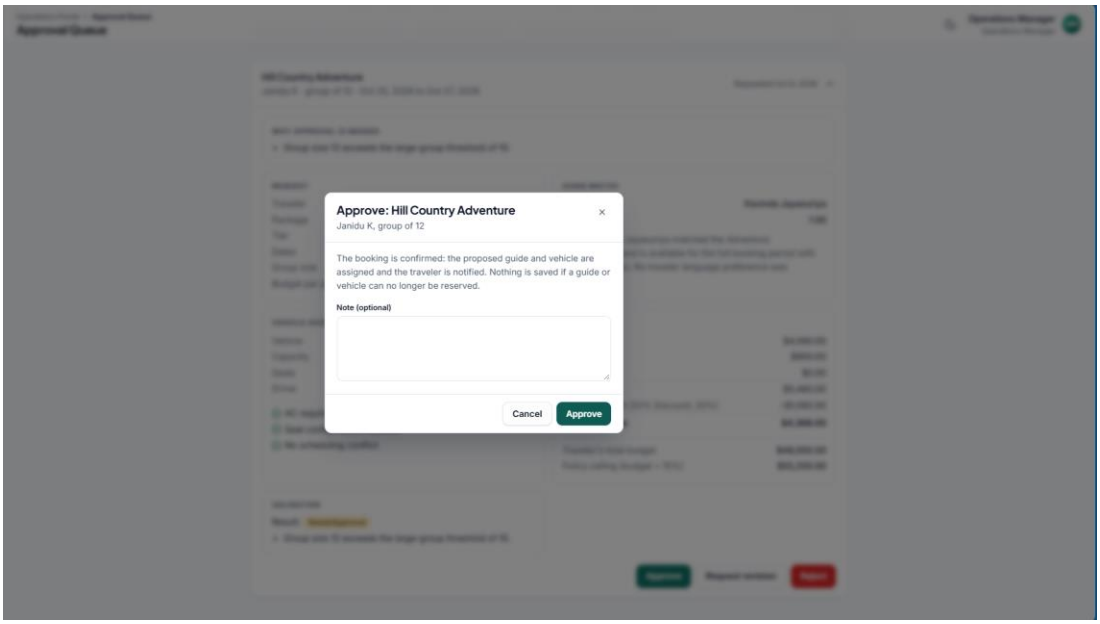
3. Agent workflow drill-down



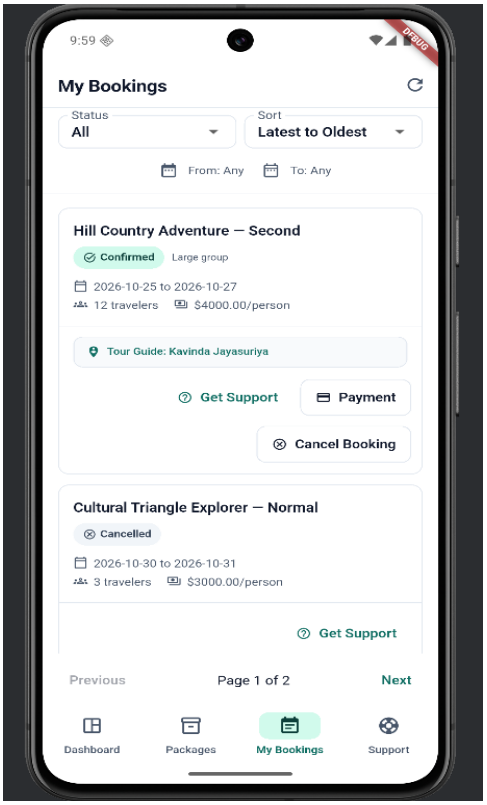
4. Approval queue with evidence



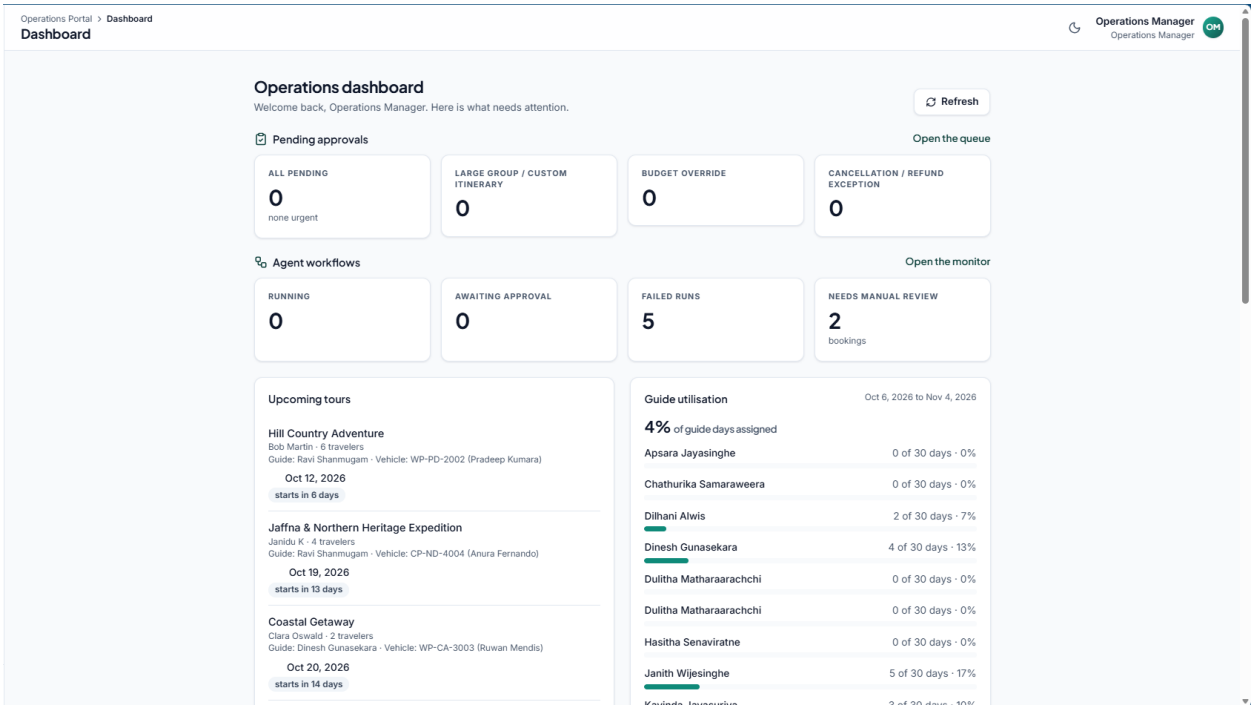
5. decision dialog



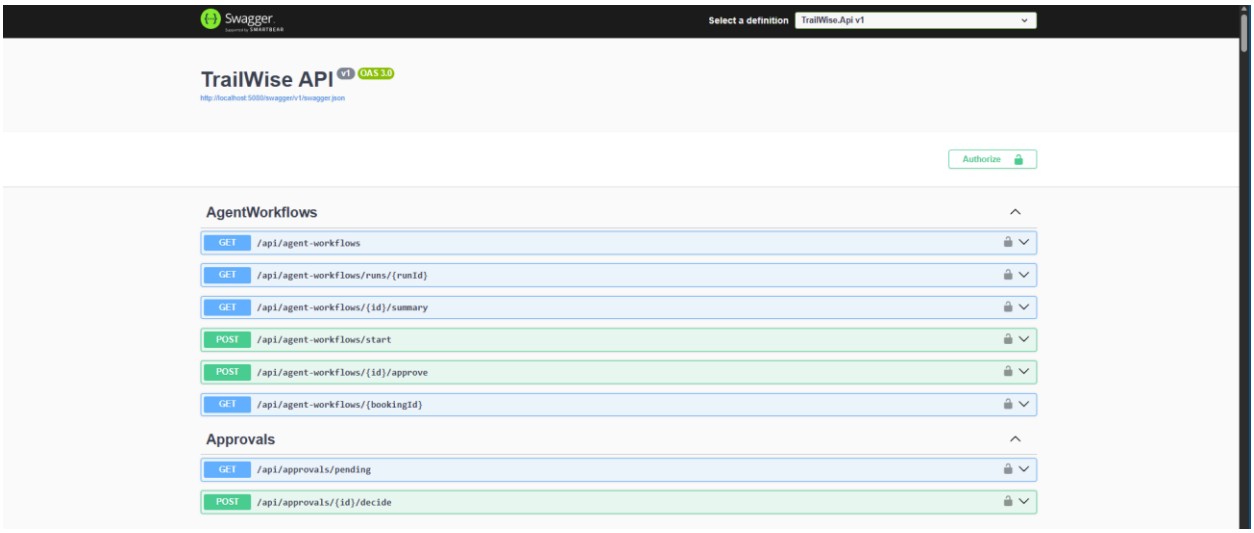
6. Booking confirmed



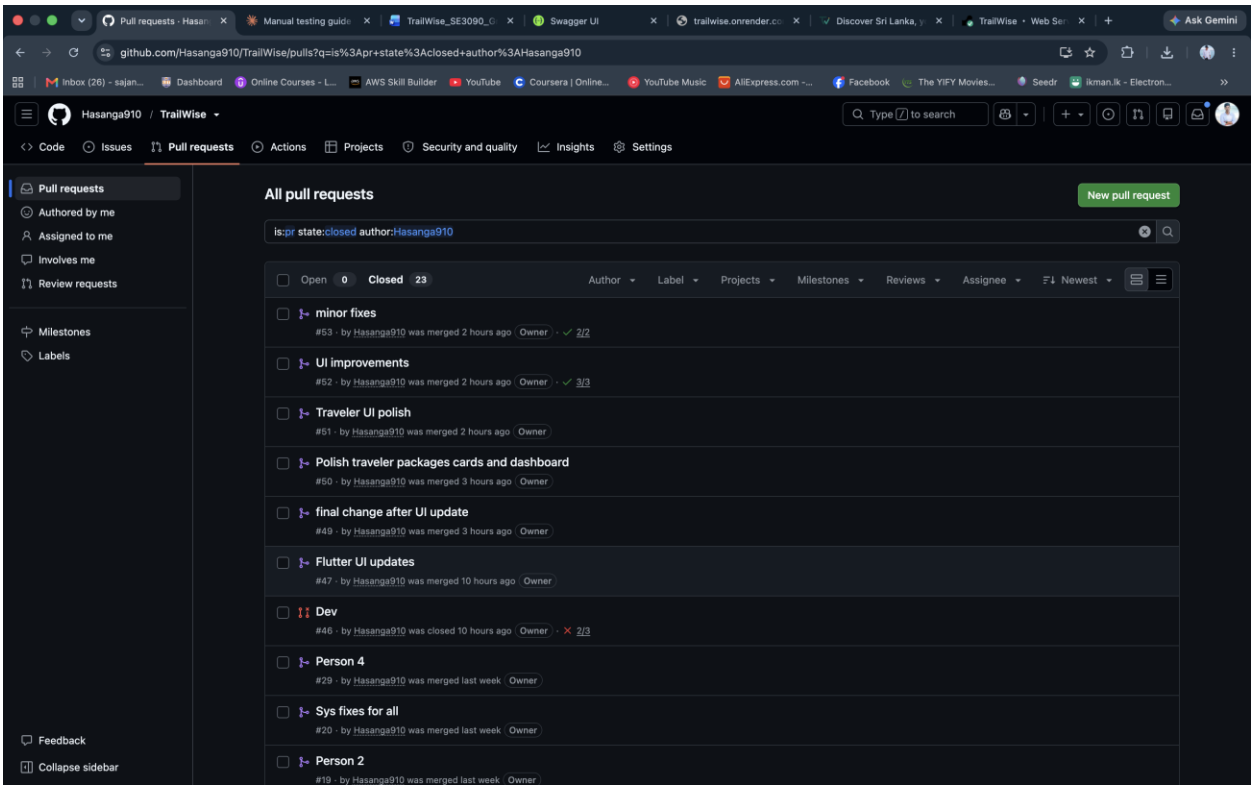
7.Ops Dashboard

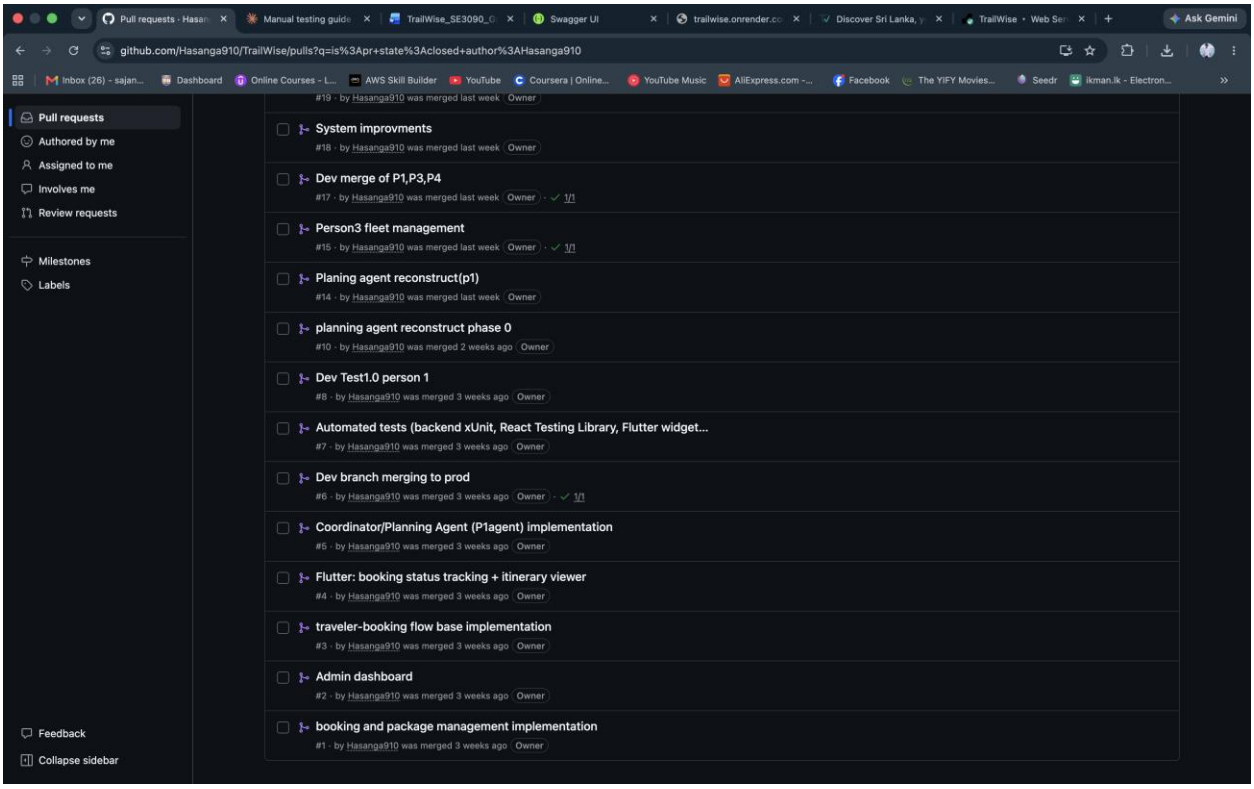


8.Swagger approvals

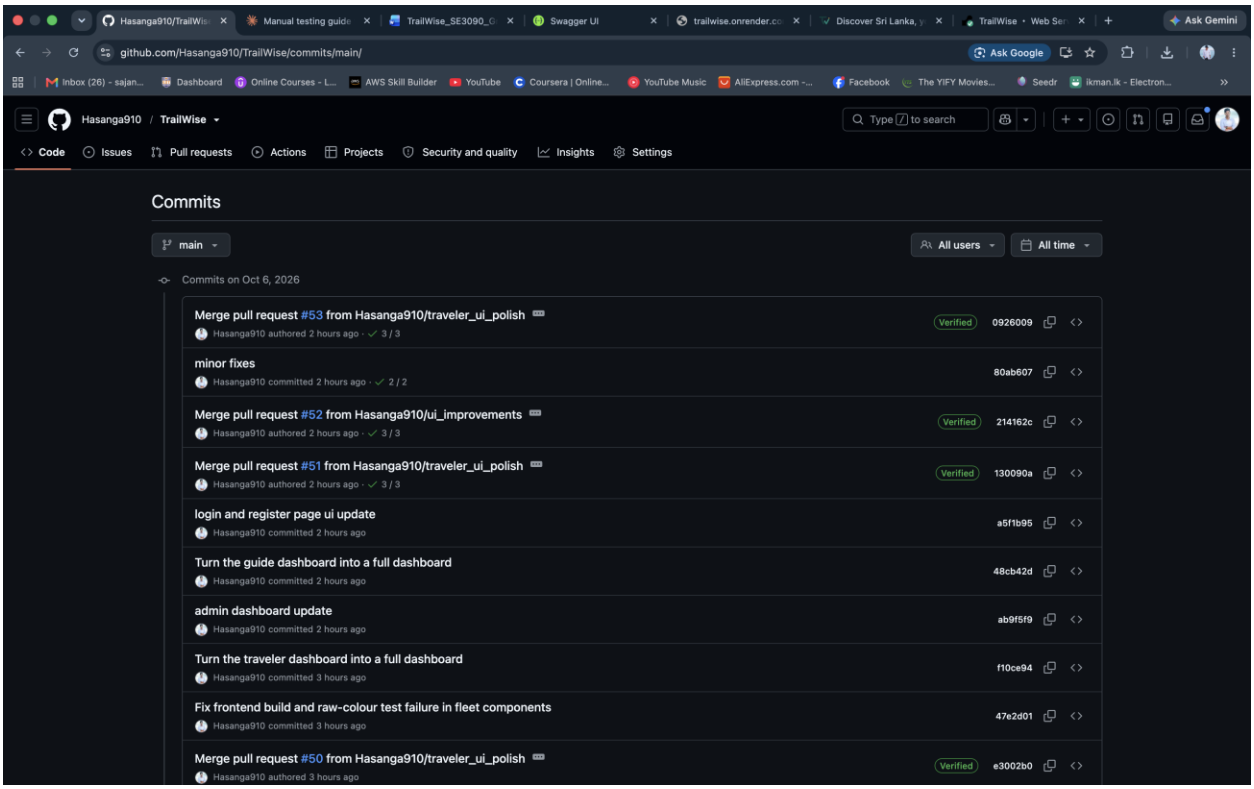


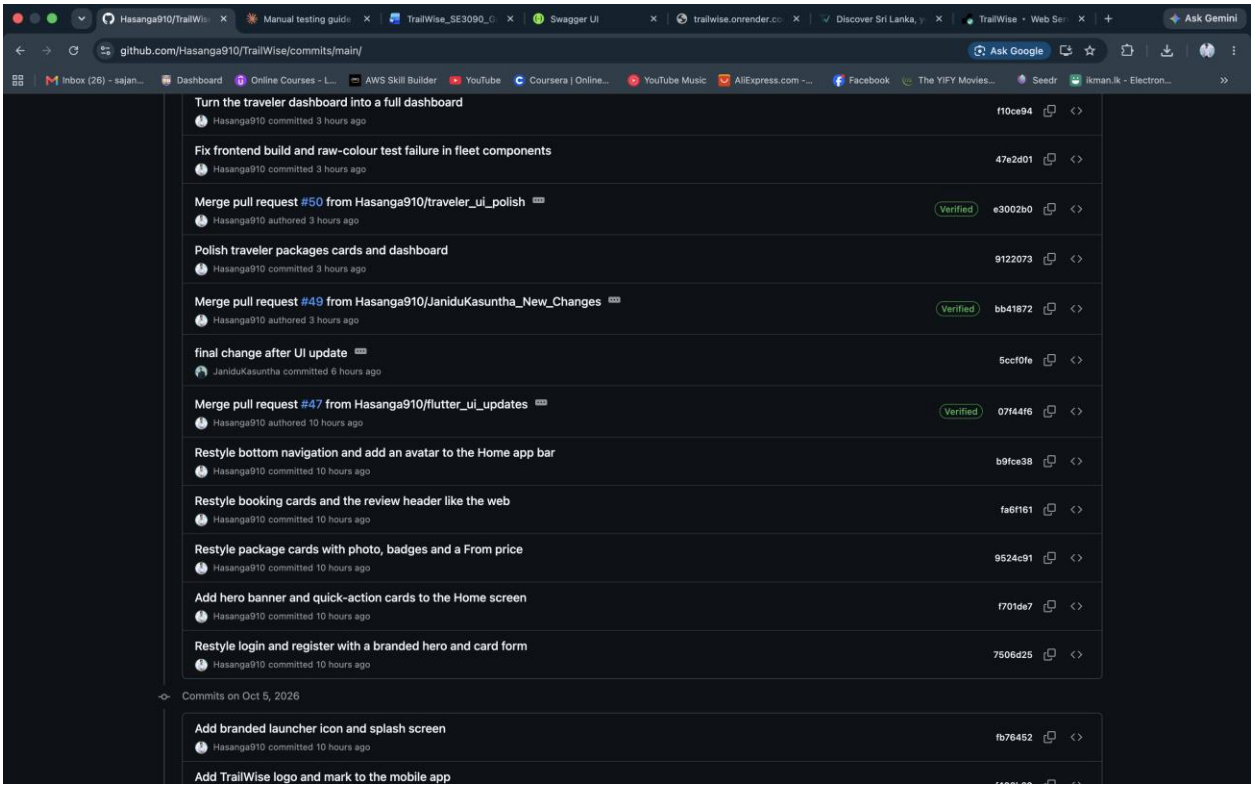
9.Github pull requests

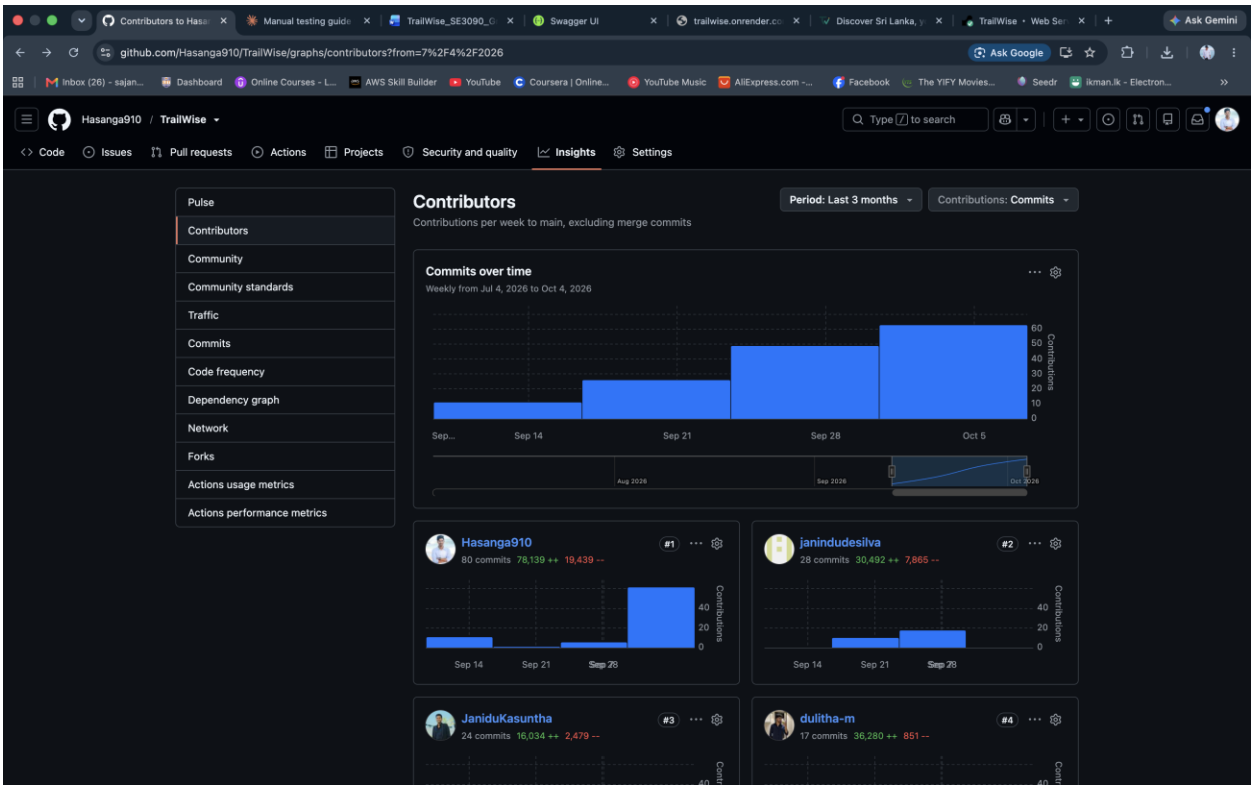




10.Github committs


github.com/Hasanga910/TrailWise/commits/main/
Hasanga910 / TrailWise
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Settings
Commits
main
All users
All time
Commits on Oct 6, 2026
Merge pull request #53 from Hasanga910/traveler_ui_polish
Hasanga910 authored 2 hours ago · 3 / 3
Verified
0926009
minor fixes
Hasanga910 committed 2 hours ago · 2 / 2
80ab607
Merge pull request #52 from Hasanga910/ui_improvements
Hasanga910 authored 2 hours ago · 3 / 3
Verified
214162c
Merge pull request #51 from Hasanga910/traveler_ui_polish
Hasanga910 authored 2 hours ago · 3 / 3
Verified
130090a
login and register page ui update
Hasanga910 committed 2 hours ago
a5f1b95
Turn the guide dashboard into a full dashboard
Hasanga910 committed 2 hours ago
48cb42d
admin dashboard update
Hasanga910 committed 2 hours ago
ab9f5f9
Turn the traveler dashboard into a full dashboard
Hasanga910 committed 3 hours ago
f10ce94
Fix frontend build and raw-colour test failure in fleet components
Hasanga910 committed 3 hours ago
47e2d01
Merge pull request #50 from Hasanga910/traveler_ui_polish
Hasanga910 authored 3 hours ago
Verified
e3002b0


github.com/Hasanga910/TrailWise/commits/main/
Turn the traveler dashboard into a full dashboard
Hasanga910 committed 3 hours ago
f10ce94
Fix frontend build and raw-colour test failure in fleet components
Hasanga910 committed 3 hours ago
47e2d01
Merge pull request #50 from Hasanga910/traveler_ui_polish
Hasanga910 authored 3 hours ago
Verified
e3002b0
Polish traveler packages cards and dashboard
Hasanga910 committed 3 hours ago
9122073
Merge pull request #49 from Hasanga910/JaniduKasuntha_New_Changes
Hasanga910 authored 3 hours ago
Verified
bb41872
final change after UI update
JaniduKasuntha committed 6 hours ago
5ccf0fe
Merge pull request #47 from Hasanga910/flutter_ui_updates
Hasanga910 authored 10 hours ago
Verified
07f44f6
Restyle bottom navigation and add an avatar to the Home app bar
Hasanga910 committed 10 hours ago
b9fce38
Restyle booking cards and the review header like the web
Hasanga910 committed 10 hours ago
fa6f161
Restyle package cards with photo, badges and a From price
Hasanga910 committed 10 hours ago
9524c91
Add hero banner and quick-action cards to the Home screen
Hasanga910 committed 10 hours ago
f701de7
Restyle login and register with a branded hero and card form
Hasanga910 committed 10 hours ago
7506d25
Commits on Oct 5, 2026
Add branded launcher icon and splash screen
Hasanga910 committed 10 hours ago
fb76452
Add TrailWise logo and mark to the mobile app



16.1.6 Challenges and Learning

- **Guide assigned without checking language.** This was the most serious problem I found in my part. When an Operations Manager approved a booking, the system auto-assigned any free guide, even one who did not speak the traveler's language. I changed the approval step so that only guides that pass the Guide Matching rules can be assigned. I learned that a feature can "work" in a demo and still be wrong for a real user, so I now test with realistic data such as different languages.
- **Backend tests failing at random.** Some tests passed on one run and failed on the next, which was frustrating. The cause was the coordinator workflow running in the background, so the result depended on timing and test order. [One sentence: what fixed it, e.g. waiting for the workflow to finish or isolating test data.] This taught me that asynchronous code needs tests that wait for the work to finish, not tests that assume it is already done.
- **Keeping the AI out of the final decision.** The coordinator uses an LLM to read preferences and write summaries, but the approval decision is made by fixed rules in the BookingApprovalEvaluator and then by a human. I designed it this way so that the result is predictable and cannot be changed by text a traveler types into special requests.
- **Moving the database and deploying.** Setting up Supabase and deploying to Render and Vercel caused connection and configuration problems, and once a temporary file with a login token was committed by mistake. I fixed it by rotating the secret and now I stage exact files before every commit.

16.1.7 Individual AI Usage Log

Draft from actual sessions - check every row against your own records and edit before submitting.

Date	Tool / Model	Task	What AI produced	What I changed / rejected	How I verified
3 Oct 2026	Claude (claude.ai)	Manual testing guide	Step-by-step test guide for the booking lifecycle	Used as test checklist	Followed it against the running app
5 Oct 2026	Claude (claude.ai)	UI improvement plan	10-session plan	Rejected scope outside the design document (wishlist, currency conversion); re-planned to the design document	Compared against design document sections
5 Oct 2026	Claude Code	Design system, public site (Sessions 1-2)	Components, dark mode, explorer, auth pages	Asked for smaller steps; removed off-topic Lisbon photo; fixed layout after screenshot review	Web test suite, Lighthouse, browser check
5 Oct 2026	Claude Code	Approvals and refund exception	ApprovalRequest model, endpoints, tests	Required guide-matching check on approval (doc 8.4); chose 7-day window; rejected auto-refund for early cancellation	Backend tests; PostgreSQL smoke run
5 Oct 2026	Claude Code	Workflow monitor, dashboard, tool calls	Endpoints, React pages, tool-call recorder	Required real (not invented) tool calls named per doc 8.4	Tests; monitor drill-down
5 Oct 2026	Claude Code	Tier management	Tier endpoints and editor	Changed uniqueness from class to (class, food) per doc 1; dropped add-ons	Tests
5 Oct 2026	Claude (claude.ai)	Report drafting	Filled report sections from the code	Kept evidence fields for real results only	Reviewed every section against the system

16.1.8 Individual AI Reflection

I used two AI tools in this project. I used Claude in the browser for planning: a manual testing guide, a step-by-step UI improvement plan, help with Supabase and deployment, and a first draft of parts of this report. I used Claude Code inside VS Code for implementation: the approval queue, the refund exception rule, the workflow monitor, the tier editor, the web and Flutter redesign, and many tests.

AI was most useful for speed. It could build a whole page with tests in minutes, and it often found problems I had not asked about, such as a button colour bug in light mode and missing contact number validation in the mobile app. It also explained errors clearly when my database connection failed.

AI also made mistakes, and I had to catch them. Its first UI plan included features that are not in our design document, such as a wishlist and currency conversion, so I rejected them and made it follow the document. It suggested a tier rule based only on class, which I changed to class plus food. It also allowed guides to be assigned

without checking language. I reviewed every change before committing, and I made all the commits myself so I always knew what went into the project.

I do not think I depended on AI too much. I used it as a fast assistant, but I decided what to build, what to reject and when something was finished. This project improved my understanding of the full booking flow, from the Flutter request to the agents and the approval in React. I would use Claude Code again for a project like this because its very easy to map our plan with the implementation by the help of claude code.

16.1.9 Individual Declaration

I declare that the contribution described in this section accurately represents my work, that any AI-assisted work has been disclosed in my AI usage log, and that I can explain, test and modify the submitted work attributed to me.

Name	Student ID	Signature	Date
Hasanga H N S	IT24100962		05/10/2026

16.2 Member 2 - Guide & Itinerary Management + Guide Matching Agent

16.2.1 Contribution Statement

Member 2 has primary ownership of the Guide & Itinerary Management component and the Guide Matching Agent. My contribution covers the guide data model, guide profiles, guide availability management, assigned-tour workflows, itinerary management, guide assignment support, guide-side tour lifecycle operations, React and Flutter guide interfaces, automated testing, and integration with the Agentic AI coordinator.

The Guide Matching Agent performs deterministic matching using tour specialization, traveler language preference, guide availability across the complete booking period, and buffer-day availability. It returns a structured match result to the Coordinator Agent so that guide selection can be validated safely before a booking is confirmed. I also implemented supporting services to prevent guide conflicts and double-bookings and to keep guide workflows consistent across the web and mobile applications.

16.2.2 Owned Technical Scope

Layer	Owned Work
Database	Guides, GuideAvailability and ItinerarySteps entities and database configuration. Guide records are linked to user accounts through UserId, and GuideAvailability enforces a

	unique GuideId + Date constraint to prevent duplicate availability records. Additional booking fields support attendance and guide notes.
Backend	Guide profile, guide CRUD, availability, assigned-tour, manual guide assignment, itinerary, guide notes, tour start and tour end functionality. Supporting services include GuideAvailabilityService, GuideAssignmentService and ItineraryService.
React	Tour Guide dashboard, assigned tours page, tour detail page, availability calendar, profile management and itinerary display/editing. Fleet Coordinator guide-assignment visibility and manual guide assignments are also integrated.
Flutter	Tour Guide navigation, assigned tours, tour details, itinerary viewing/editing, start/end tour workflow, attendance/completion workflow, guide notes and profile management.
Agentic AI	GuideMatchingAgent implements specialization matching, language matching, complete date-range availability checking, deterministic scoring, buffer-day scoring and safe failure when no eligible guide exists.
Testing	Backend unit/integration tests for guides, availability, guide assignment, itinerary management, Tour Guide profiles, notes and the Guide Matching Agent. React and Flutter tests cover guide dashboard/navigation, assigned tours, availability, itinerary, profile and tour-detail workflows.

16.2.3 Guide Matching Agent Contract

The Guide Matching Agent implements the GuideMatchingAgent contract and exposes MatchAsync(Guid bookingId). It returns a structured GuideMatchResult containing GuideId, MatchScore and Reasoning.

When a suitable guide is found, GuideId contains the selected guide identifier and MatchScore is a deterministic value from 0.0 to 1.0. When no suitable guide can be found, the agent returns Guid.Empty, a score of 0 and a clear human-readable reason instead of throwing an unhandled failure.

This result is passed to the Coordinator Agent and later evaluated by the deterministic booking approval workflow. A failed or insufficient guide match therefore results in safe manual review rather than an unsafe automatic assignment.

16.2.4 Matching Considerations

- Tour specialization compatibility: the guide must have a specialization matching the TourPackage theme.
- Traveler language preference: when a language preference is provided, the guide must speak the requested language.
- Complete date-range availability: the guide must be free for every date from the booking start date to the booking end date.
- Conflict prevention: unavailable dates and dates already linked to another booking are treated as conflicts.
- Buffer-day availability: availability on the day before and the day after the booking contributes to the deterministic match score.

- Deterministic scoring: when a language preference is provided, specialization and language compatibility with availability produce a base score of 0.8, with free buffer days increasing the score to 1.0. Without a language preference, the base score is 0.7 and free buffer days increase it to 1.0.
- Deterministic tie-breaking: candidates are ordered by MatchScore, then by guide name and guide ID.
- Safe failure: if the booking, package, theme or eligible guide is unavailable, the agent returns Guid.Empty with MatchScore 0 and an explanatory reason instead of causing an unhandled exception.

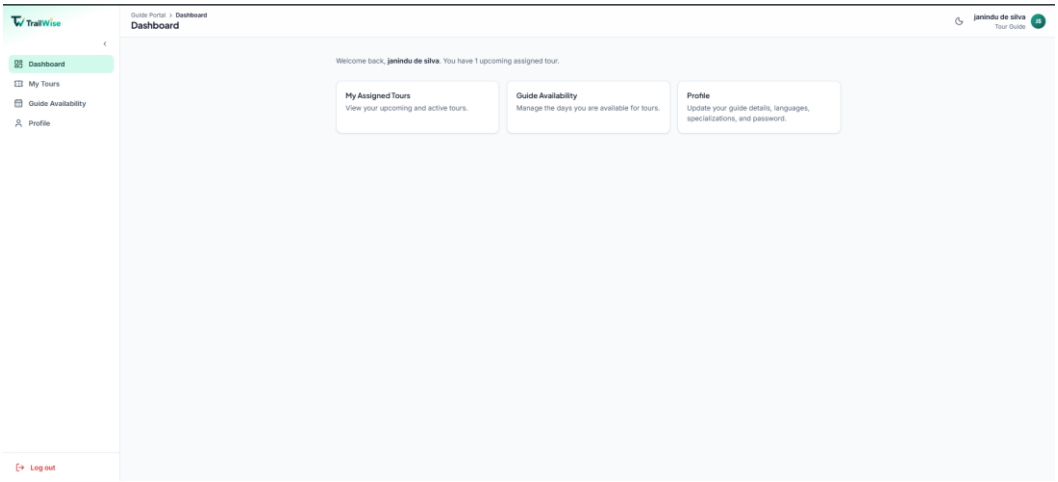
16.2.5 Key Implementation Evidence

Evidence	Details
Branch	person-2
Key commits	610e09a – Implement guide CRUD API 2c5114c – Add guide availability API and validation d607e9c – Add assigned tours API for guides 6c9c493 – Add assigned tours screen for guides 514e310 – Add guide attendance and tour notes 4ac268a – Add booking itinerary management 056d0ad – Implement real guide matching agent 547bec4 – Add transactional guide assignment protection 95b63cc – Create linked Guide profile for TourGuide users 7b31525 – Add Guide Profile 87da14d – Update Tour Guide Itinerary workflow 3eb7ea6 – Add clean Tour Guide dashboard
Pull request(s)	https://github.com/Hasanga910/TrailWise/pull/21 https://github.com/Hasanga910/TrailWise/pull/25 https://github.com/Hasanga910/TrailWise/pull/26 https://github.com/Hasanga910/TrailWise/pull/27 https://github.com/Hasanga910/TrailWise/pull/28 https://github.com/Hasanga910/TrailWise/pull/30 https://github.com/Hasanga910/TrailWise/pull/36 https://github.com/Hasanga910/TrailWise/pull/39 https://github.com/Hasanga910/TrailWise/pull/41
Backend files	GuidesController.cs BookingsController.cs GuideAvailabilityService.cs GuideAssignmentService.cs ItineraryService.cs GuideMatchingAgent.cs IGuideMatchingAgent.cs
React files	GuideDashboardPage.tsx GuideAvailabilityPage.tsx AssignedToursPage.tsx

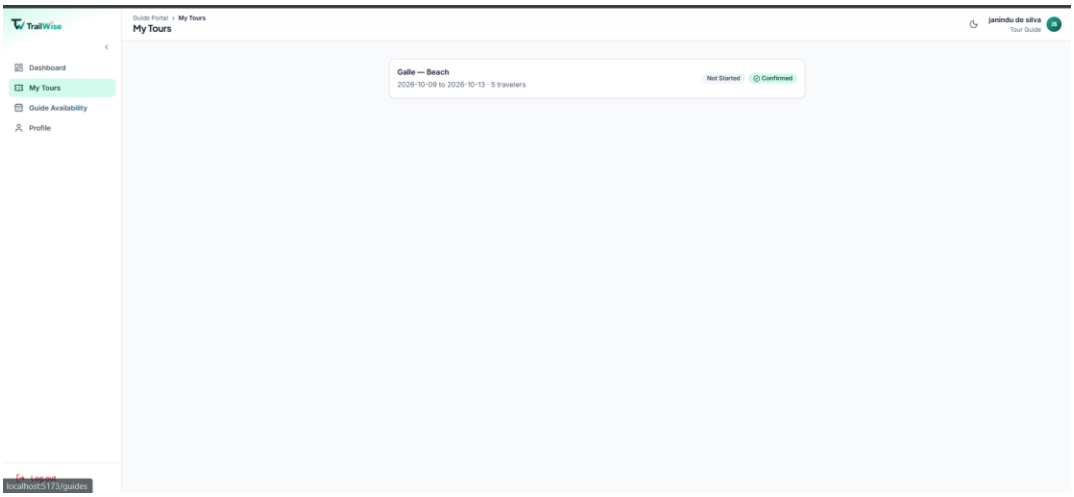
	TourDetailPage.tsx GuideProfilePage.tsx ItineraryEditor.tsx ItineraryList.tsx FleetGuideAssignmentsPage.tsx
Flutter files	assigned_tours_screen.dart tour_detail_screen.dart guide_itinerary_edit_screen.dart guide_profile_screen.dart guide_info_card.dart itinerary_screen.dart
Database migration(s)	20260922182353_AddGuideUserAndUniqueAvailability.cs 20260923104345_AddBookingAttendanceAndGuideNotes.cs
Automated tests	GuideAssignmentServiceTests.cs GuideAvailabilityEndpointsTests.cs GuideAvailabilityServiceTests.cs GuideEndpointsTests.cs GuideMatchingAgentTests.cs ItineraryEndpointsTests.cs ManualGuideAssignmentEndpointsTests.cs TourGuideCreationTests.cs TourGuideNotesEndpointsTests.cs TourGuideProfileEndpointsTests.cs

16.2.6 Screenshots / Demonstration Evidence

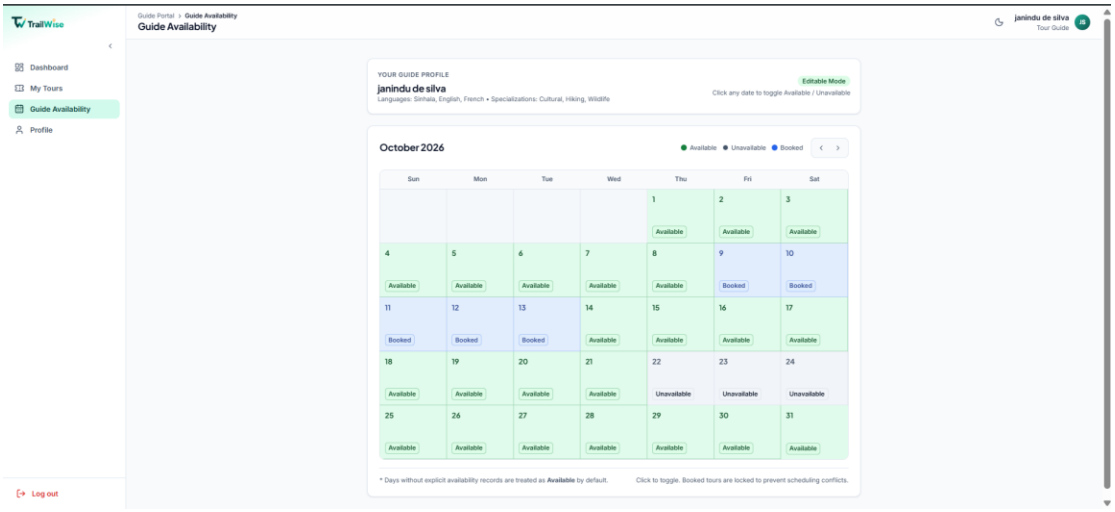
Screenshot 1 - Tour Guide Dashboard



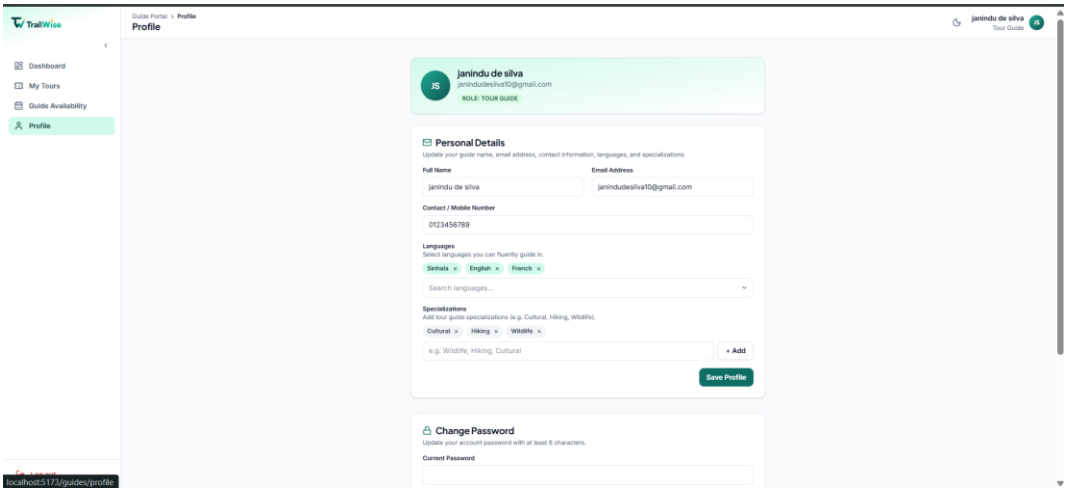
Screenshot 2 - Assigned Tours



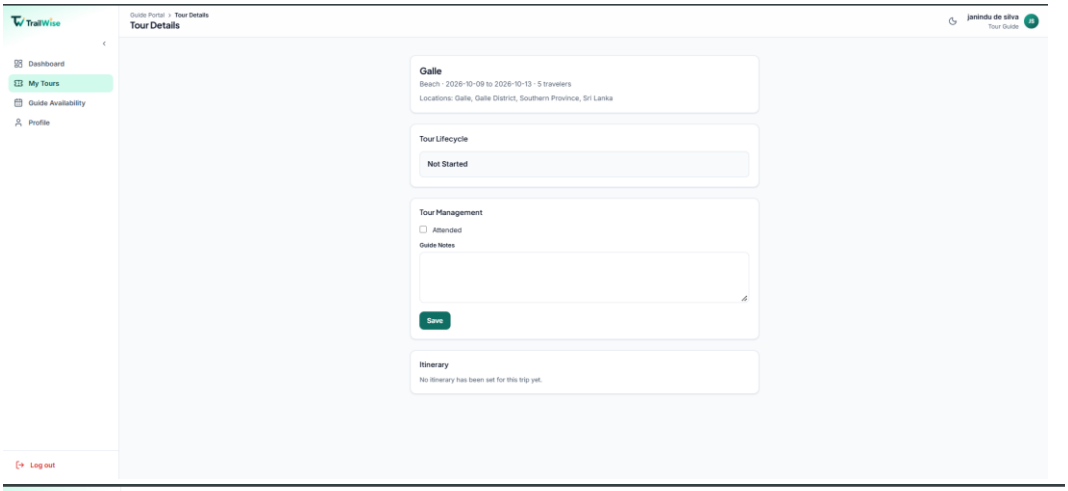
Screenshot 3 - Guide Availability Calendar

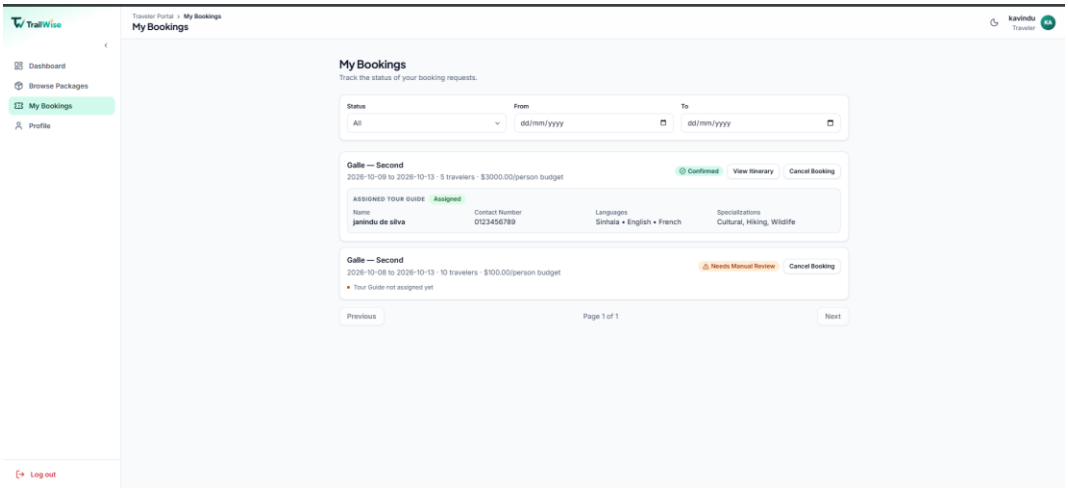


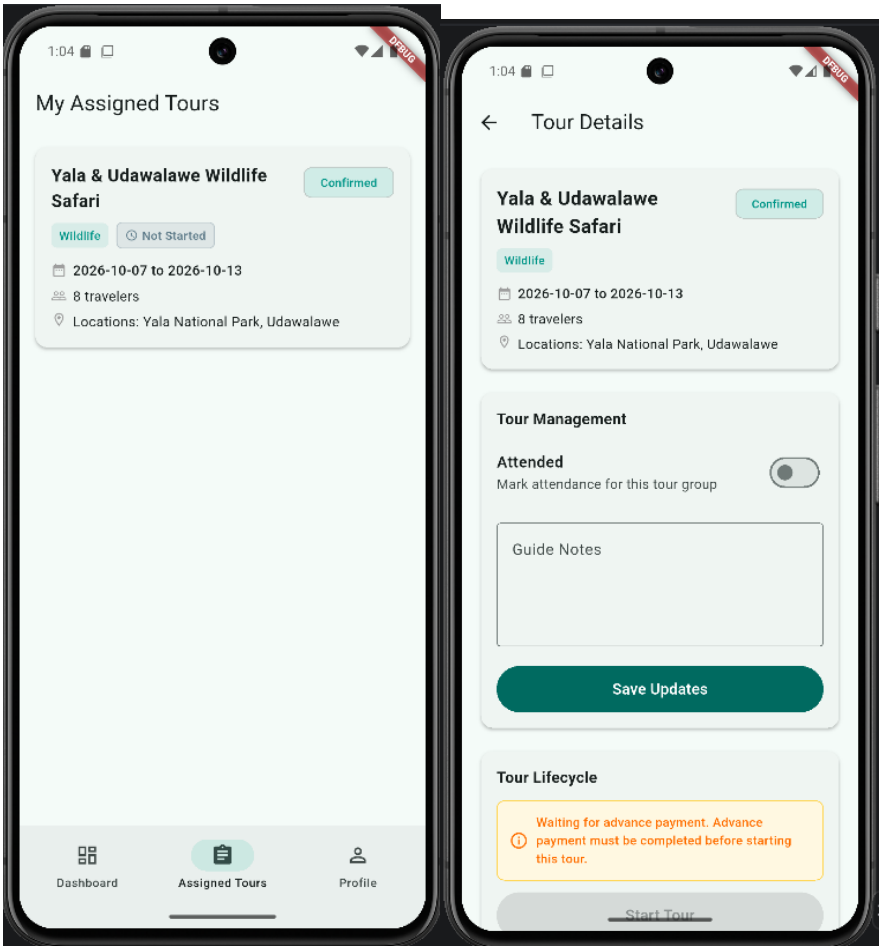
Screenshot 4 - Tour Guide Profile

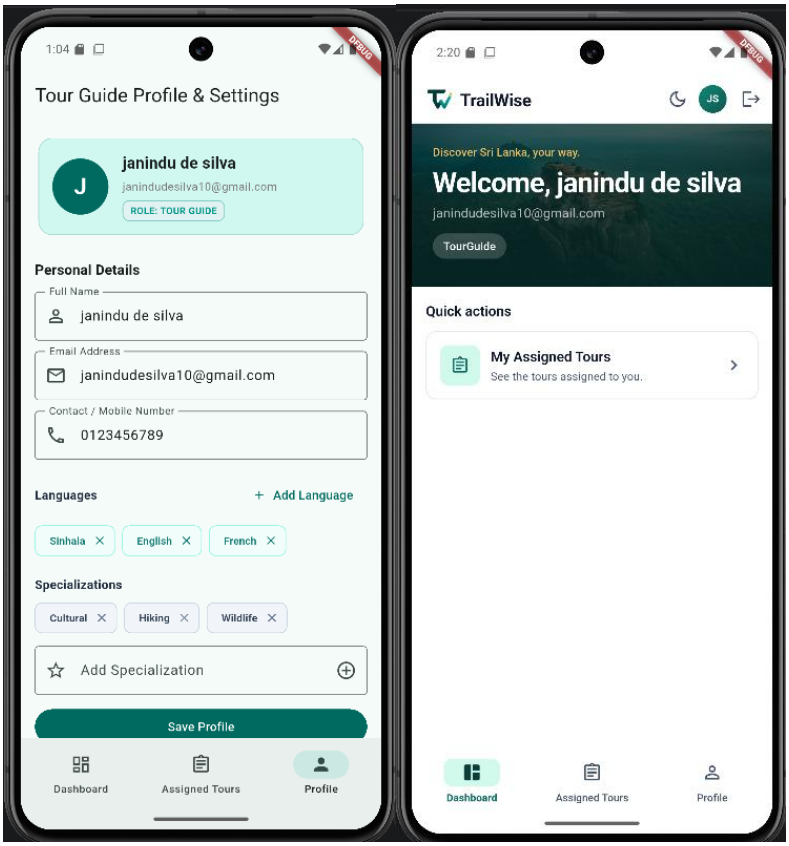


Screenshot 5 - Tour Detail and Guide Notes






1:04
DEBUG
My Assigned Tours
Yala & Udawalawe Wildlife Safari
Confirmed
Wildlife
Not Started
2026-10-07 to 2026-10-13
8 travelers
Locations: Yala National Park, Udawalawe
Dashboard
Assigned Tours
Profile
1:04
DEBUG
Tour Details
Yala & Udawalawe Wildlife Safari
Confirmed
Wildlife
2026-10-07 to 2026-10-13
8 travelers
Locations: Yala National Park, Udawalawe
Tour Management
Attended
Mark attendance for this tour group
Guide Notes
Save Updates
Tour Lifecycle
Waiting for advance payment. Advance payment must be completed before starting this tour.
Start Tour



16.2.7 Challenges and Learning

One of the main challenges was preventing guide scheduling conflicts. A guide can be assigned for several consecutive days, so checking only the start date was not sufficient. I solved this by implementing complete date-range availability checks and by enforcing a unique GuideId and Date database constraint. This helped prevent duplicate availability records and reduced the possibility of double-booking.

Another challenge was integrating my Guide Matching Agent with the booking and coordinator components developed by another team member. The agent needed to return a predictable structured result without breaking the entire workflow when no guide was available. I therefore used a deterministic GuideMatchResult containing GuideId, MatchScore and Reasoning, and returned Guid.Empty with score 0 for safe-failure cases.

Keeping the React, Flutter and ASP.NET Core models aligned was also important. Changes such as guide notes, attendance, itinerary details and guide profiles had to remain compatible across both clients. I learned the importance of using DTOs and shared API contracts rather than exposing database entities directly.

Testing also showed me that successful matching alone was not enough. I had to test unavailable guides, missing language matches, specialization mismatches, assignment conflicts, unauthorized access and itinerary validation. This improved my understanding of defensive programming and cross-platform integration.

16.2.8 Individual AI Usage Log

Date	Tool / Model	Task / Section	What AI Produced	What I Changed / Rejected	How I Verified
22 September 2026	ChatGPT	Guide Management Backend	Suggested guide CRUD, availability and assigned-tour implementation.	Adjusted it to match the TrailWise architecture and role permissions.	Reviewed the code and tested backend APIs and tests.
24 September 2026	Antigravity / Gemini	Guide Web Interface	Helped create the guide dashboard, availability, tours, details and profile UI.	Updated the UI and API integration to match the backend.	Tested the pages and ran React tests.
25 September 2026	ChatGPT / Antigravity	Guide Matching Agent	Suggested matching logic using availability, language and specialization.	Added deterministic scoring and safe-failure handling.	Tested successful and no-match scenarios.
28 September 2026	Antigravity / Gemini	Tour Guide Mobile Workflow	Helped create Flutter guide screens and workflows.	Updated navigation, validation and API calls.	Tested the Flutter app and related tests.
4 October 2026	ChatGPT	Testing and Documentation	Helped identify report content, files, commits and evidence.	Checked and corrected everything using the actual project.	Compared it with source code, Git history and system results.

16.2.9 Individual AI Reflection (Approximately One Page)

During the development of my Guide and Itinerary Management component, I used AI tools such as ChatGPT and Antigravity with Gemini. I mainly used them when I needed help understanding the existing project structure, planning functions, generating code ideas, improving the user interface and preparing tests. AI was especially useful when I was working with ASP.NET Core, React, Flutter and the Guide Matching Agent.

AI helped me complete tasks faster by giving suggestions and examples. For example, it helped me understand how to build guide CRUD functions, availability management, assigned tours, itinerary functions and guide-related user interfaces. It also helped me plan the guide matching logic using availability, language preference and specialization. These suggestions gave me a good starting point when I was not sure how to continue.

However, I did not use AI output directly without checking it. Sometimes the suggestions did not fully match the TrailWise architecture or the requirements of our project. I had to change API calls, role permissions, navigation, DTOs and some business logic. In the Guide Matching Agent, I also made sure the final logic used

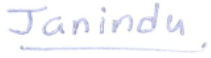
deterministic scoring and safe failure instead of depending only on AI-generated decisions.

I verified the AI-assisted work by checking the source code manually and testing the system. I used backend tests, React tests, Flutter tests, Swagger and manual testing of the web and mobile applications. I also checked my Git commits and compared the final implementation with the project document. This helped me make sure that the final work was actually working and matched the requirements.

Using AI helped me learn more about full-stack development and how different parts of a system work together. I improved my understanding of ASP.NET Core APIs, Entity Framework Core, React, Flutter, role-based access, database relationships and testing. I also learned that AI is useful as a supporting tool, but I still need to understand the code, check the results and make the final technical decisions myself.

16.2.10 Individual Declaration

I declare that the contribution described in this section accurately represents my work, that any AI-assisted work has been disclosed in my AI usage log, and that I can explain, test and modify the submitted work attributed to me.

Name	Student ID	Signature	Date
De Silva L.J.	IT 24101297		05/10/2026

16.3 Member 3 - Fleet & Transport Management + Fleet & Capacity Agent

[Rajapaksha R.A.J.K - IT24101236]

16.3.1 Contribution Statement

Member 3 owns the Fleet & Transport Management component and the Fleet & Capacity Agent. This encompasses the vehicle and driver inventory data models; backend C# reservation and cancellation services; the dedicated `/fleet/vehicles` CRUD dashboard; the redesigned central operational allocation workspace (`/fleet`) featuring smart date-aware availability fading and capacity validations; the vehicle assignments audit tracker with cancellation visibility; the dynamic driver user account provisioning flow; the package-aware cross-platform driver portal (React Web & Flutter Mobile); and the SMSGo.lk automated notification integration. Member 3 also collaborated on resolving cross-role booking lifecycle conflicts between fleet transport and tour guide assignments.

16.3.2 Owned Technical Scope

Layer	Owned work
Database	`Vehicle`, `Driver`, `VehicleAssignment` entities and cascading delete relationships; database migration updates supporting registration numbers and user account bindings.

Backend	`VehiclesController.cs`, `DriversController.cs`, `FleetReservationService.cs`, booking cancellation release logic, SmsGo notification dispatch service (`ISmsService`), and enhanced assignment query DTOs.
Agentic AI	`FleetCapacityAgent`: deterministic date-range overlap verification, seating capacity checks, ACID transactional reservations, and bundle integration with the `GuideMatchingAgent` during `PlanProposed` workflows.
React	Dedicated Vehicle Management Dashboard (`/fleet/vehicles`), redesigned Central Operational Allocation Workspace (`/fleet`), Vehicle Assignments audit page (`/fleet/assignments`), and Driver Task Dashboard (`/driver/dashboard`).
Flutter	Mobile authentication routing for drivers and the package-aware Driver Task Screen (`driver_tasks_screen.dart`) displaying itinerary highlights, vehicle specifications, and one-tap traveler calling.
Testing	Unit and integration test suites in `FleetEndpointsTests.cs`, `FleetCapacityAgentTests.cs`, and `BookingNotificationServiceTests.cs`.

16.3.3 Fleet & Capacity Agent Contract

- **Input:** A booking allocation event containing required tour dates (`startDate`, endDate), group size (`groupSize`), and package requirements (AC / non-AC preferences).

- **Output:** A deterministic validation result (`True`/`False`), a matching optimal `VehicleId` and `DriverId`, and conflict check logs. If a conflict or capacity overflow occurs, it triggers a safe fallback to `NeedsManualReview`.

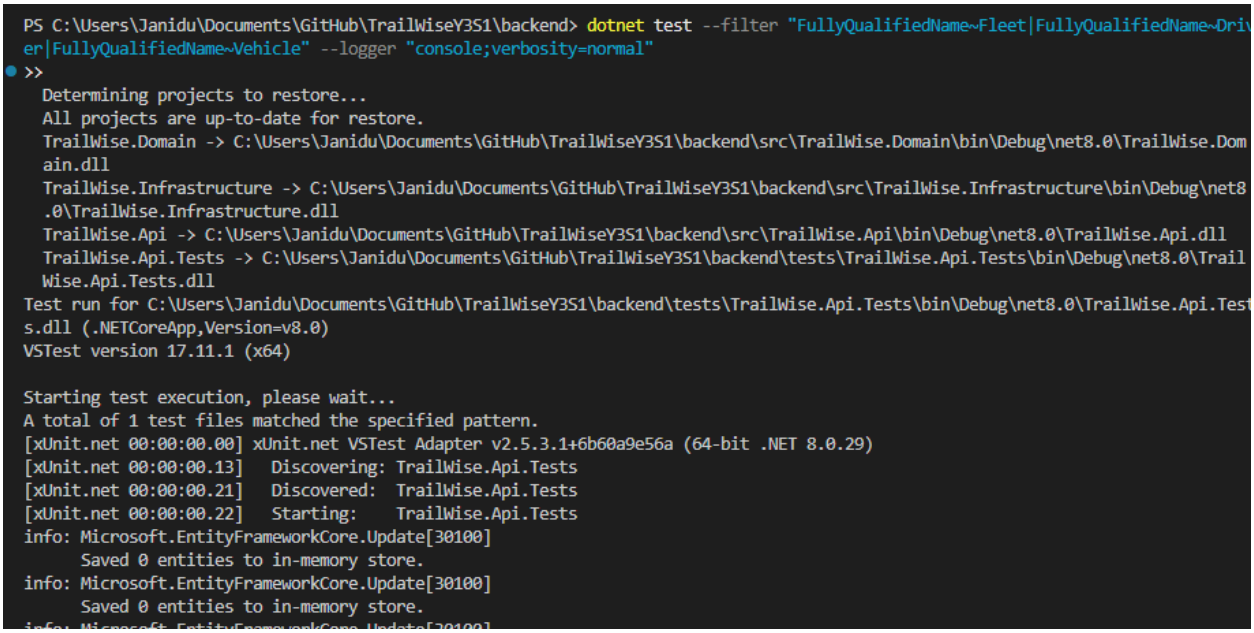
- **Tools Used:** `vehicle_availability_read`, `vehicle_availability_write` (gated behind coordinator approval), and `pricing_calculator`.

16.3.4 Key Implementation Evidence

Evidence	**Details**
Branch	`feature/fleet-management`
Key commits	• Add dedicated vehicle inventory dashboard (`/fleet/vehicles..`) • implement smart date-aware vehicle/guide availability fading in `/fleet` • Add dynamic driver user account provisioning on registration • Integrate SMSGo.lk API for traveler confirmation notifications

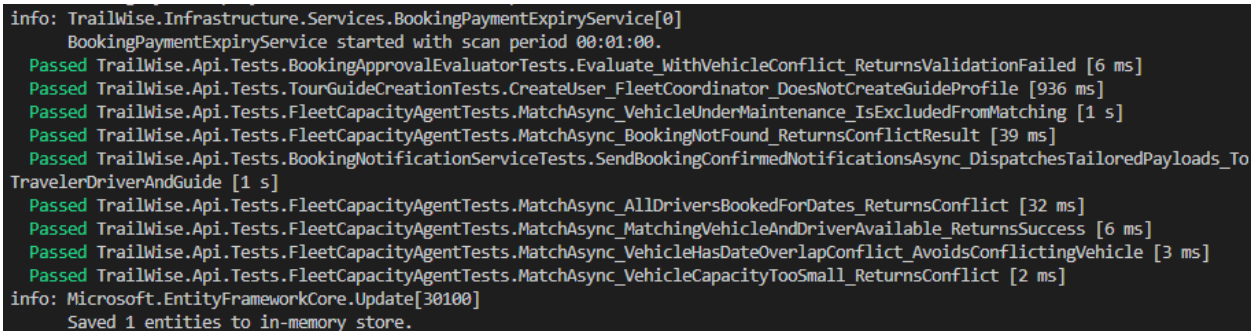
Pull requests	https://github.com/Hasanga910/TrailWise/pull/43, https://github.com/Hasanga910/TrailWise/pull/41 https://github.com/Hasanga910/TrailWise/pull/39 https://github.com/Hasanga910/TrailWise/pull/36 https://github.com/Hasanga910/TrailWise/pull/30 https://github.com/Hasanga910/TrailWise/pull/28 https://github.com/Hasanga910/TrailWise/pull/27 https://github.com/Hasanga910/TrailWise/pull/26 https://github.com/Hasanga910/TrailWise/pull/25 https://github.com/Hasanga910/TrailWise/pull/21
Backend files	**VehiclesController.cs, DriversController.cs, BookingsController.cs, Vehicle.cs, Driver.cs, VehicleAssignment.cs, VehicleMaintenanceStatus.cs, VehicleType.cs, FleetReservationService.cs, CoordinatorAgentService.cs, FleetEndpointsTests.cs**
React files	**FleetManagementPage.tsx, DriverDashboardPage.tsx, FleetManager.tsx, vehicles.ts, bookings.ts, App.tsx**
Automated tests	`FleetCapacityAgentTests.cs`, `FleetEndpointsTests.cs`, `BookingNotificationServiceTests.cs` (All tests passing)

16.3.5 Screenshots / Demonstration Evidence

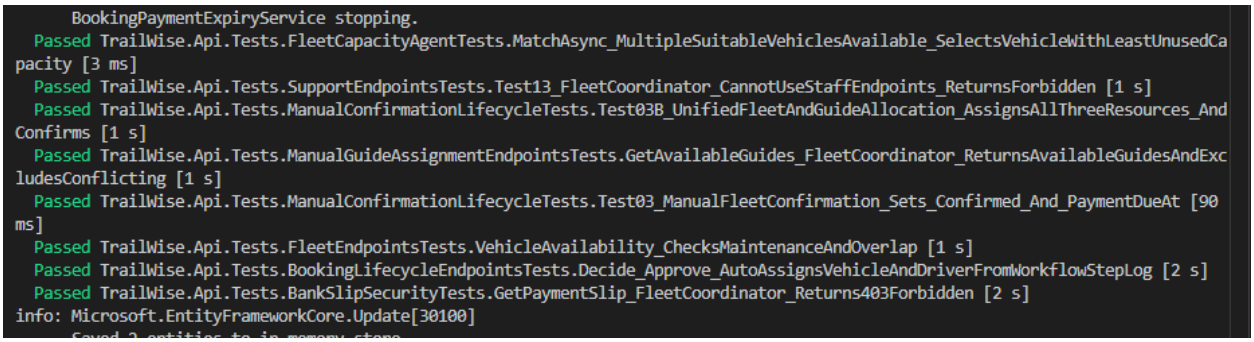


```
PS C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend> dotnet test --filter "FullyQualifiedName~Fleet|FullyQualifiedName~Driver|FullyQualifiedName~Vehicle" --logger "console;verbosity=normal"
>>
  Determining projects to restore...
  All projects are up-to-date for restore.
  TrailWise.Domain -> C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\src\TrailWise.Domain\bin\Debug\net8.0\TrailWise.Domain.dll
  TrailWise.Infrastructure -> C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\src\TrailWise.Infrastructure\bin\Debug\net8.0\TrailWise.Infrastructure.dll
  TrailWise.Api -> C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\src\TrailWise.Api\bin\Debug\net8.0\TrailWise.Api.dll
  TrailWise.Api.Tests -> C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\tests\TrailWise.Api.Tests\bin\Debug\net8.0\TrailWise.Api.Tests.dll
Test run for C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\tests\TrailWise.Api.Tests\bin\Debug\net8.0\TrailWise.Api.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.5.3.1+6b60a9e56a (64-bit .NET 8.0.29)
[xUnit.net 00:00:00.13]   Discovering: TrailWise.Api.Tests
[xUnit.net 00:00:00.21]   Discovered:  TrailWise.Api.Tests
[xUnit.net 00:00:00.22]   Starting:    TrailWise.Api.Tests
info: Microsoft.EntityFrameworkCore.Update[30100]
      Saved 0 entities to in-memory store.
info: Microsoft.EntityFrameworkCore.Update[30100]
      Saved 0 entities to in-memory store.
```



```
info: TrailWise.Infrastructure.Services.BookingPaymentExpiryService[0]
      BookingPaymentExpiryService started with scan period 00:01:00.
  Passed TrailWise.Api.Tests.BookingApprovalEvaluatorTests.Evaluate_WithVehicleConflict_ReturnsValidationFailed [6 ms]
  Passed TrailWise.Api.Tests.TourGuideCreationTests.CreateUser_FleetCoordinator_DoesNotCreateGuideProfile [936 ms]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_VehicleUnderMaintenance_IsExcludedFromMatching [1 s]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_BookingNotFound_ReturnsConflictResult [39 ms]
  Passed TrailWise.Api.Tests.BookingNotificationServiceTests.SendBookingConfirmedNotificationsAsync_DispatchesTailoredPayloads_ToTravelerDriverAndGuide [1 s]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_AllDriversBookedForDates_ReturnsConflict [32 ms]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_MatchingVehicleAndDriverAvailable_ReturnsSuccess [6 ms]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_VehicleHasDateOverlapConflict_AvoidsConflictingVehicle [3 ms]
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_VehicleCapacityTooSmall_ReturnsConflict [2 ms]
info: Microsoft.EntityFrameworkCore.Update[30100]
      Saved 1 entities to in-memory store.
```

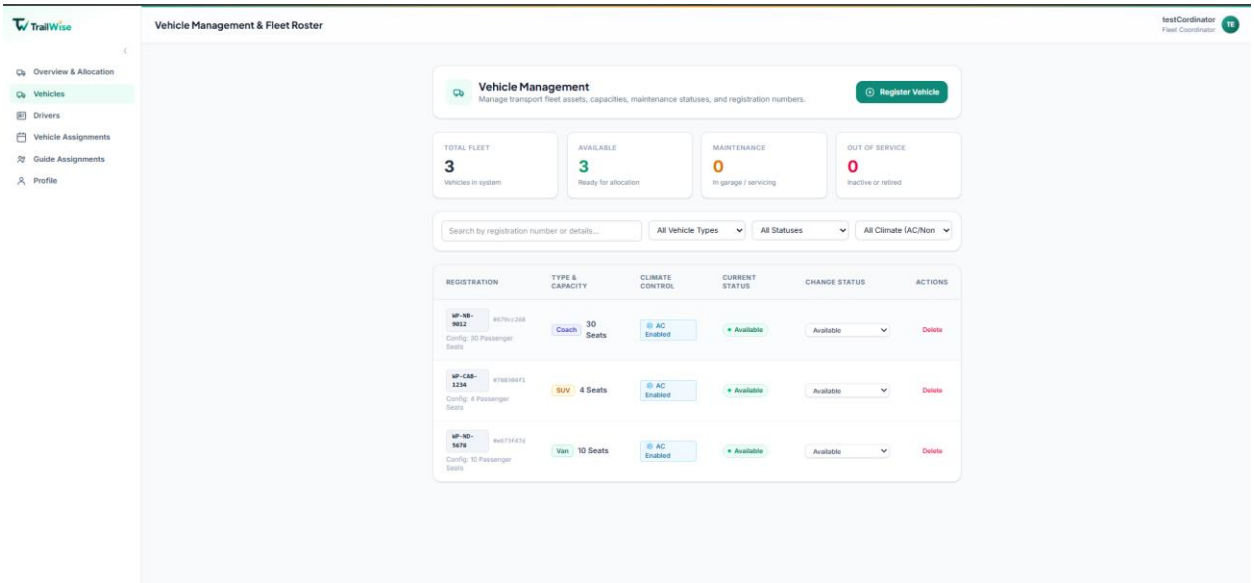


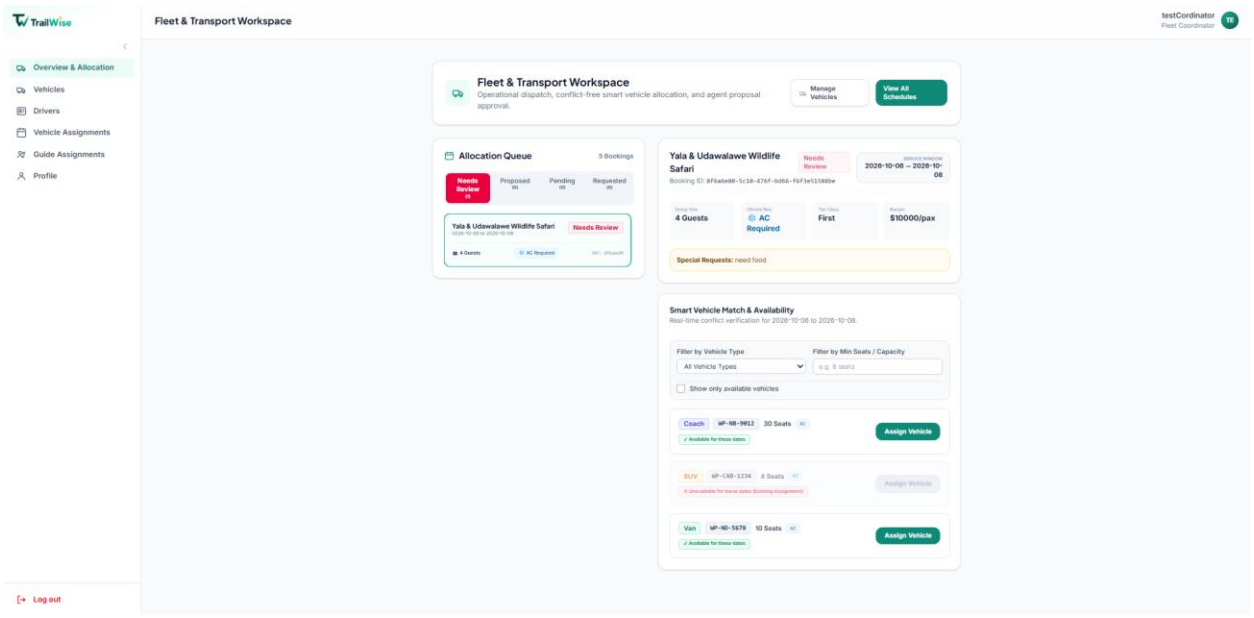
```
      BookingPaymentExpiryService stopping.
  Passed TrailWise.Api.Tests.FleetCapacityAgentTests.MatchAsync_MultipleSuitableVehiclesAvailable_SelectsVehicleWithLeastUnusedCapacity [3 ms]
  Passed TrailWise.Api.Tests.SupportEndpointsTests.Test13_FleetCoordinator_CannotUseStaffEndpoints_ReturnsForbidden [1 s]
  Passed TrailWise.Api.Tests.ManualConfirmationLifecycleTests.Test03B_UnifiedFleetAndGuideAllocation_AssignsAllThreeResources_AndConfirms [1 s]
  Passed TrailWise.Api.Tests.ManualGuideAssignmentEndpointsTests.GetAvailableGuides_FleetCoordinator_ReturnsAvailableGuidesAndExcludesConflicting [1 s]
  Passed TrailWise.Api.Tests.ManualConfirmationLifecycleTests.Test03_ManualFleetConfirmation_Sets_Confirmed_And_PaymentDueAt [90 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.VehicleAvailability_ChecksMaintenanceAndOverlap [1 s]
  Passed TrailWise.Api.Tests.BookingLifecycleEndpointsTests.Decide_Approve_AutoAssignsVehicleAndDriverFromWorkflowStepLog [2 s]
  Passed TrailWise.Api.Tests.BankSlipSecurityTests.GetPaymentSlip_FleetCoordinator_Returns403Forbidden [2 s]
info: Microsoft.EntityFrameworkCore.Update[30100]
```

```
9.
  Passed TrailWise.Api.Tests.ManualGuideAssignmentEndpointsTests.AssignGuide_FleetCoordinator_AssignsGuideToNeedsManualReviewBook
ing_ConfirmsAndReservesAvailability [318 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateDriver_WithEmailAndPassword_ProvisionsUserAccountAndEnablesLogin [311 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.GetAllVehicles_Anonymous_ReturnsOkWithFilterSupport [84 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateDriver_DuplicateLicenseNumber_ReturnsConflictWithErrorMessage [156 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.Driver_CanLogin_And_QueryAssignedToursPortal [347 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CancelBooking_ReleasesVehicleAssignment_RestoresVehicleAvailability [326 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.UpdateMaintenanceStatus_UpdatesStatusCorrectly [73 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.DeleteVehicle_AsAdminOrCoordinator_CascadesAssignmentsAndRemovesVehicle [63 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.GetAssignmentByBookingId_TravelerCanRetrieveTheirAssignment [196 ms]
info: Microsoft.EntityFrameworkCore.Update[30100]
      Saved 1 entities to in-memory store.
info: Microsoft.EntityFrameworkCore.Update[30100]
      at TrailWise.Infrastructure.Agents.Coordinator.CoordinatorAgentService.StartWorkflowAsync(Guid bookingId, CancellationToken ct) in C
:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\src\TrailWise.Infrastructure\Agents\Coordinator\CoordinatorAgentService.cs:
line 319
      at TrailWise.Api.Controllers.BookingsController.<>c__DisplayClass24_0.<<DispatchCoordinatorWorkflow>b__0>d.MoveNext() in
C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend\src\TrailWise.Api\Controllers\BookingsController.cs:line 161
  Passed TrailWise.Api.Tests.FleetEndpointsTests.UpdateDriver_UpdatesDetails_And_DeleteDriver_RemovesDriver [215 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateVehicle_MissingRegistrationNumber_ReturnsBadRequest [166 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateVehicle_DuplicateRegistrationNumber_ReturnsConflict [172 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.DriversEndpoints_SupportGetAllGetByIdAndCreate [65 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateVehicle_FleetCoordinator_ReturnsCreated [170 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CreateVehicle_UnauthorizedOrForbidden_WhenNotFleetCoordinatorOrAdmin [113 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.GetAssignments_ReturnsAssignmentsWithDetails [228 ms]
  Passed TrailWise.Api.Tests.FleetEndpointsTests.CheckDriverAvailability_ReturnsTrue_AndFalseWhenConflicted [198 ms]

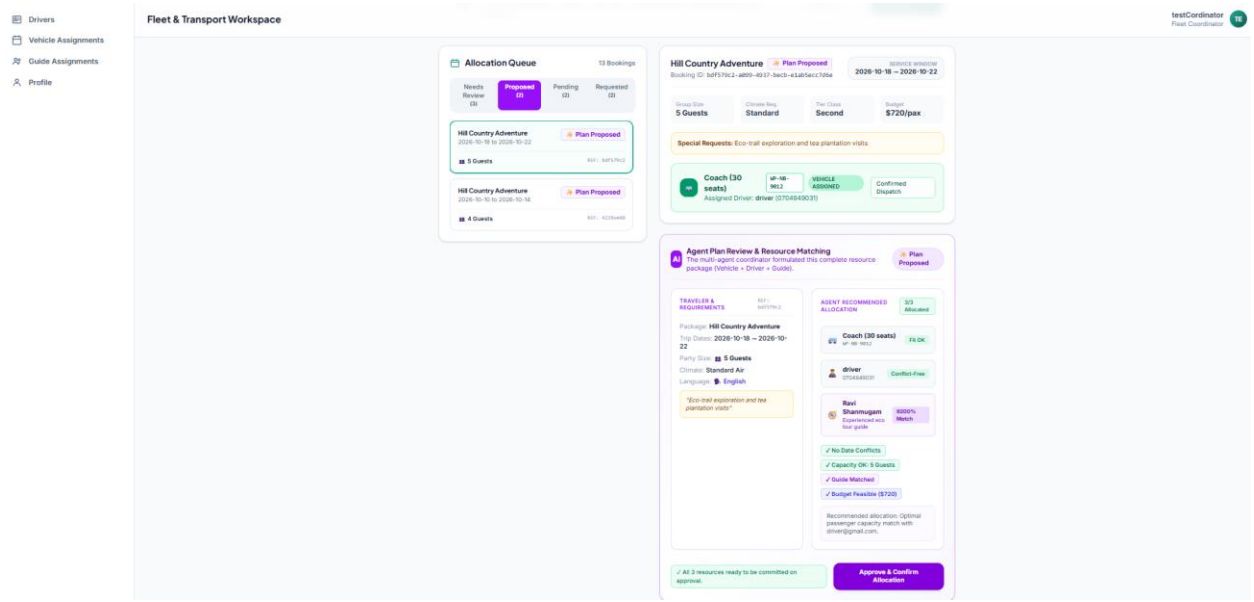
Test Run Successful.
Total tests: 34
     Passed: 34
 Total time: 5.6854 Seconds
PS C:\Users\Janidu\Documents\GitHub\TrailWiseY3S1\backend>
```

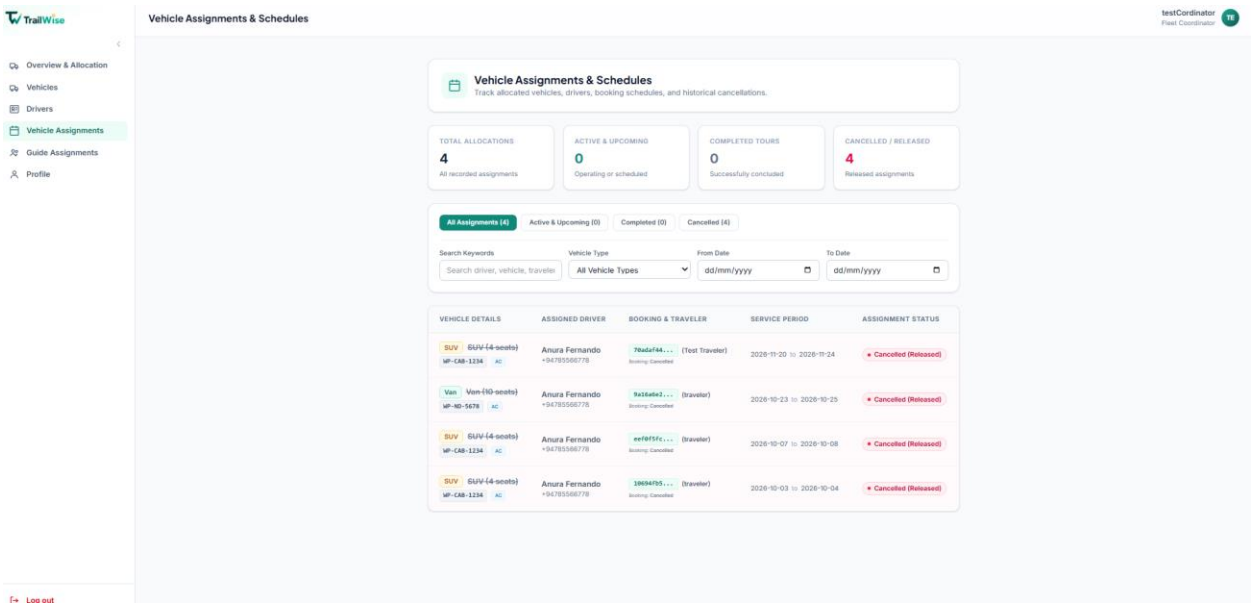
Screenshot 2: Dedicated Vehicle Management Dashboard (/fleet/vehicles)

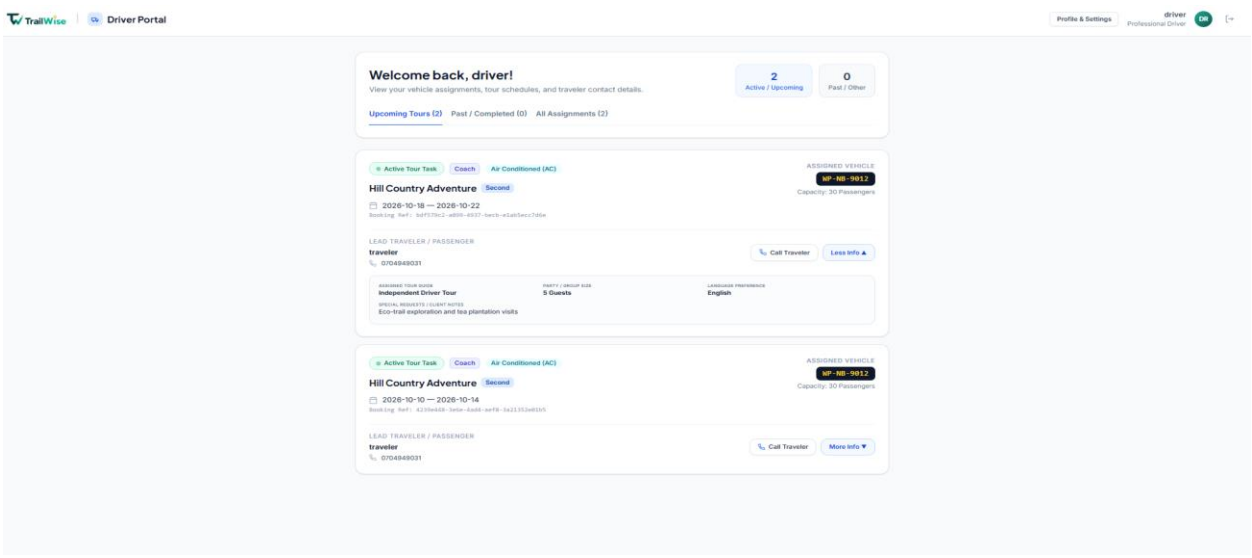




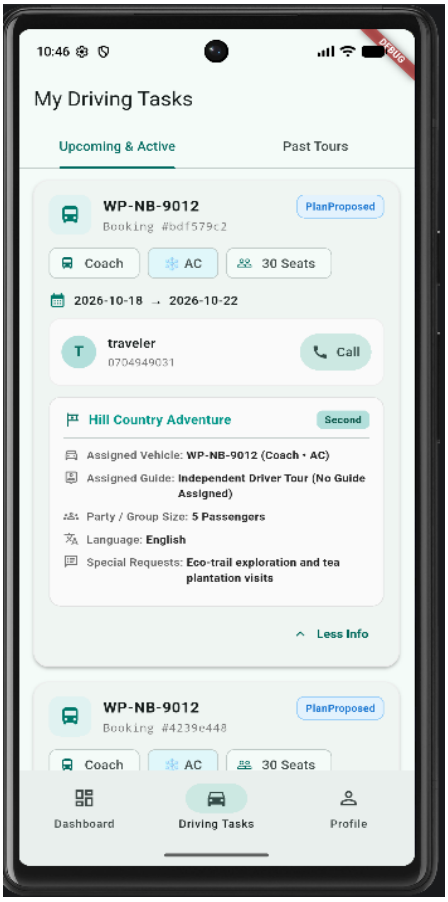
Screenshot 4: Agent Proposal Review Card (PlanProposed)




TrailWise
Vehicle Assignments & Schedules
testCordinator
Fleet Coordinator
TE
Overview & Allocation
Vehicles
Drivers
Vehicle Assignments
Guide Assignments
Profile
Log out
Vehicle Assignments & Schedules
Track allocated vehicles, drivers, booking schedules, and historical cancellations.
TOTAL ALLOCATIONS
4
All recorded assignments
ACTIVE & UPCOMING
0
Operating or scheduled
COMPLETED TOURS
0
Successfully concluded
CANCELLED / RELEASED
4
Released assignments
All Assignments (4)
Active & Upcoming (0)
Completed (0)
Cancelled (4)
Search Keywords
Search driver, vehicle, traveler
Vehicle Type
All Vehicle Types
From Date
dd/mm/yyyy
To Date
dd/mm/yyyy
VEHICLE DETAILS
ASSIGNED DRIVER
BOOKING & TRAVELER
SERVICE PERIOD
ASSIGNMENT STATUS
SUV
SUV (4 seats)
WP-CAB-1234
AC
Anura Fernando
+9478556778
70ada744...
(Test Traveler)
Booking: Cancelled
2026-11-20 to 2026-11-24
Cancelled (Released)
Van
Van (10 seats)
WP-ND-5678
AC
Anura Fernando
+9478556778
9a16a8e2...
(traveler)
Booking: Cancelled
2026-10-23 to 2026-10-25
Cancelled (Released)
SUV
SUV (4 seats)
WP-CAB-1234
AC
Anura Fernando
+9478556778
eef0f5fc...
(traveler)
Booking: Cancelled
2026-10-07 to 2026-10-08
Cancelled (Released)
SUV
SUV (4 seats)
WP-CAB-1234
AC
Anura Fernando
+9478556778
10694fb5...
(traveler)
Booking: Cancelled
2026-10-03 to 2026-10-04
Cancelled (Released)


TrailWise
Driver Portal
Profile & Settings
driver
Professional Driver
DR
Welcome back, driver!
View your vehicle assignments, tour schedules, and traveler contact details.
2
Active / Upcoming
0
Past / Other
Upcoming Tours (2)
Past / Completed (0)
All Assignments (2)
Active Tour Task
Coach
Air Conditioned (AC)
Hill Country Adventure
Second
2026-10-18 — 2026-10-22
ASSIGNED VEHICLE
WP-NB-9812
Capacity: 30 Passengers
LEAD TRAVELER / PASSENGER
traveler
0704949031
Call Traveler
Less Info ▲
Independent Driver Tour
5 Guests
English
Eco-trail exploration and tea plantation visits
Active Tour Task
Coach
Air Conditioned (AC)
Hill Country Adventure
Second
2026-10-10 — 2026-10-14
ASSIGNED VEHICLE
WP-NB-9812
Capacity: 30 Passengers
LEAD TRAVELER / PASSENGER
traveler
0704949031
Call Traveler
More Info ▼

Screenshot 7: Driver Mobile View – Flutter App



16.3.6 Challenges and Learning

- **Preventing Double-Booking via Date-Range Overlaps:** Implementing robust temporal checking was essential. We had to write precise LINQ expressions in C# (`Start A <= End B AND End A >= Start B`) against PostgreSQL to ensure that vehicles and guides assigned to overlapping dates are instantly recognized, dimmed, and blocked from concurrent assignment.

- **Dynamic Driver User Provisioning:** Initially, drivers were treated purely as isolated table entities without login credentials. We restructured the `DriversController` so that when a fleet coordinator registers a driver with an email and password, the backend transactionally generates or links a corresponding system user account (`UserRole.Driver`), enabling secure cross-platform login.

- **Resolving Cross-Department Role Conflicts:** A major architectural challenge arose when the Operations Manager assigned guides (making a booking `Confirmed`) while the Fleet Coordinator hadn't yet assigned vehicles and drivers. We resolved this cleanly by consolidating tour guide assignment into the Fleet Coordinator's unified operational workspace, ensuring a single transactional confirmation gate.

- **Local SMS Gateway Integration:** Configuring the SMSGo.lk API required mapping Sri Lankan phone formats (converting local `077...` headers to standard `9477...` international strings) and handling API keys securely through environment configurations (`SmsGo_API_KEY`).

16.3.7 Individual AI Usage Log

Date	Tool / Model	Task	What AI Produced	What I Changed / Rejected	How I Verified
Sep 2026	Antigravity / gemini	Vehicle CRUD & Metrics Bar	Scaffolded `FleetVehiclesPage.tsx` layout and backend controllers	Added inline maintenance status patching and deletion modals	Compiled frontend build and tested
Sep 2026	Antigravity / gemini	Smart Allocation Workspace (`/fleet`)	Date-overlap conflict logic and CSS fading styles	Ensured capacity violation blocks assignment completely	Ran backend integration tests (`dotn test`)
Oct 2026	Antigravity / gemini	Driver User Account Linking	Controller logic to bind Driver records to User accounts	Added password hashing and role enforcement for `UserRole.Driver`	Executed unit test `CreateDriver_WithEmailAndPas`
Oct 2026	GPT/ Notify	Notify.lk Integration	`ISmsService` implementation using `HttpClient`	Added phone number cleaner to enforce `94XXXXXXXXX` formatting	Tested with sandbox API key and mo assertions
Oct 2026	Antigravity / gemini	Consolidating Guide Assignment	Updated allocation drawer to include guide dropdowns with overlap checks	Ensured booking status transitions to `Confirmed` only when resource bundles are complete	Verified via end-to-end workflow execution

16.3.8 Individual AI Reflection

During my development of the Fleet & Transport Management module in TrailWise, I used AI helpers (gemini via Antigravity and gpt) as a collaborator programming buddy. During the architectural phase, AI was most

useful in helping me create repetitive boilerplate codes such as Entity Framework Core migrations config, DTO contracts, and responsive Tailwind CSS layout grids for the fleet management dashboard pages (/fleet/vehicles and /fleet management).

But AI did need strict supervision about business logic correctness and domain rules. For example, the first attempt of vehicle availability checking was missing important boundary conditions which could allow double-bookings when making concurrent schedule bookings. I helped improve this problem through manual editing and implementing proper date-math SQL conditions. In another case when implementing the Notify.lk service, the AI was attempting to format local mobile number using the standard domestic prefix (07...). I changed this behavior to use the international (94...) mobile number prefix instead, through coding a custom normalizer function.

Working with AI significantly increased my speed, giving more time to concentrate on transactional consistencies (safely assign vehicles and release drivers and guides atomically).

16.3.9 Individual Declaration -

I declare that the contribution described in this section accurately represents my work, that any AI-assisted work has been disclosed in my AI usage log, and that I can explain, test and modify the submitted work attributed to me.

Name	Student ID	Signature	Date
Rajapaksha R.A.J.K	IT24101236		05/10/2026

16.4 Member 4 - Payments, Reviews & Reporting + Pricing & Validation Agent

Name: Matharaarachchi D.C.M

Student ID: IT24101252

16.4.1 Contribution Statement

Member 4 owns the Payments, Reviews and Reporting component together with the Pricing & Validation Agent. My work mainly focused on the financial side of TrailWise, including deterministic booking price calculation, group discounts, bank-transfer payments, payment verification, payment deadlines, traveler reviews, reporting functions and audit logging.

I also worked on the customer support functionality and several shared booking and payment lifecycle improvements. The Pricing & Validation Agent is integrated into the coordinator workflow and produces the authoritative tour cost and pricing breakdown that is later used by the payment system.

16.4.2 Owned Technical Scope

Layer	Owned Work
Database	Payments, Discounts, Reviews and AuditLogs. Also contributed to SupportTickets and SupportMessages. Added payment deadline fields such as PaymentDueAt and BalancePaymentDueAt and discount validity fields such as IsActive, ValidFrom and ValidUntil.
Backend	PaymentService, ReviewService, AuditLogService, OperationsReportService, discount services and related API controllers. Implemented bank-transfer payment submission, private slip handling, payment approval/rejection, payment status, review submission, discount management, reporting and support integration.
Agentic AI	PricingValidationAgent: deterministic price calculation using package tier price, group size, catering, add-ons and group discounts. Also validates budget and operational constraints and returns TotalCost, Breakdown and ValidationResult.
React	Operations Manager functions for payment verification, discount management, reports, support and related operational views.
Flutter	Traveler payment-status screen, bank-slip selection/upload, payment deadline countdowns, financial summary, detailed cost breakdown, review submission, discount information and support integration.
Testing	Backend unit/integration tests for payments, reviews, discounts, reporting, audit logging and PricingValidationAgent. Flutter widget/API tests for payment and review flows and React tests for discount and operational functions.

16.4.3 Pricing & Validation Agent Contract

Input: The agent receives the booking ID together with the Guide Matching Agent result and Fleet & Capacity Agent result. It loads the selected package tier, group size, duration, selected add-ons and currently valid global discounts.

Main calculations:

Base Cost = PackageTier.BasePricePerPerson × GroupSize

Catering Cost = 15 × GroupSize × DurationDays, when food is included in the selected tier.

Add-ons Cost = Sum of selected BookingAddOn costs.

Subtotal = Base Cost + Catering Cost + Add-ons Cost

Discount Amount = Subtotal × applicable discount percentage

Final Total = Subtotal - Discount Amount

The agent selects the best eligible active discount based on minimum group size and validity period. The pricing calculation is implemented with deterministic C# logic instead of an LLM so that the same input produces the same financial result.

Output:

```
{
    TotalCost,
    Breakdown,
    ValidationResult
}
```

The breakdown contains:

tierBasePrice
cateringCost
addOnsCost
subtotal
discountDescription
discountPercentage
groupDiscount
finalTotal

Validation results are:

Valid
NeedsApproval
Failed

The pricing result is stored in `AgentStepLogs`, which provides a permanent pricing snapshot for the booking. The payment service later reads this stored result instead of recalculating the booking price.

16.4.4 Key Implementation Evidence

Evidence	Details
Branch	`person-4`
Pull Request	Person 4 pull request merged into dev through PR #33
Core Backend Files	PricingValidationAgent.cs, PaymentService.cs, ReviewService.cs, DiscountService.cs, AuditLogService.cs, OperationsReportService.cs, PaymentsController.cs, ReviewsController.cs, DiscountsController.cs, ReportsController.cs
Core Domain / DTO Areas	Payment, Discount, Review, AuditLog, payment-status and pricing-breakdown DTOs
Flutter Files	payment_status_screen.dart, payment_status.dart, my_bookings_screen.dart, booking_request_screen.dart and related review/support screens
React Areas	DiscountManager.tsx, operations payment/report/support screens and related API clients
Automated Testing	PricingValidationAgent tests, payment endpoint/service tests, review endpoint tests, audit log tests, discount validity/pricing-breakdown tests, reporting tests and Flutter/React tests

16.4.5 Main Functional Implementation

Payment processing was implemented as a controlled bank-transfer flow.

A traveler can submit a bank transfer and upload the payment slip. The uploaded slip is stored in private storage rather than exposed as a public file. Only the traveler who owns the payment and authorized staff can access it. The system restricts uploaded slips to JPG, PNG, WEBP and PDF formats with a maximum size of 5 MB. TRAILWISE

The payment initially remains pending until an Operations Manager or Admin verifies it. Staff can approve or reject the payment, and rejected payments can contain a reason.

For the first payment, the traveler must pay at least 50% of the authoritative tour cost. A confirmed booking receives an advance payment deadline and the Flutter app displays a live countdown. After the tour is completed, any remaining balance can be paid within the final balance period.

The Payment Status screen displays both the payment progress and the original pricing calculation:

Base Cost
Catering
Add-ons
Subtotal
Group Discount
Final Total

This makes the price transparent to the traveler and uses the same pricing snapshot originally generated by the Pricing Validation Agent.

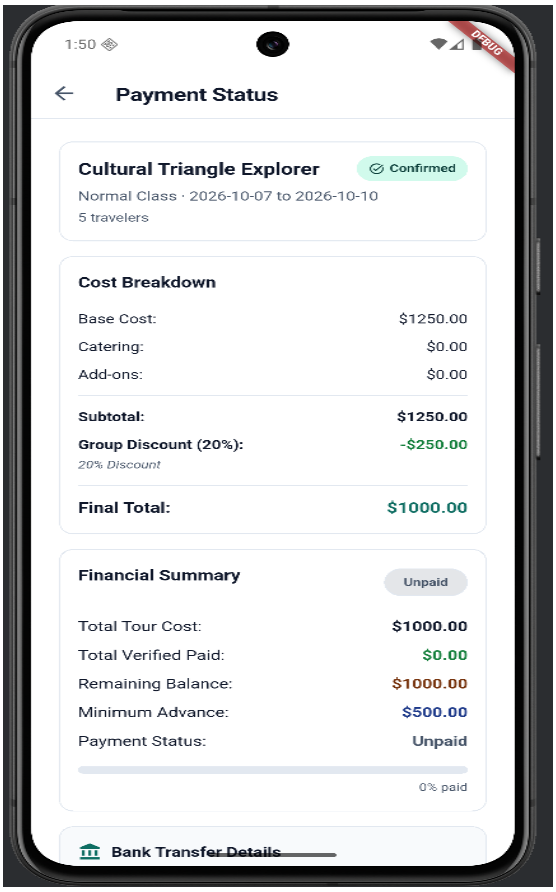
Reviews are only allowed after the booking is completed and fully paid. The service checks booking ownership, prevents duplicate reviews and validates the rating. Public package reviews display the average rating and review count without exposing private booking information.

The discount system supports global group-size discounts. Discounts can be activated or deactivated and can contain optional start and expiry dates. The Pricing Validation Agent only considers currently active and valid discounts.

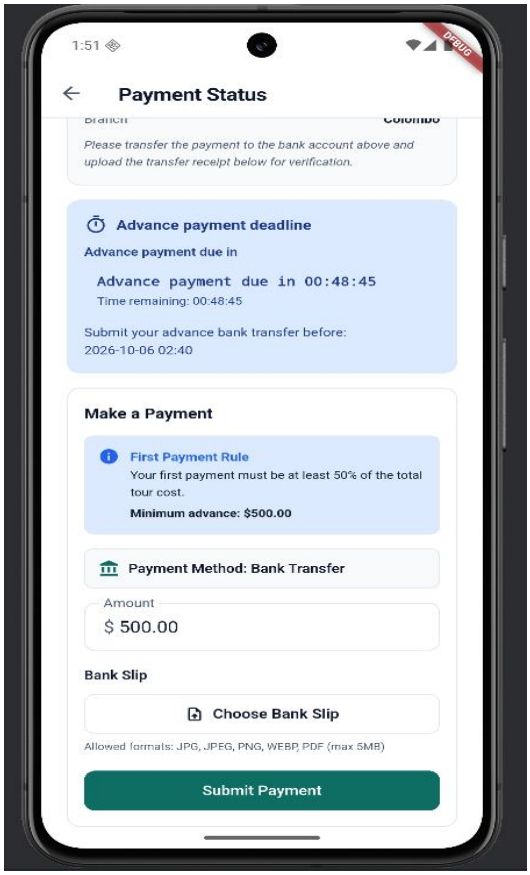
16.4.6 Screenshots / Demonstration Evidence

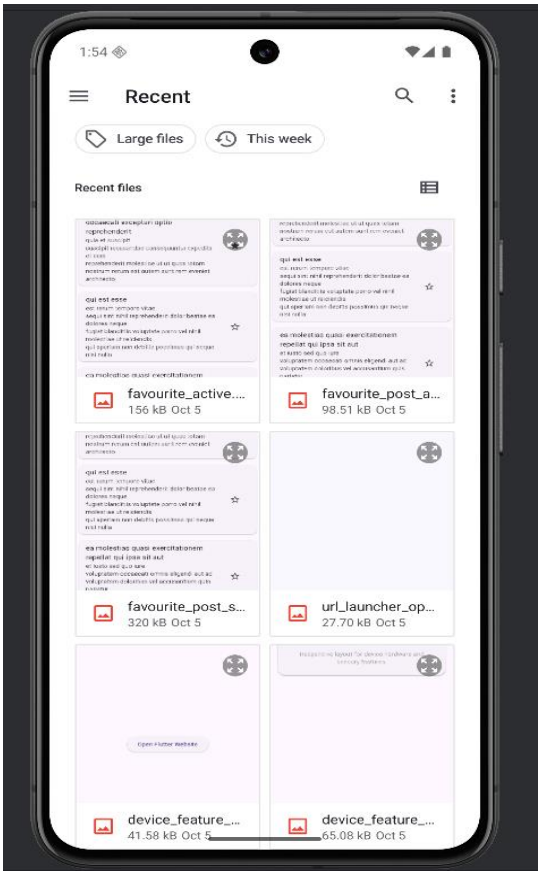
For my individual section I will include screenshots showing:

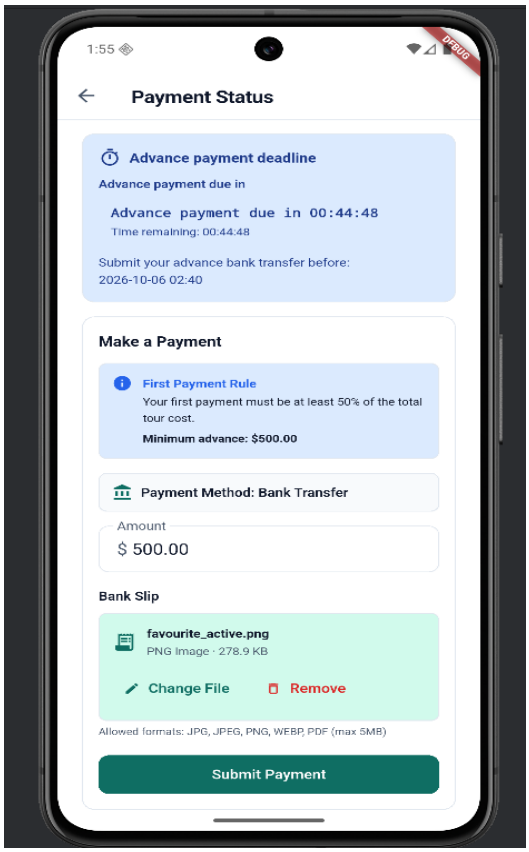
1. Traveler Payment Status screen with Financial Summary and Cost Breakdown.



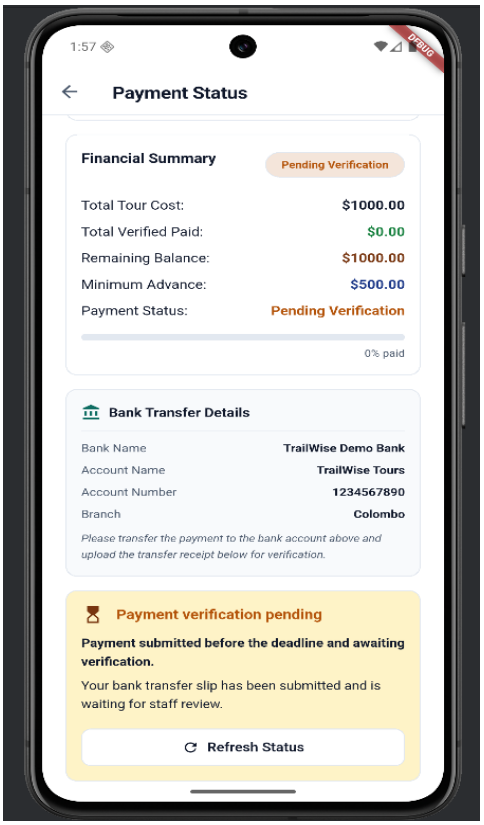
2. Advance payment countdown, Bank-transfer slip selection and upload.

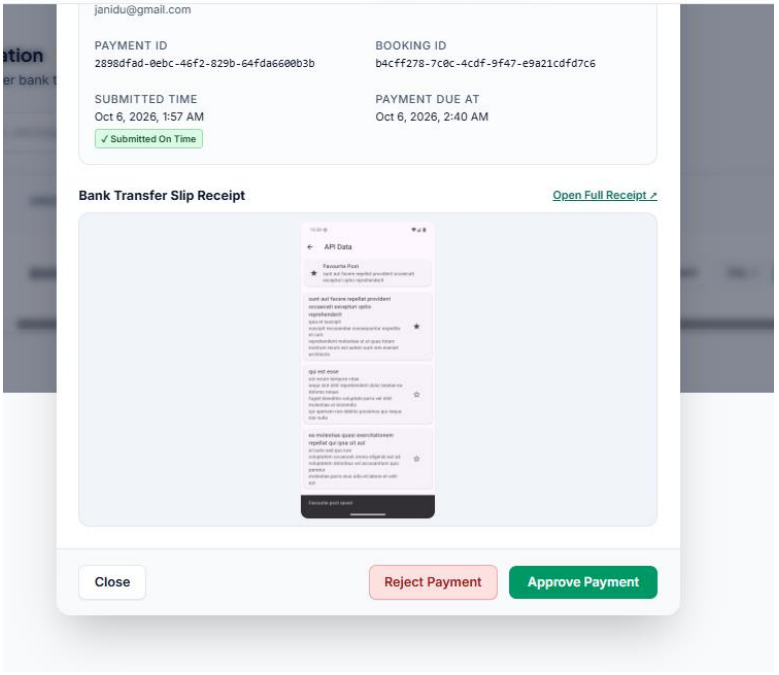


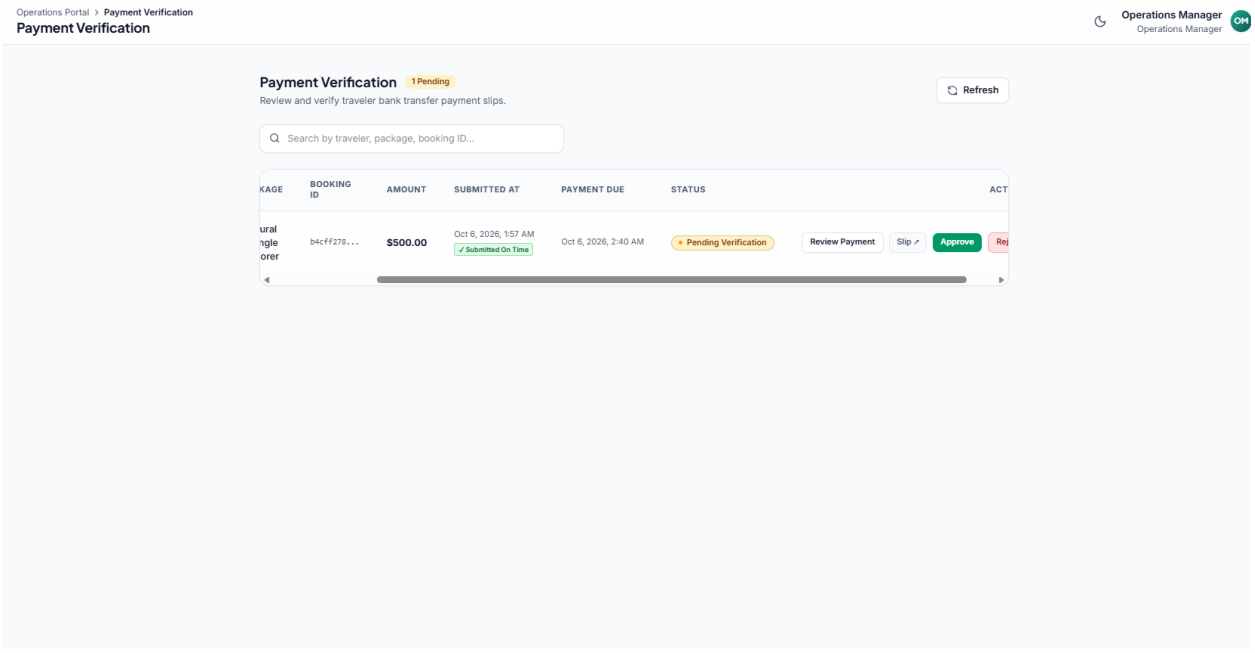




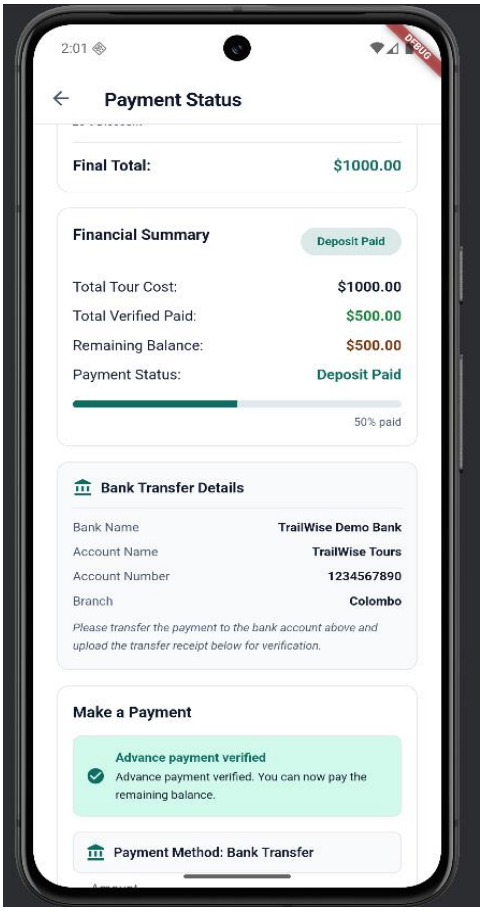
3. Operations Manager pending-payment verification screen.



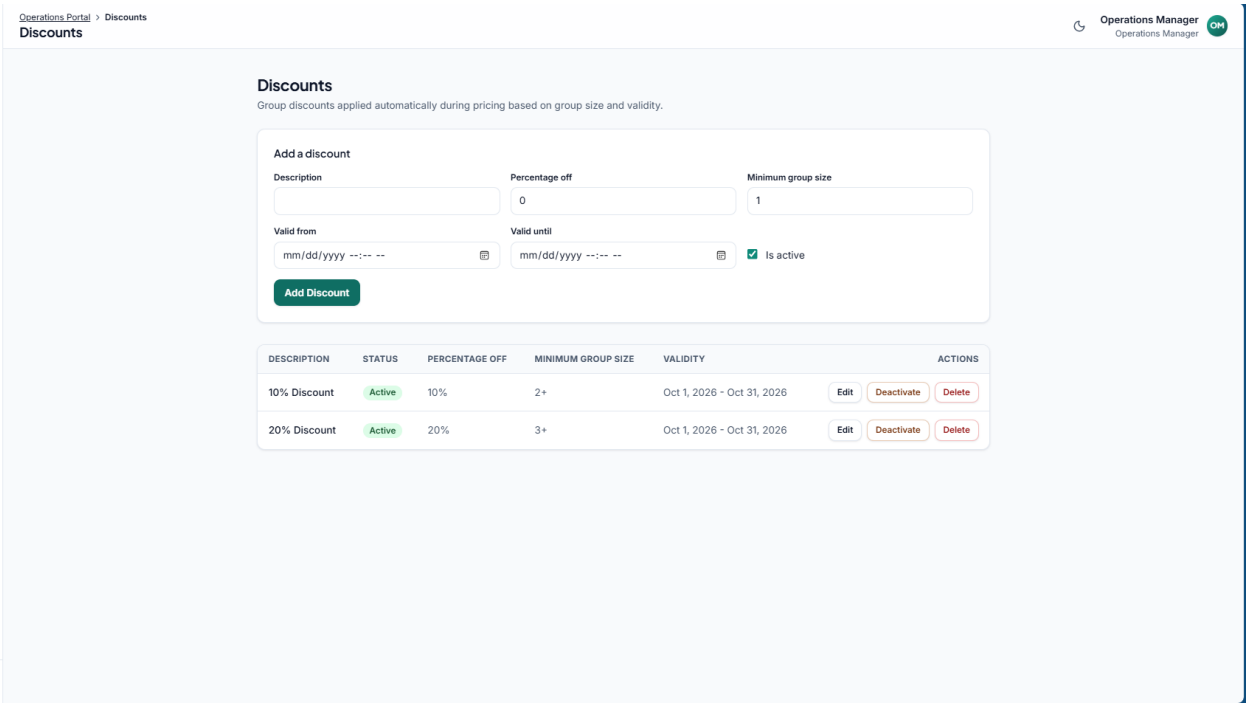


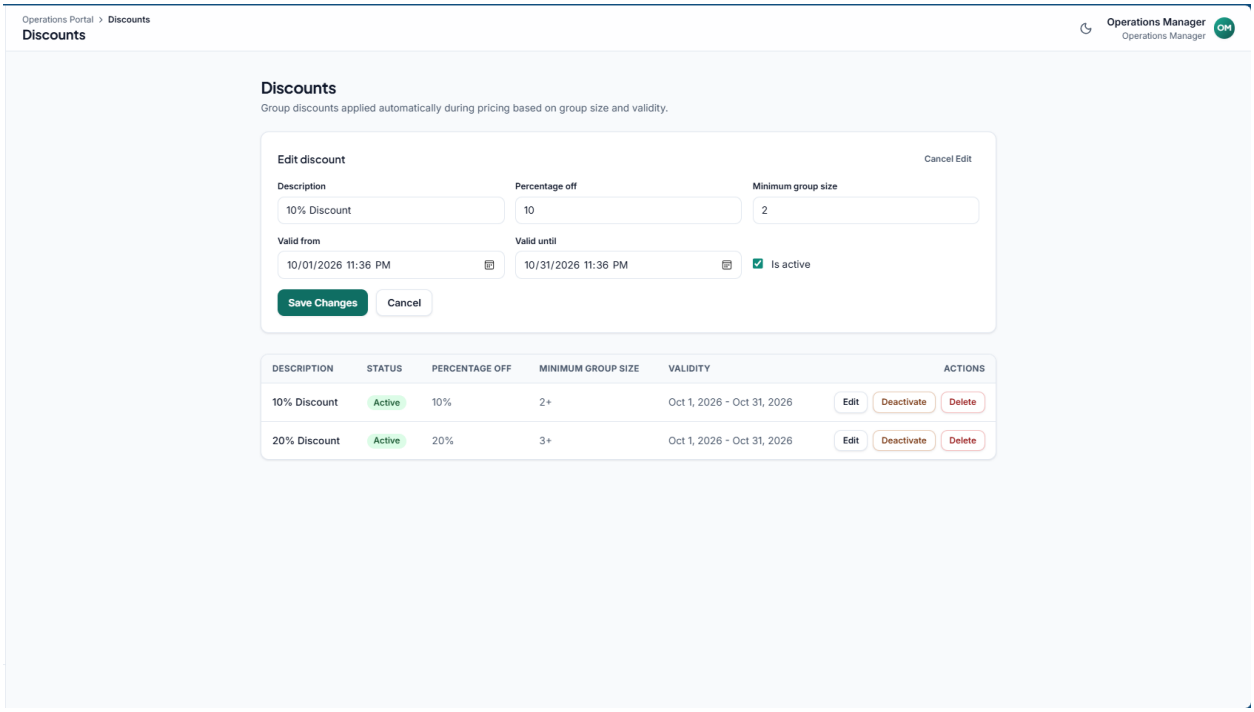


4. Approved / rejected payment status.

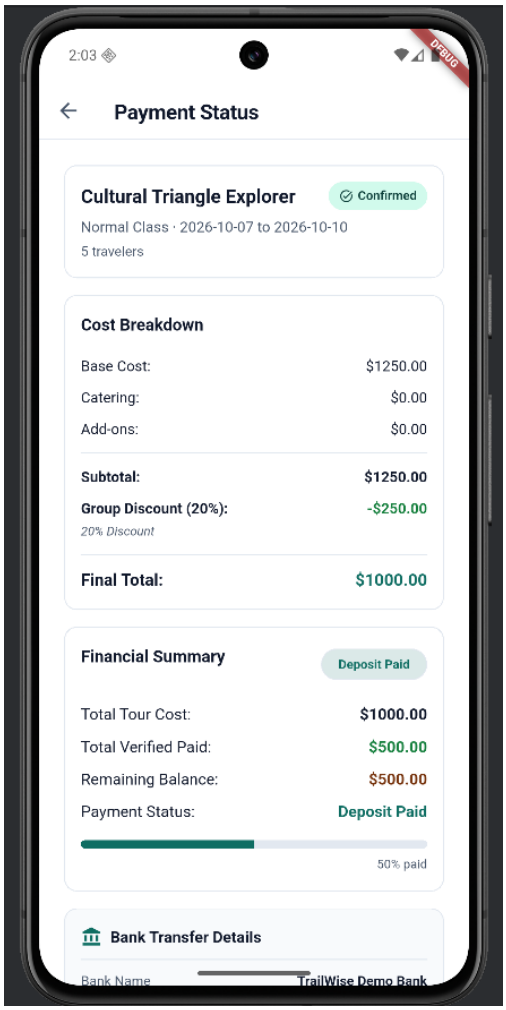


5. Discount management screen showing percentage, minimum group size, active status and validity period.

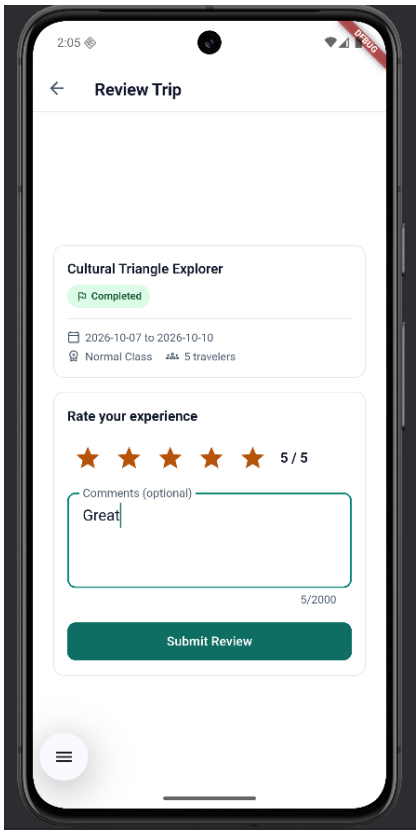


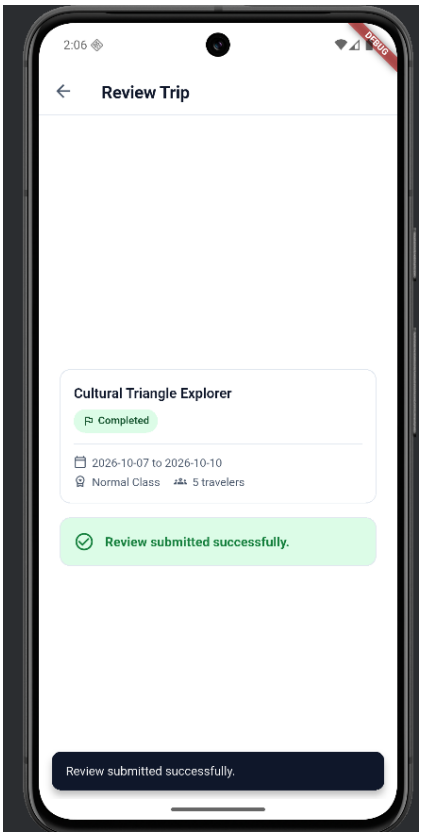


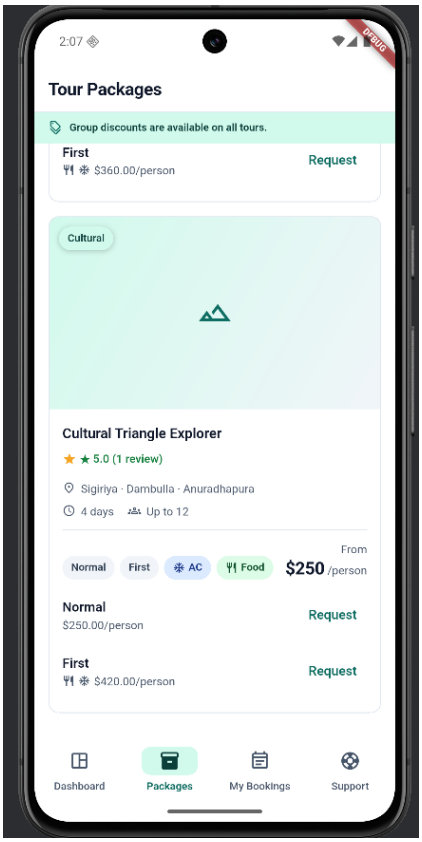
6. Traveler-facing available group discounts.

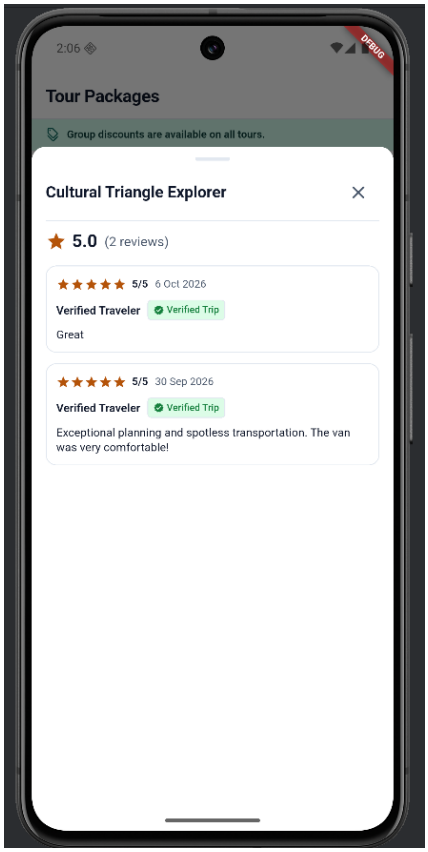


7. Review submission screen and package review display.

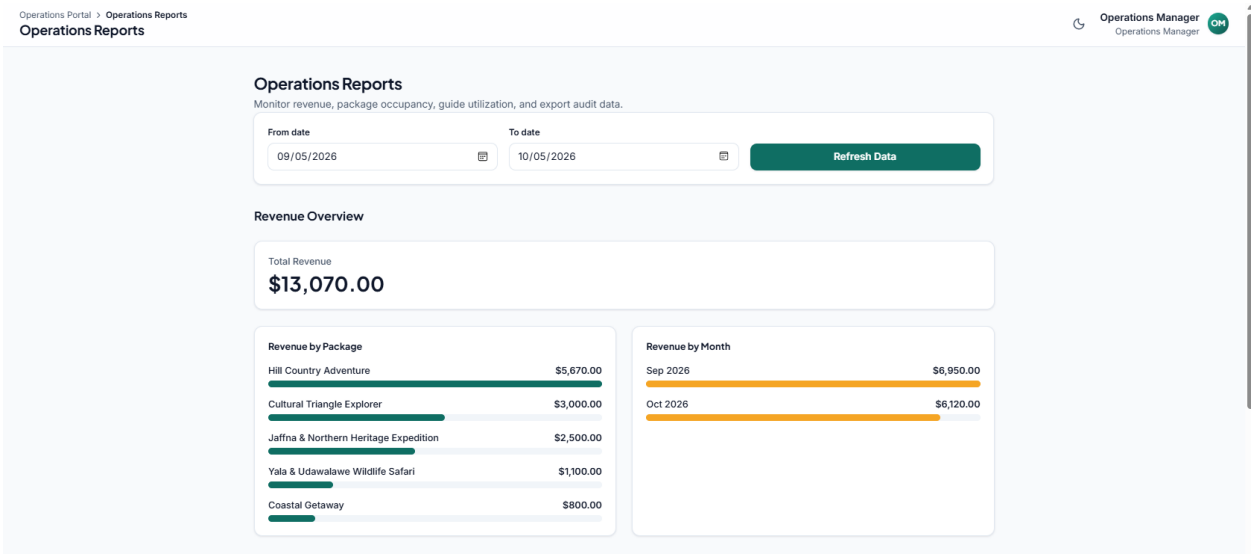


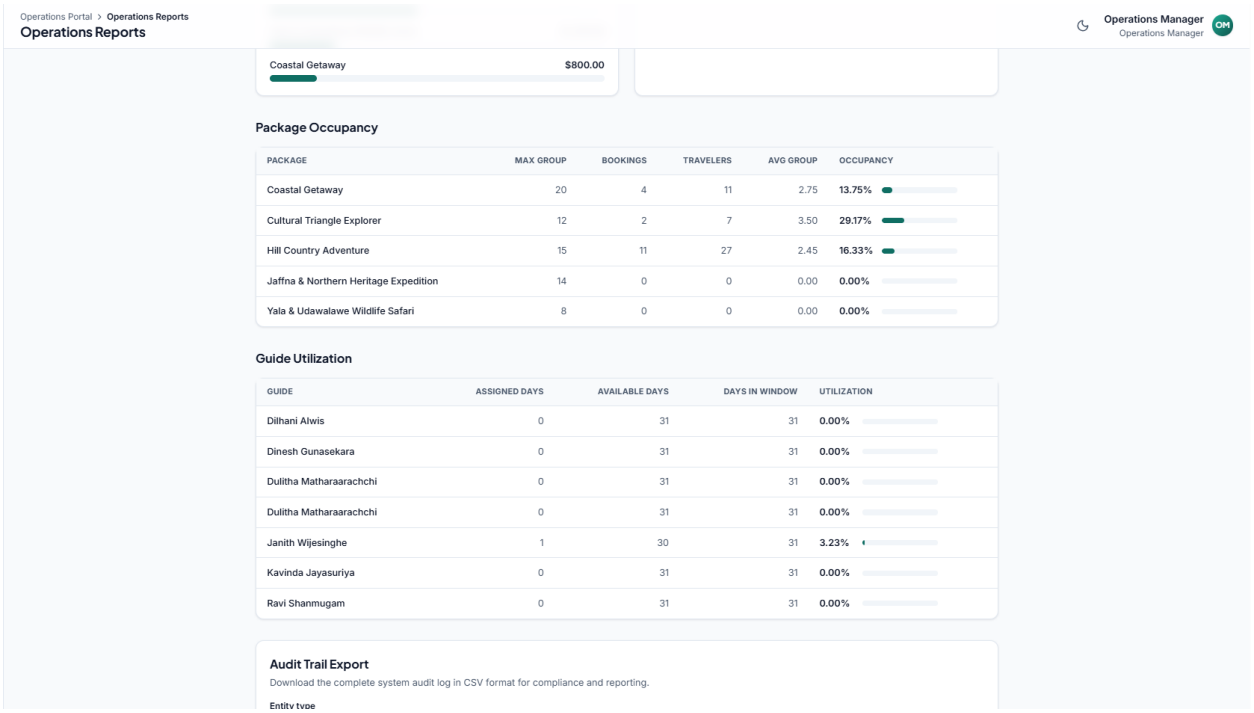




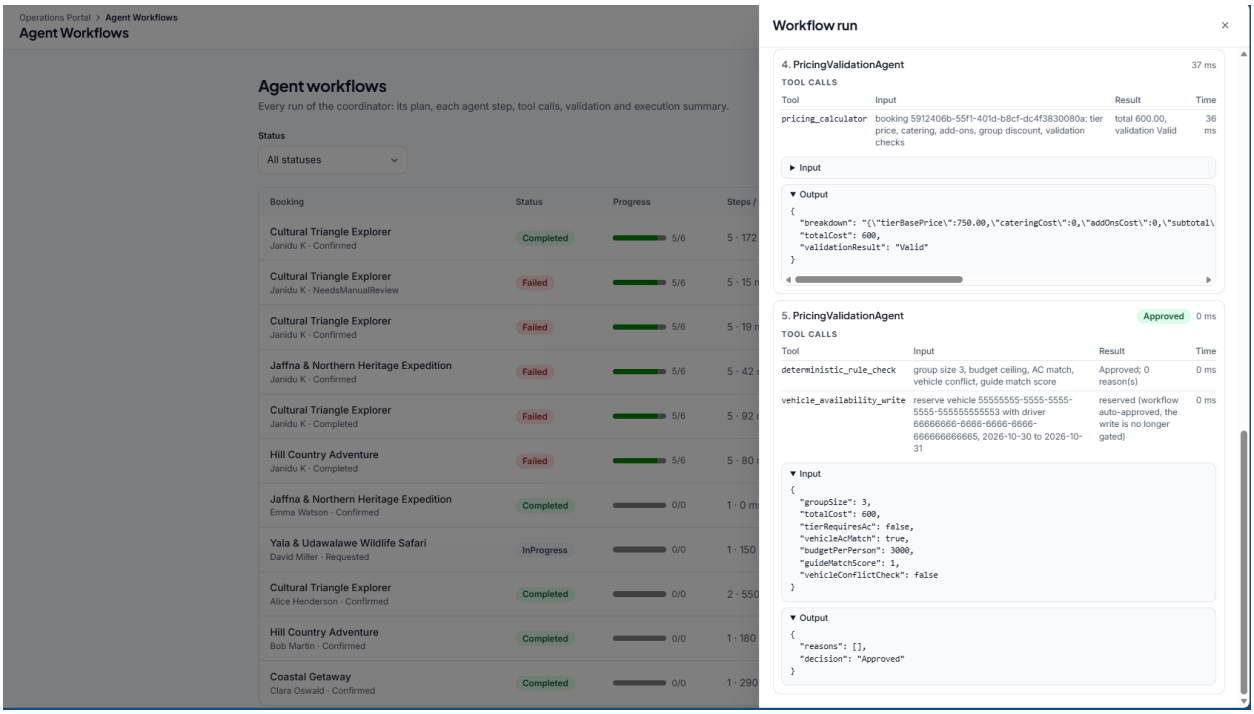


8. Revenue / occupancy / guide-utilization reporting screens.





9. Agent Workflow screen showing `PricingValidationAgent` output with pricing breakdown and validation result.



16.4.7 Challenges and Learning

One of the main challenges was keeping the payment amount consistent with the price generated by the Pricing Validation Agent. Recalculating the price later could create differences if a discount was edited or expired after the booking was created. I solved this by storing the original pricing breakdown in the agent step log and using that stored snapshot as the authoritative price for later payments.

Another challenge was managing the payment lifecycle separately from the booking lifecycle. A tour can be completed while a final balance is still unpaid. I had to make sure that a completed booking could still accept its remaining payment and that the review option only became available after the booking was both completed and fully paid.

Payment deadlines were another important area. A confirmed booking needs an advance payment deadline, while the remaining balance after completion uses a separate deadline. Keeping these two deadlines separate helped avoid incorrect booking status changes and made the Flutter countdown easier to understand.

I also had to consider payment-slip security. Initially it would have been easier to expose uploaded files directly, but this would create a privacy risk. The final implementation stores slips privately and provides them only through an authenticated backend endpoint.

The discount feature also required careful handling. Discounts must be active, satisfy the minimum group size and be inside their validity period. I learned that historical booking prices should never change when an administrator later edits or disables a discount. Saving the applied discount as part of the original pricing snapshot solved this problem.

Integrating the Pricing Validation Agent with the Guide Matching and Fleet Capacity results also improved my understanding of multi-agent systems. My agent does not work alone in the booking workflow. It consumes structured results from other agents, performs deterministic financial and validation logic, and then gives a structured result back to the coordinator.

16.4.8 Individual AI Usage Log

You can use this table, but check the dates against your own records before the final submission.

Date	Tool / Model	Task	What AI Produced	What I Changed / Rejected	How I Verified
Sep 2026	ChatGPT / Antigravity	PricingValidationAgent implementation	Suggested pricing calculation, breakdown structure and validation flow	Kept financial calculations deterministic in C# and prevented traveler special requests from changing price logic	Backend PricingValidationAgent tests and database AgentStepLog inspection
Sep 2026	ChatGPT / Antigravity	Payment and bank-slip flow	Generated payment endpoint/service structure and verification logic	Improved private slip storage, first-payment rules, payment status handling and staff authorization	Backend payment tests and manual Flutter payment testing
Sep 2026	ChatGPT / Antigravity	Reviews	Suggested review DTOs, service and endpoint tests	Added Completed + FullyPaid requirement, ownership check, duplicate prevention and privacy-safe public review output	Review endpoint tests and Flutter manual testing
Sep-Oct 2026	ChatGPT / Antigravity	Audit Logging and Reports	Helped generate shared audit logging and reporting queries	Checked formulas for occupancy, revenue and guide utilisation and corrected data-selection rules	PostgreSQL queries and backend automated tests

Oct 2026	ChatGPT / Antigravity	Discount improvement	Suggested active status, validity period and traveler pricing display	Kept discounts global instead of package-specific and added historical price snapshot protection	Backend, React and Flutter tests plus manual booking test
Oct 2026	ChatGPT / Antigravity	Payment UI improvements	Suggested payment-status breakdown, deadline states, sorting and responsive layouts	Adjusted UI based on real screenshots and preserved existing financial summary and payment actions	Flutter analyze, widget tests and emulator testing

16.4.9 Individual AI Reflection

During the development of my Person 4 tasks in TrailWise, I used AI tools such as ChatGPT and Antigravity as development assistants. They helped me understand the existing project structure, generate initial implementation ideas, find possible issues and prepare tests. AI was especially useful because my work covered several connected areas including the Pricing Validation Agent, payments, reviews, discounts, reports, audit logging and Flutter/React integration.

One important area where AI helped was the Pricing Validation Agent. It helped me separate the agent's responsibility from the rest of the coordinator workflow and understand that financial calculations should not depend on an LLM. I kept calculations such as base price, catering, add-ons, discounts and final cost as deterministic C# logic. This was important because financial values must be predictable and testable. I also ensured that traveler special requests could not influence the pricing calculation.

AI was also useful when implementing the payment flow. It suggested service and DTO structures and helped identify several edge cases. However, I did not use every suggestion directly. For example, during development I explored adding a manual pricing fallback where an Operations Manager could replace a failed agent result. After testing and reviewing the design, I decided that this added unnecessary complexity and removed that feature. I kept the normal automatic Pricing Validation Agent workflow instead. This showed me that AI suggestions still need to be evaluated against the actual project requirements.

Another challenge was payment lifecycle handling. The initial implementation needed improvements for completed bookings that still had a remaining balance. I changed the logic so booking completion and payment completion are treated separately. I also added separate advance and final balance deadlines and tested the countdowns in Flutter.

For discounts, AI helped suggest active/inactive controls and validity periods. I decided not to make discounts package-specific because the existing project used global group discounts. I kept the existing design and improved it with validity dates, active status and traveler-facing information. I also made sure previous bookings keep the discount originally applied to them even if the discount changes later.

I verified AI-assisted changes using automated backend, React and Flutter tests together with manual testing. I also checked PostgreSQL data directly when necessary, especially for pricing logs, payments and audit entries. Screenshots from the Flutter emulator and React operations portal were useful for finding UI issues that automated tests did not clearly show.

Overall, AI helped me work faster, but it did not replace my responsibility for the implementation. I had to understand the code, decide which suggestions matched the project requirements, reject unnecessary changes and verify the final behavior. Through this work I improved my understanding of ASP.NET Core services, EF Core, Flutter and React integration, secure file handling, deterministic agent workflows, financial validation and automated testing.

16.4.10 Individual Declaration

I declare that the contribution described in this section accurately represents my work, that any AI-assisted work has been disclosed in my AI usage log, and that I can explain, test and modify the submitted work attributed to me.

Name	Student ID	Signature	Date
Matharaarachchi D.C.M	IT24101252		06/10/2026

17. References

- SLIIT (2026) SE3090 Assignment 1 - Integrated Full-Stack and Agentic AI Application Development: Specification with Marking Scheme.
- TrailWise Team (2026) TrailWise AI - Project Design Document.
- Microsoft (2026) ASP.NET Core and Entity Framework Core documentation. Available at: https://learn.microsoft.com/aspnet/core
- Npgsql (2026) Entity Framework Core provider for PostgreSQL. Available at: https://www.npgsql.org/efcore/
- React (2026) React documentation. Available at: https://react.dev
- Zustand (2026) Zustand documentation. Available at: https://zustand.docs.pmnd.rs
- Flutter (2026) Flutter documentation. Available at: https://docs.flutter.dev
- OpenStreetMap Foundation (2026) Nominatim usage policy. Available at: https://operations.osmfoundation.org/policies/nominatim/
- Perkins, M., Furze, L., Roe, J. and MacVaugh, J. (2024) The AI Assessment Scale.

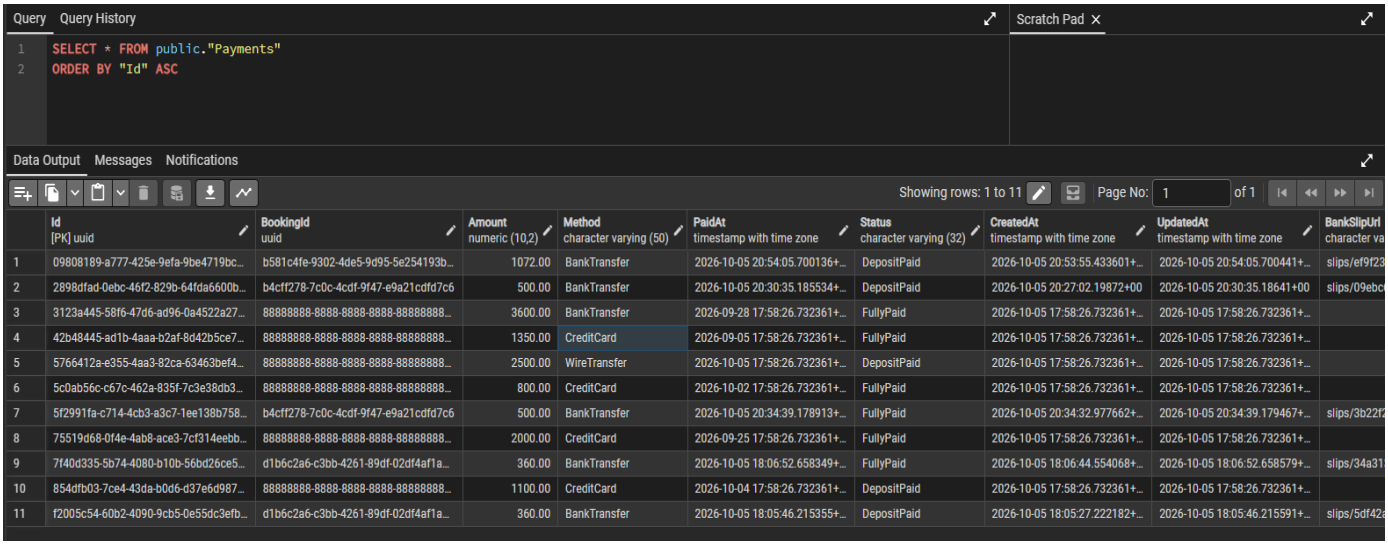
18. Appendices

Appendix A - Final URLs and Evaluator Access

Resource	URL / Details
GitHub repository	https://github.com/Hasanga910/TrailWise
React	https://trailwise-xi.vercel.app
API health	https://trailwise.onrender.com/health
Swagger	https://trailwise.onrender.com/swagger
Demo video	https://drive.google.com/drive/folders/1WQBnnaMKBW1p79MF9oIhYurU4N-9fg9R?usp=shari
APK	https://drive.google.com/drive/folders/1O3mV2JGF5MCdc4SF8WNbP16otNSG9Tdn?usp=sha
Database evidence	Supabase PostgreSQL (Session pooler, ap-northeast-1); migrations applied on API start; RLS enabled

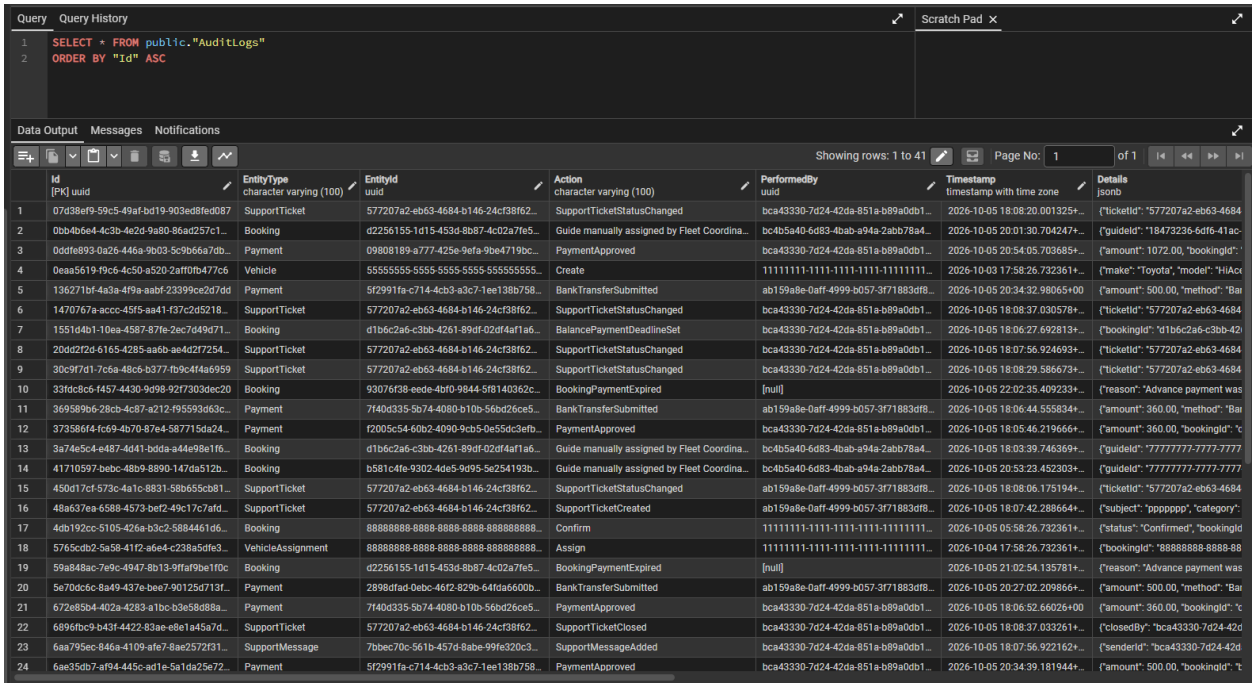
Appendix B - Screenshots and Evidence Index

Figure B1 – PostgreSQL / pgAdmin evidence for payments



	Id [PK] uuid	BookingId uuid	Amount numeric (10,2)	Method character varying (50)	PaidAt timestamp with time zone	Status character varying (32)	CreatedAt timestamp with time zone	UpdatedAt timestamp with time zone	BankSlipUrl character va
1	09808189-a777-425e-9efa-9be4719bc...	b581c4fe-9302-4de5-9d95-5e254193b...	1072.00	BankTransfer	2026-10-05 20:54:05.700136+...	DepositPaid	2026-10-05 20:53:55.433601+...	2026-10-05 20:54:05.700441+...	slips/ef9f23
2	2898dfad-0ebc-46f2-829b-64fda6600b...	b4cff278-7c0c-4cdf-9f47-e9a21cdfd7c6	500.00	BankTransfer	2026-10-05 20:30:35.185534+...	DepositPaid	2026-10-05 20:27:02.19872+00	2026-10-05 20:30:35.18641+00	slips/09ebc
3	3123a445-58f6-47d6-ad96-0a4522a27...	88888888-8888-8888-8888-88888888...	3600.00	BankTransfer	2026-09-28 17:58:26.732361+...	FullyPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
4	42b48445-ad1b-4aaa-b2af-8d42b5ce7...	88888888-8888-8888-8888-88888888...	1350.00	CreditCard	2026-09-05 17:58:26.732361+...	FullyPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
5	5766412a-e355-4aa3-82ca-63463bef4...	88888888-8888-8888-8888-88888888...	2500.00	WireTransfer	2026-10-05 17:58:26.732361+...	DepositPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
6	5c0ab56c-c67c-462a-835f-7c3e38db3...	88888888-8888-8888-8888-88888888...	800.00	CreditCard	2026-10-02 17:58:26.732361+...	FullyPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
7	5f2991fa-c714-4cb3-a3c7-1ee138b758...	b4cff278-7c0c-4cdf-9f47-e9a21cdfd7c6	500.00	BankTransfer	2026-10-05 20:34:39.178913+...	FullyPaid	2026-10-05 20:34:32.977662+...	2026-10-05 20:34:39.179467+...	slips/3b22f2
8	75519d68-0f4e-4ab8-ace3-7cf314eebb...	88888888-8888-8888-8888-88888888...	2000.00	CreditCard	2026-09-25 17:58:26.732361+...	FullyPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
9	7f40d335-5b74-4080-b10b-56bd26ce5...	d1b6c2a6-c3bb-4261-89df-02df4af1a...	360.00	BankTransfer	2026-10-05 18:06:52.658349+...	FullyPaid	2026-10-05 18:06:44.554068+...	2026-10-05 18:06:52.658579+...	slips/34a31
10	854dfb03-7ce4-43da-b0d6-d37e6d987...	88888888-8888-8888-8888-88888888...	1100.00	CreditCard	2026-10-04 17:58:26.732361+...	DepositPaid	2026-10-05 17:58:26.732361+...	2026-10-05 17:58:26.732361+...	
11	f2005c54-60b2-4090-9cb5-0e55dc3efb...	d1b6c2a6-c3bb-4261-89df-02df4af1a...	360.00	BankTransfer	2026-10-05 18:05:46.215355+...	DepositPaid	2026-10-05 18:05:27.222182+...	2026-10-05 18:05:46.215591+...	slips/5df42

Figure B2 – PostgreSQL / pgAdmin evidence for audit logs agent logs



```
SELECT * FROM public."AuditLogs"
ORDER BY "Id" ASC
```

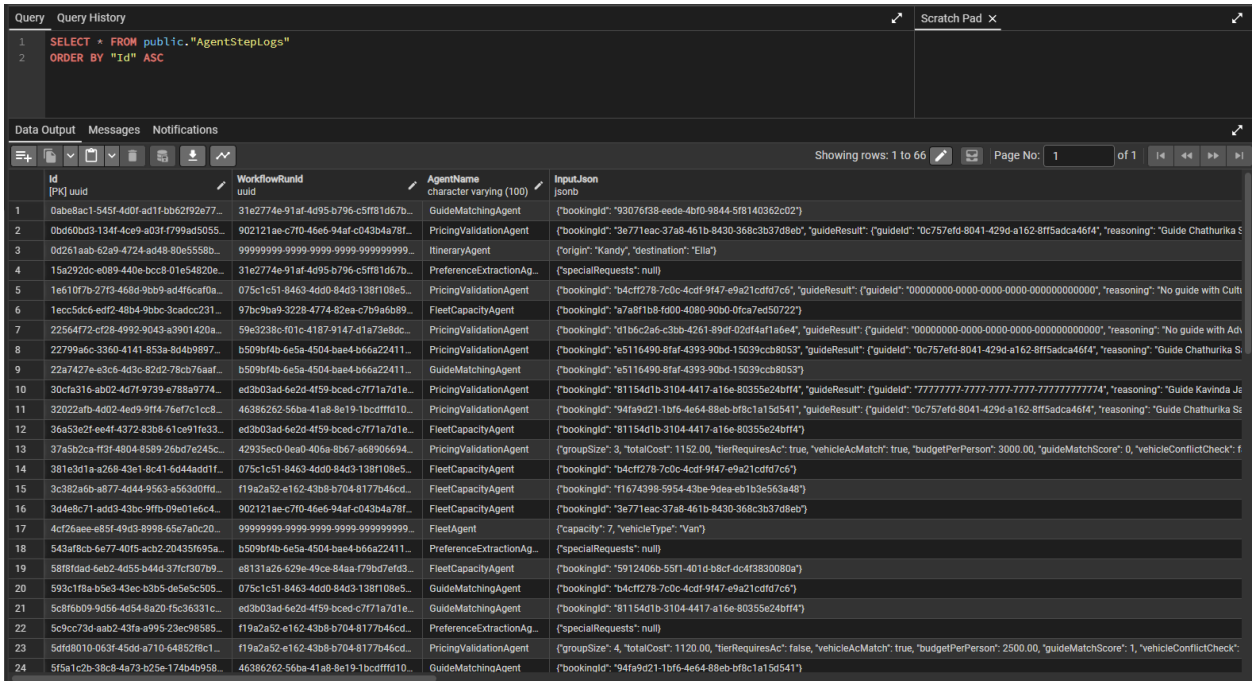
Query | Query History | Scratch Pad

Data Output | Messages | Notifications

Showing rows: 1 to 41 | Page No: 1 of 1

	Id [PK] uuid	EntityType character varying (100)	EntityId uuid	Action character varying (100)	PerformedBy uuid	Timestamp timestamp with time zone	Details jsonb
1	07d38ef9-59c5-49af-bd19-903ed8fed087	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketStatusChanged	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:08:20.001325+...	{"ticketId": "577207a2-eb63-4684
2	0bb4b6e4-4c3b-4e2d-9a80-86ad257c1...	Booking	d2256155-1d15-453d-8b87-4c02a7fe5...	Guide manually assigned by Fleet Coordina...	bc4b5a40-6d83-4bab-a94a-2abb78a4...	2026-10-05 20:01:30.704247+...	{"guideId": "18473236-6df6-41ac-
3	0ddfe893-0a26-446a-9b03-5c9b66a7db...	Payment	09808189-a777-425e-9efa-9be4719bc...	PaymentApproved	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 20:54:05.703685+...	{"amount": 1072.00, "bookingId": "
4	0eaa5619-f9c6-4c50-a520-2aff0fb477c6	Vehicle	55555555-5555-5555-5555-555555555...	Create	11111111-1111-1111-1111-11111111...	2026-10-03 17:58:26.732361+...	{"make": "Toyota", "model": "HiAc
5	136271bf-4a3a-4f9a-aabf-23399ce2d7dd	Payment	5f2991fa-c714-4cb3-a3c7-1ee138b758...	BankTransferSubmitted	ab159a8e-0aff-4999-b057-3f71883df8...	2026-10-05 20:34:32.98065+00	{"amount": 500.00, "method": "Ba
6	1470767a-accc-45f5-aa41-f37c2d5218...	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketStatusChanged	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:08:37.030578+...	{"ticketId": "577207a2-eb63-4684
7	1551d4b1-10ea-4587-87fe-2ec7d49d71...	Booking	d1b6c2a6-c3bb-4261-89df-02df4af1a6...	BalancePaymentDeadlineSet	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:06:27.692813+...	{"bookingId": "d1b6c2a6-c3bb-42
8	20dd2f2d-6165-4285-aa6b-ae4d2f7254...	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketStatusChanged	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:07:56.924693+...	{"ticketId": "577207a2-eb63-4684
9	30c9f7d1-7c6a-48c6-b377-fb9c4f4a6959	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketStatusChanged	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:08:29.586673+...	{"ticketId": "577207a2-eb63-4684
10	33fdc8c6-f457-4430-9d98-92f7303dec20	Booking	93076f38-eede-4bf0-9844-5f8140362c...	BookingPaymentExpired	[null]	2026-10-05 22:02:35.409233+...	{"reason": "Advance payment was
11	369589b6-28cb-4c87-a212-f95593d63c...	Payment	7f40d335-5b74-4080-b10b-56bd26ce5...	BankTransferSubmitted	ab159a8e-0aff-4999-b057-3f71883df8...	2026-10-05 18:06:44.555834+...	{"amount": 360.00, "method": "Ba
12	373586f4-fc69-4b70-87e4-587715da24...	Payment	f2005c54-60b2-4090-9cb5-0e55dc3efb...	PaymentApproved	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:05:46.219666+...	{"amount": 360.00, "bookingId": "c
13	3a74e5c4-e487-4d41-bdda-a44e98e1f6...	Booking	d1b6c2a6-c3bb-4261-89df-02df4af1a6...	Guide manually assigned by Fleet Coordina...	bc4b5a40-6d83-4bab-a94a-2abb78a4...	2026-10-05 18:03:39.746369+...	{"guideId": "77777777-7777-7777
14	41710597-bebc-48b9-8890-147da512b...	Booking	b581c4fe-9302-4de5-9d95-5e254193b...	Guide manually assigned by Fleet Coordina...	bc4b5a40-6d83-4bab-a94a-2abb78a4...	2026-10-05 20:53:23.452303+...	{"guideId": "77777777-7777-7777
15	450d17cf-573c-4a1c-8831-58b655cb81...	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketStatusChanged	ab159a8e-0aff-4999-b057-3f71883df8...	2026-10-05 18:08:06.175194+...	{"ticketId": "577207a2-eb63-4684
16	48a637ea-6588-4573-bef2-49c17c7afd...	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketCreated	ab159a8e-0aff-4999-b057-3f71883df8...	2026-10-05 18:07:42.288664+...	{"subject": "ppppppp", "category":
17	4db192cc-5105-426a-b3c2-5884461d6...	Booking	88888888-8888-8888-8888-888888888...	Confirm	11111111-1111-1111-1111-11111111...	2026-10-05 05:58:26.732361+...	{"status": "Confirmed", "bookingId
18	5765cdb2-5a58-41f2-a6e4-c238a5dfe3...	VehicleAssignment	88888888-8888-8888-8888-888888888...	Assign	11111111-1111-1111-1111-11111111...	2026-10-04 17:58:26.732361+...	{"bookingId": "88888888-8888-88
19	59a848ac-7e9c-4947-8b13-9ffaf9be1f0c	Booking	d2256155-1d15-453d-8b87-4c02a7fe5...	BookingPaymentExpired	[null]	2026-10-05 21:02:54.135781+...	{"reason": "Advance payment was
20	5e70dc6c-8a49-437e-bee7-90125d713f...	Payment	2898dfad-0ebc-46f2-829b-64fda6600b...	BankTransferSubmitted	ab159a8e-0aff-4999-b057-3f71883df8...	2026-10-05 20:27:02.209866+...	{"amount": 500.00, "method": "Ba
21	672e85b4-402a-4283-a1bc-b3e58d88a...	Payment	7f40d335-5b74-4080-b10b-56bd26ce5...	PaymentApproved	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:06:52.66026+00	{"amount": 360.00, "bookingId": "c
22	6896fbc9-b43f-4422-83ae-e8e1a45a7d...	SupportTicket	577207a2-eb63-4684-b146-24cf38f62...	SupportTicketClosed	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:08:37.033261+...	{"closedBy": "bca43330-7d24-42d
23	6aa795ec-846a-4109-afe7-8ae2572f31...	SupportMessage	7bbec70c-561b-457d-8abe-99fe320c3...	SupportMessageAdded	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 18:07:56.922162+...	{"senderId": "bca43330-7d24-42d
24	6ae35db7-af94-445c-ad1e-5a1da25e72...	Payment	5f2991fa-c714-4cb3-a3c7-1ee138b758...	PaymentApproved	bca43330-7d24-42da-851a-b89a0db1...	2026-10-05 20:34:39.181944+...	{"amount": 500.00, "bookingId": "c

Figure B3 – PostgreSQL / pgAdmin evidence for agent logs



```
SELECT * FROM public."AgentStepLogs"
ORDER BY "Id" ASC
```

Query | Query History | Scratch Pad

Data Output | Messages | Notifications

Showing rows: 1 to 66 | Page No: 1 of 1

	Id [PK] uuid	WorkflowRunId uuid	AgentName character varying (100)	InputJson jsonb
1	0abe8ac1-545f-4d0f-ad1f-bb62f92e77...	31e2774e-91af-4d95-b796-c5ff81d67b...	GuideMatchingAgent	{"bookingId": "93076f38-eede-4bf0-9844-5f8140362c02"}
2	0bd60bd3-134f-4ce9-a03f-f799ad5055...	902121ae-c7f0-46e6-94af-c043b4a78f...	PricingValidationAgent	{"bookingId": "3e771eac-37a8-461b-8430-368c3b37d8eb", "guideResult": {"guideId": "0c757efd-8041-429d-a162-8ff5adca46f4", "reasoning": "Guide Chathurika S
3	0d261aab-62a9-4724-ad48-80e5558b...	99999999-9999-9999-9999-999999999...	ItineraryAgent	{"origin": "Kandy", "destination": "Ella"}
4	15a292dc-e089-440e-bcc8-01e54820e...	31e2774e-91af-4d95-b796-c5ff81d67b...	PreferenceExtractionAg...	{"specialRequests": null}
5	1e610f7b-27f3-468d-9bb9-ad4f6caf0a...	075c1c51-8463-4dd0-84d3-138f108e5...	PricingValidationAgent	{"bookingId": "b4cff278-7c0c-4cdf-9f47-e9a21cdfd7c6", "guideResult": {"guideId": "00000000-0000-0000-0000-000000000000", "reasoning": "No guide with Cult
6	1ecc5dc6-edf2-48b4-9bbc-3cadcc231...	97bc9ba9-3228-4774-82ea-c7b9a6b89...	FleetCapacityAgent	{"bookingId": "a7a8f1b8-fd00-4080-90b0-0fca7ed50722"}
7	22564f72-cf28-4992-9043-a3901420a...	59e3238c-f01c-4187-9147-d1a73e8dc...	PricingValidationAgent	{"bookingId": "d1b6c2a6-c3bb-4261-89df-02df4af1a6e4", "guideResult": {"guideId": "00000000-0000-0000-0000-000000000000", "reasoning": "No guide with Adv
8	22799a6c-3360-4141-853a-8d4b9897...	b509bf4b-6e5a-4504-bae4-b66a22411...	PricingValidationAgent	{"bookingId": "e5116490-8faf-4393-90bd-15039ccb8053", "guideResult": {"guideId": "0c757efd-8041-429d-a162-8ff5adca46f4", "reasoning": "Guide Chathurika S
9	22a7427e-e3c6-4d3c-82d2-78cb76aaf...	b509bf4b-6e5a-4504-bae4-b66a22411...	GuideMatchingAgent	{"bookingId": "e5116490-8faf-4393-90bd-15039ccb8053"}
10	30cfa316-ab02-4d7f-9739-e788a9774...	ed3b03ad-6e2d-4f59-bced-c7f71a7d1e...	PricingValidationAgent	{"bookingId": "81154d1b-3104-4417-a16e-80355e24bff4", "guideResult": {"guideId": "77777777-7777-7777-7777-777777777774", "reasoning": "Guide Kavinda Ja
11	32022afb-4d02-4ed9-9ff4-76ef7c1cc8...	46386262-56ba-41a8-8e19-1bcdfffd10...	PricingValidationAgent	{"bookingId": "94fa9d21-1bf6-4e64-88eb-bf8c1a15d541", "guideResult": {"guideId": "0c757efd-8041-429d-a162-8ff5adca46f4", "reasoning": "Guide Chathurika Sa
12	36a53e2f-ee4f-4372-83b8-61ce91fe33...	ed3b03ad-6e2d-4f59-bced-c7f71a7d1e...	FleetCapacityAgent	{"bookingId": "81154d1b-3104-4417-a16e-80355e24bff4"}
13	37a5b2ca-ff3f-4804-8589-26bd7e245c...	42935ec0-0ea0-406a-8b67-a68906694...	PricingValidationAgent	{"groupSize": 3, "totalCost": 1152.00, "tierRequiresAc": true, "vehicleAcMatch": true, "budgetPerPerson": 3000.00, "guideMatchScore": 0, "vehicleConflictCheck": f
14	381e3d1a-a268-43e1-8c41-6d44add1f...	075c1c51-8463-4dd0-84d3-138f108e5...	FleetCapacityAgent	{"bookingId": "b4cff278-7c0c-4cdf-9f47-e9a21cdfd7c6"}
15	3c382a6b-a877-4d44-9563-a563d0ffd...	f19a2a52-e162-43b8-b704-8177b46cd...	FleetCapacityAgent	{"bookingId": "f1674398-5954-43be-9dea-eb1b3e563a48"}
16	3d4e8c71-add3-43bc-9ffb-09e01e6c4...	902121ae-c7f0-46e6-94af-c043b4a78f...	FleetCapacityAgent	{"bookingId": "3e771eac-37a8-461b-8430-368c3b37d8eb"}
17	4cf26aee-e85f-49d3-8998-65e7a0c20...	99999999-9999-9999-9999-999999999...	FleetAgent	{"capacity": 7, "vehicleType": "Van"}
18	543af8cb-6e77-40f5-acb2-20435f695a...	b509bf4b-6e5a-4504-bae4-b66a22411...	PreferenceExtractionAg...	{"specialRequests": null}
19	58f8fdad-6eb2-4d55-b44d-37fcf307b9...	e8131a26-629e-49ce-84aa-f79bd7efd3...	FleetCapacityAgent	{"bookingId": "5912406b-55f1-401d-b8cf-dc4f3830080a"}
20	593c1f8a-b5e3-43ec-b3b5-de5e5c505...	075c1c51-8463-4dd0-84d3-138f108e5...	GuideMatchingAgent	{"bookingId": "b4cff278-7c0c-4cdf-9f47-e9a21cdfd7c6"}
21	5c8f6b09-9d56-4d54-8a20-f5c36331c...	ed3b03ad-6e2d-4f59-bced-c7f71a7d1e...	GuideMatchingAgent	{"bookingId": "81154d1b-3104-4417-a16e-80355e24bff4"}
22	5c9cc73d-aab2-43fa-a995-23ec98585...	f19a2a52-e162-43b8-b704-8177b46cd...	PreferenceExtractionAg...	{"specialRequests": null}
23	5dfd8010-063f-45dd-a710-64852f8c1...	f19a2a52-e162-43b8-b704-8177b46cd...	PricingValidationAgent	{"groupSize": 4, "totalCost": 1120.00, "tierRequiresAc": false, "vehicleAcMatch": true, "budgetPerPerson": 2500.00, "guideMatchScore": 1, "vehicleConflictCheck":
24	5f5a1c2b-38c8-4a73-b25e-174b4b958...	46386262-56ba-41a8-8e19-1bcdfffd10...	GuideMatchingAgent	{"bookingId": "94fa9d21-1bf6-4e64-88eb-bf8c1a15d541"}

Figure B4 – Backend automated test results using dotnet test

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend> dotnet test
  Determining projects to restore...
  All projects are up-to-date for restore.
  TrailWise.Domain -> D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend\src\TrailWise.Domain\bin\Debug\net8.0\Tra
  ilWise.Domain.dll
  TrailWise.Infrastructure -> D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend\src\TrailWise.Infrastructure\bin\
  Debug\net8.0\TrailWise.Infrastructure.dll
  TrailWise.Api -> D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend\src\TrailWise.Api\bin\Debug\net8.0\TrailWise
  .Api.dll
  TrailWise.Api.Tests -> D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend\tests\TrailWise.Api.Tests\bin\Debug\ne
  t8.0\TrailWise.Api.Tests.dll
Test run for D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend\tests\TrailWise.Api.Tests\bin\Debug\net8.0\TrailWis
e.Api.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:   892, Skipped:     0, Total:   892, Duration: 30 s - TrailWise.Api.Tests.d
ll (net8.0)
PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\backend>
```

Figure B5 – React automated test results using Vitest

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\frontend-web> npm test
      ✓ renders backend field errors next to the corresponding inputs  951ms
   ✓ src/pages/traveler/TravelerProfileSettingsPage.test.tsx (2 tests) 1332ms
      ✓ toasts after saving the profile  385ms
      ✓ shows a password mismatch inline, not as a toast  942ms
   ✓ src/components/explorer/explorerFilters.test.ts (8 tests) 10ms
   ✓ src/components/packages/locationCoordinates.test.ts (4 tests) 6ms
   ✓ src/components/packages/tierHelpers.test.ts (3 tests) 7ms
   ✓ src/pages/traveler/MyBookingsPage.test.tsx (14 tests) 5548ms
      ✓ renders paginated bookings from getMyBookings  435ms
      ✓ disables Previous on page 1 and Next on the last page  510ms
      ✓ shows a Cancel Booking button for an upcoming, non-terminal booking  390ms
      ✓ does not show a Cancel Booking button for a completed booking  377ms
      ✓ does not show a Cancel Booking button for a past booking  372ms
      ✓ cancels an upcoming booking through the confirm dialog  914ms
      ✓ hides "Cancel Booking" button if tour has started  344ms
      ✓ hides "Cancel Booking" button if tour has ended  336ms
      ✓ shows the itinerary when View Itinerary is clicked for a confirmed booking  478ms
      ✓ does not show a View Itinerary button for a non-confirmed booking  338ms
      ✓ displays assigned tour guide details including name, contact, languages, and specializations  340ms
      ✓ displays "Tour Guide not assigned yet" when booking has no assigned guide  323ms
      ✓ does not display guide info or unassigned note for cancelled booking  328ms

   Test Files  53 passed (53)
        Tests  471 passed (471)
     Start at  03:58:48
     Duration  53.65s (transform 14.31s, setup 43.57s, import 200.08s, tests 127.51s, environment 161.34s)

PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\frontend-web>
```

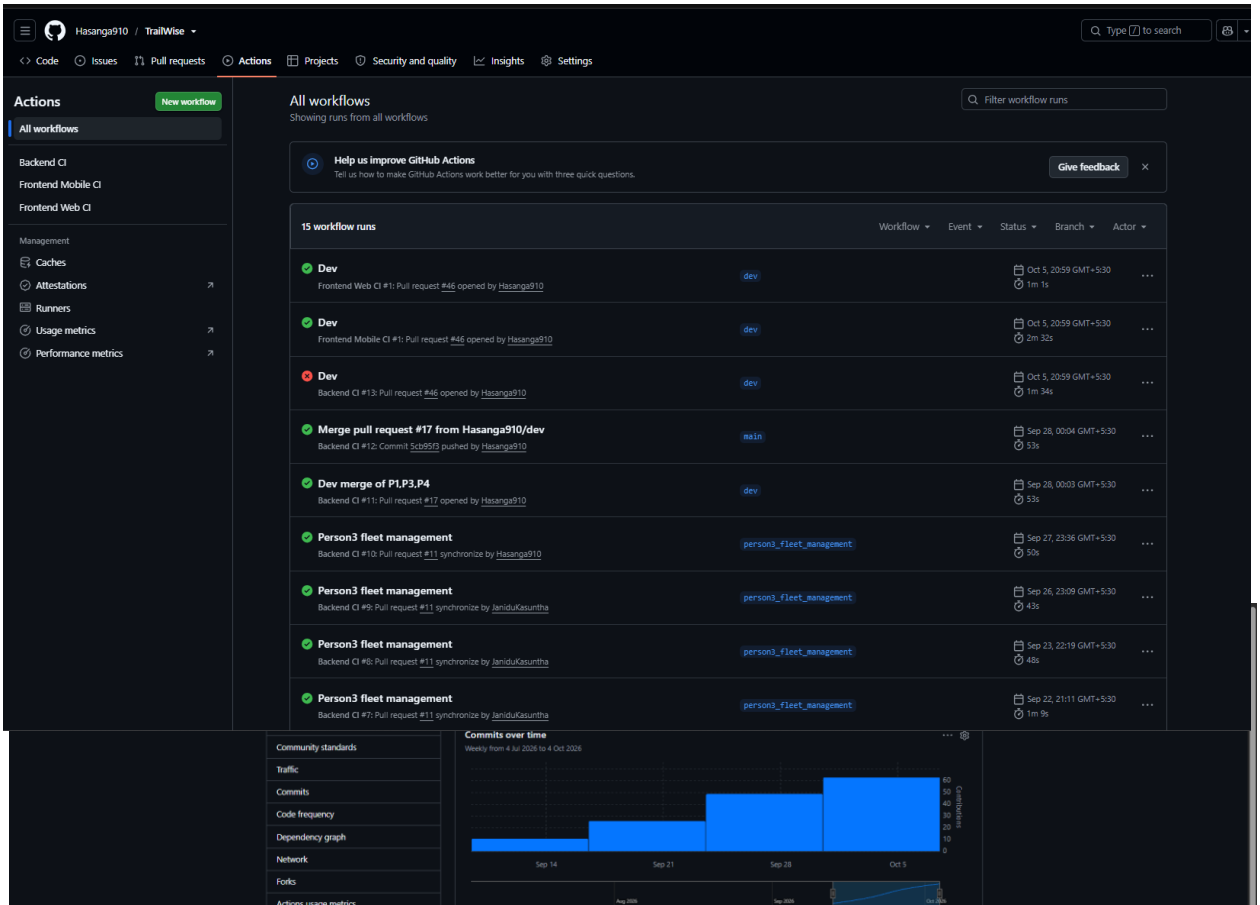
Figure B6 – Flutter automated test results using flutter_test

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\frontend-mobile> flutter test
Warning: A call to tap() with finder "Found 1 widget with text "More Info": [
  Text("More Info", dependencies: [DefaultSelectionStyle, DefaultTextStyle, MediaQuery]),
]" derived an Offset (Offset(240.7, 783.0)) that would not hit test on the specified widget.
Maybe the widget is actually off-screen, or another widget is obscuring it, or the widget cannot receive poin
ter events.
Indeed, Offset(240.7, 783.0) is outside the bounds of the root of the render tree, Size(360.0, 640.0).
The finder corresponds to this RenderBox: RenderParagraph#de09f relayoutBoundary=up27
The hit test result at that offset is: HitTestResult(HitTestEntry<HitTestTarget>#1670a(_ReusableRenderView#ee
895), HitTestEntry<HitTestTarget>#2a128(<AutomatedTestWidgetsFlutterBinding>))
#0      WidgetController._getElementPoint (package:flutter_test/src/controller.dart:2165:25)
#1      WidgetController.getCenter (package:flutter_test/src/controller.dart:1947:12)
#2      WidgetController.tap (package:flutter_test/src/controller.dart:1080:7)
#3      main.<anonymous closure> (file:///D:/SLIIT/Y3%20S1/SEF/Project/TrailWise/frontend-mobile/test/layout/
driver_support_layout_test.dart:122:20)
<asynchronous suspension>
#4      testWidgets.<anonymous closure>.<anonymous closure> (package:flutter_test/src/widget_tester.dart:192:
15)
<asynchronous suspension>
#5      TestWidgetsFlutterBinding._runTestBody (package:flutter_test/src/binding.dart:1953:5)
<asynchronous suspension>
#6      StackZoneSpecification._registerCallback.<anonymous closure> (package:stack_trace/src/stack_zone_spec
ification.dart:114:42)
<asynchronous suspension>
To silence this warning, pass "warnIfMissed: false" to "tap()".
To make this warning fatal, set WidgetController.hitTestWarningShouldBeFatal to true.

00:44 +309: All tests passed!
PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise\frontend-mobile>
```

Figure B7 – Successful GitHub Actions CI pipeline



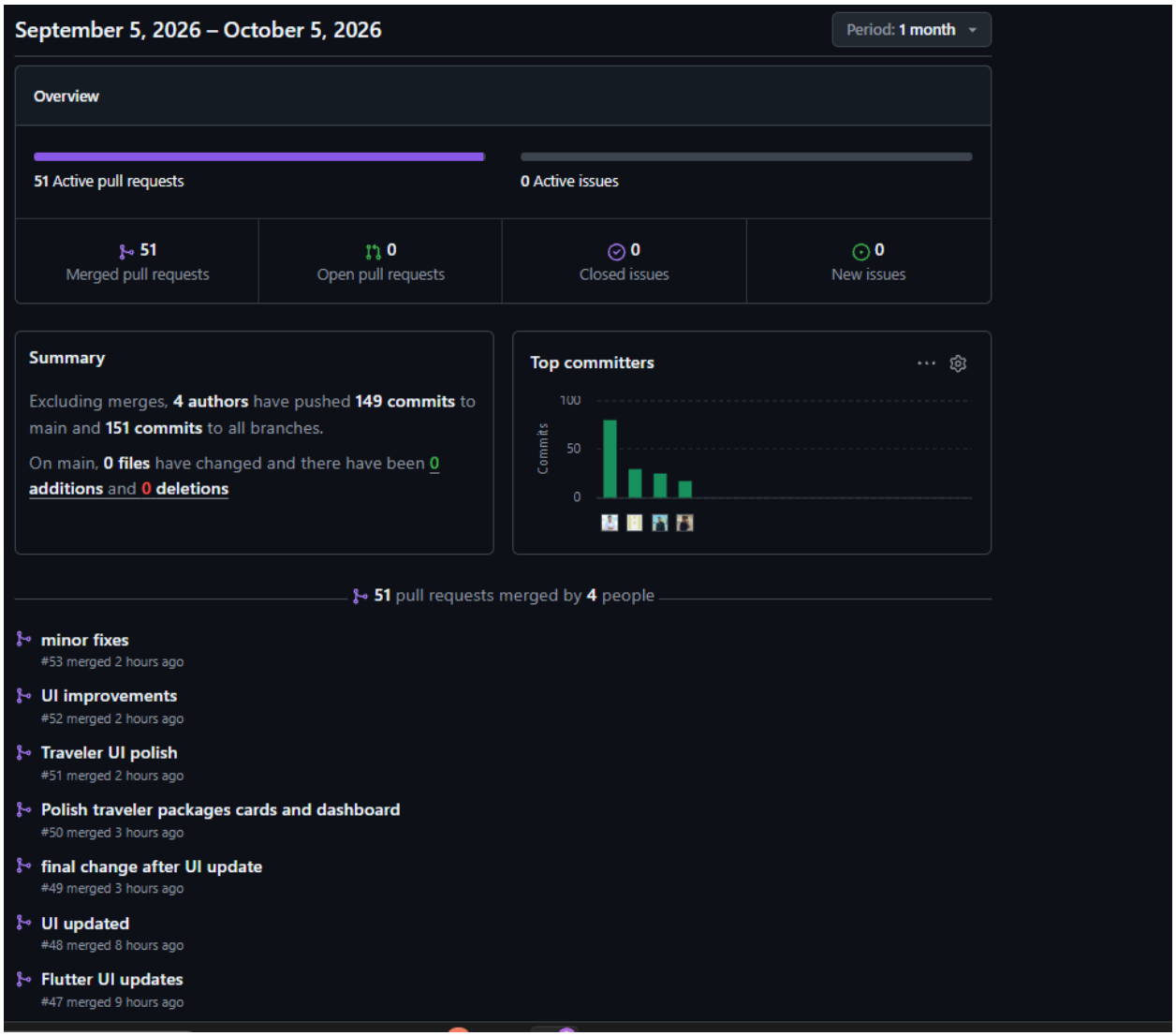


Figure B8 – TrailWise local API performance test results

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise> powershell -ExecutionPolicy Bypass -File .\performance_test.ps1
========================================
TrailWise Local API Performance Test
========================================

1. Testing GET /api/packages...
Request 1 : 123.22 ms - HTTP 200
Request 2 : 22.01 ms - HTTP 200
Request 3 : 21.97 ms - HTTP 200
Request 4 : 19.92 ms - HTTP 200
Request 5 : 17.35 ms - HTTP 200
Request 6 : 20.47 ms - HTTP 200
Request 7 : 19.16 ms - HTTP 200
Request 8 : 18.08 ms - HTTP 200
Request 9 : 18.12 ms - HTTP 200
Request 10 : 20.77 ms - HTTP 200

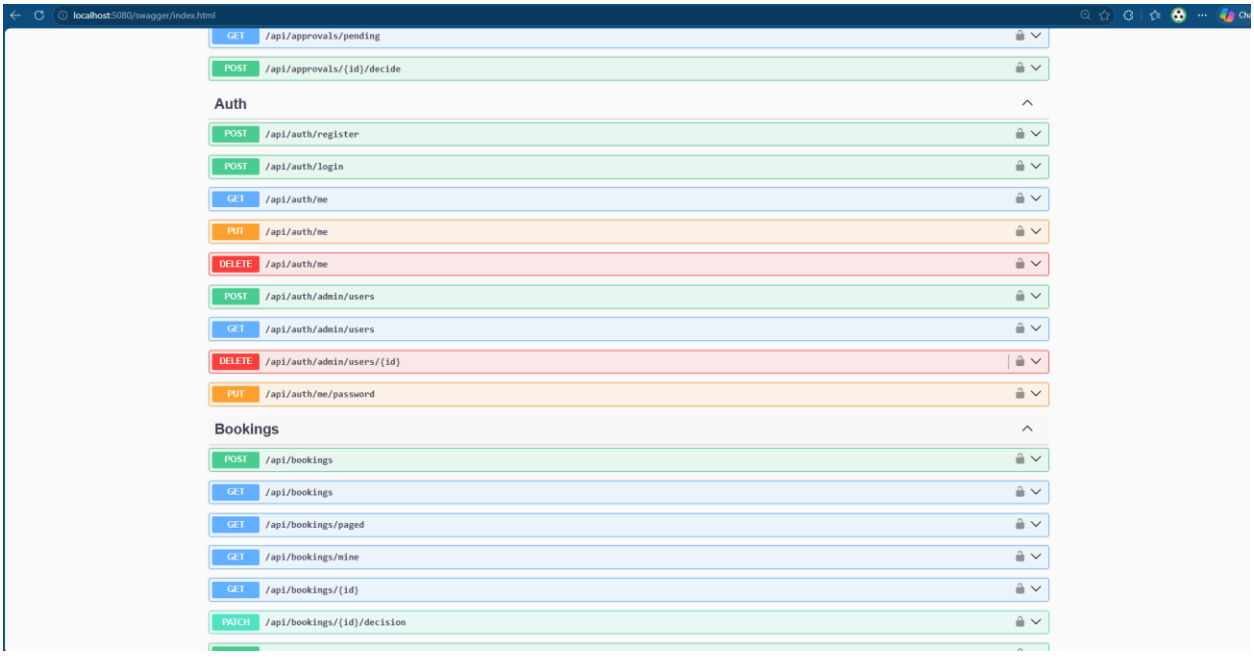
GET /api/packages Results
Average : 30.11 ms
p50     : 19.92 ms
p95     : 22.01 ms

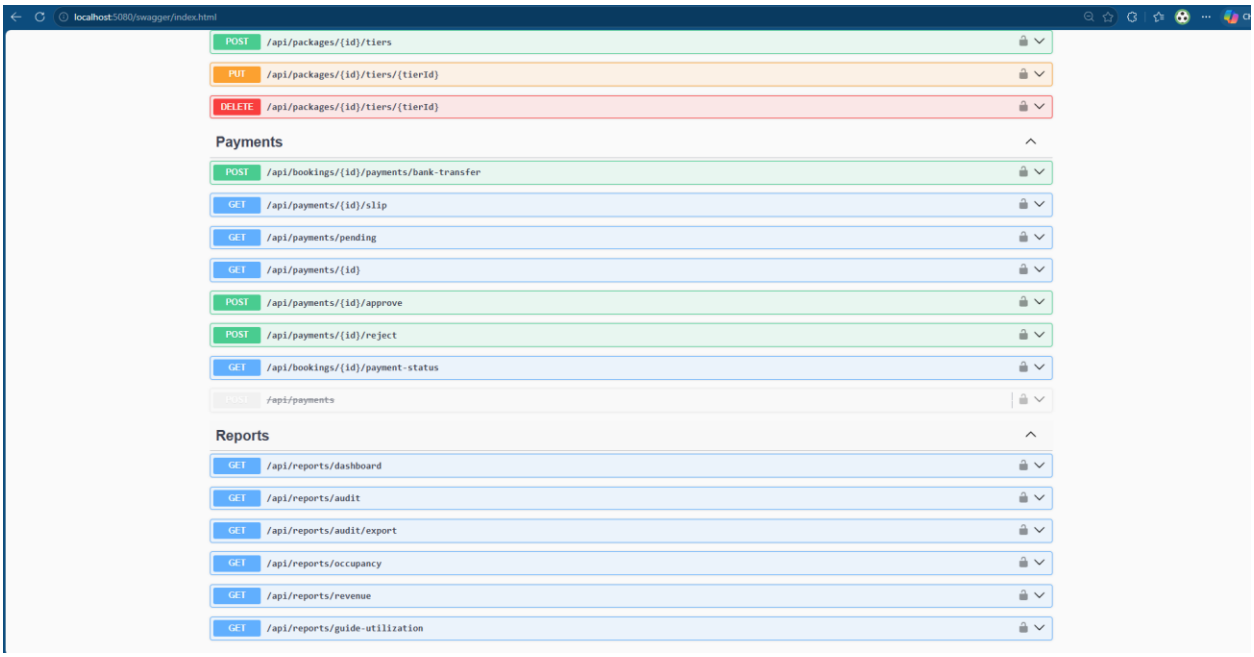
2. Testing POST /api/auth/login...
Login 1 : 205.61 ms - HTTP 401
Login 2 : 73.35 ms - HTTP 401
Login 3 : 89.3 ms - HTTP 401
Login 4 : 84.94 ms - HTTP 401

POST /api/auth/login Results
Average : 113.3 ms
p50     : 84.94 ms
p95     : 89.3 ms

========================================
Performance test completed
========================================
PS D:\SLIIT\Y3 S1\SEF\Project\TrailWise>
```

Figure B9 – TrailWise ASP.NET Core Swagger API documentation





Appendix C - Installation Quick Reference

See Section 12.3.

Appendix D - Git / Pull Request Evidence

Member	Commits	Pull requests	Reviewed files
Member 1	67872b2 Booking/package management; c764472 Traveler booking flow; fb27103 Coordinator/Planning Agent; 3115e76 P1 agent LLM integration; a82472e Approval queue API; 5f56c2c Refund exception approval; 4c7dae2 Workflow monitor/Ops dashboard; f03763d Agent tool-call recording	PR #1, #2, #3, #4, #5, #7, #10, #14, #18, #20	`PackagesController.cs`, `BookingsController.cs`, `ApprovalsController.cs`, `AgentWorkflowsController.cs`, `CoordinatorAgentService.cs`, `ApprovalService.cs`, `OpsApprovalsPage.tsx`, `OpsWorkflowsPage.tsx`, `OpsDashboardPage.tsx`
Member 2	610e09a Guide CRUD API; 2c5114c Guide availability API; d607e9c Assigned tours API; 6c9c493 Guide assigned tours screen; 514e310 Attendance/tour notes; 4ac268a Itinerary management; 056d0ad Guide Matching Agent; 547bec4	PR #21, #25, #26, #27, #28, #30, #36, #39, #41	`GuidesController.cs`, `BookingsController.cs`, `GuideAvailabilityService.cs`, `GuideAssignmentService.cs`, `ItineraryService.cs`, `GuideMatchingAgent.cs`, `IGuideMatchingAgent.cs`, `GuideDashboardPage.tsx`,

	Transactional guide assignment; 95b63cc Linked Guide profile; 7b31525 Guide Profile; 87da14d Tour Guide itinerary workflow		GuideAvailabilityPage.tsx, AssignedToursPage.tsx, TourDetailPage.tsx
Member 3	Add vehicle inventory dashboard; smart date-aware vehicle/guide availability; driver account provisioning; SMS integration	PR #21, #25, #26, #27, #28, #30, #36, #39, #41, #43	VehiclesController.cs, DriversController.cs, FleetReservationService.cs, Vehicle.cs, Driver.cs, VehicleAssignment.cs, FleetCapacityAgent.cs, FleetManagementPage.tsx, DriverDashboardPage.tsx, FleetManager.tsx
Member 4	Person 4 implementation commits for Pricing Validation Agent, payments, reviews, reports, discounts, audit logging	PR #33	PricingValidationAgent.cs, PaymentService.cs, ReviewService.cs, DiscountService.cs, AuditLogService.cs, OperationsReportService.cs, PaymentsController.cs, ReviewsController.cs, DiscountsController.cs, ReportsController.cs, payment_status_screen.dart, DiscountManager.tsx